

Pseudorandom Correlation Functions for Garbled Circuits

Geoffroy Couteau¹, Srinivas Devadas²,

Alexander Koch^{3*}, and Sacha Servan-Schreiber^{4**}

¹ Université Paris Cité, CNRS, IRIF

² MIT

³ Secorvo Security Consulting GmbH

⁴ Tinfoil

Abstract. In this paper, we define the notion of pseudorandom correlation generators (PCGs) and functions (PCFs) for garbled circuit correlations.

With a Garbling PCG or PCF, two parties can non-interactively generate a virtually unbounded number of secret-shared garbled circuits and corresponding secret-shared garbled inputs. With the shares of the garbled circuit and garbled input, anyone can recover the garbled circuit and evaluate it to obtain the result of the computation in the clear.

In the process of constructing Garbling PCFs, we introduce a new primitive that we call a *Topology-Adaptive* PCF (TAPCF), which we construct from two different variants of the learning parity with noise (LPN) assumption. Informally, a TAPCF is a PCF that additionally allows the target correlation to be specified on-demand (i.e., at evaluation time). As a contribution of independent interest, we show that TAPCFs enable the first silent secure computation protocol with function-dependent silent preprocessing. Using our TAPCF construction as a building block, we construct a Garbling PCF that allows the parties to specify the circuit they wish to garble on the fly. Under realistic parameter settings, we estimate that, with our construction, two parties can generate one garbled circuit per second, for circuits with 10,000 AND gates.

Garbling PCFs have several applications: We provide constructions for (1) an efficient homomorphic secret-sharing scheme for specific high-depth circuits, (2) a zero-knowledge proof system over secret shares that supports checking unstructured languages, and (3) a semi-honest reusable two-round, two-party computation protocol supporting non-interactive public outputs.

* Work done while the author was at Université Paris Cité, CNRS, IRIF

** Work done while the author was at MIT and visiting IRIF.

1	Introduction	3
1.1	Overview of applications	4
1.2	Organization	5
2	Technical Overview	6
2.1	A strawman approach	6
2.2	How to refresh a secret-shared garbled circuit with a PCG	8
2.3	How to rewire a secret-shared garbled circuit with a TAPCF	10
2.4	How to define a TAPCF	10
2.5	How to build a TAPCF	11
2.6	How to garble secret shared inputs	12
3	Preliminaries	13
3.1	Cryptographic assumptions	13
3.2	Pseudorandom correlation generators	13
3.3	Programmable PCGs	14
3.4	Pseudorandom correlation functions	15
3.5	Homomorphic secret sharing	17
3.6	Function secret sharing	19
4	Building Garbling PCGs	19
4.1	PCG for secret-shared garbling correlations	19
4.2	Secret-shared gate garbling	20
4.3	Garbling PCG construction	21
4.4	Concrete parameters and efficiency estimates	24
4.5	Practical optimizations	25
4.6	Reconfigurable garbled circuits via universal circuits	25
5	Topology-Adaptive PCFs	26
5.1	Definition	26
5.2	TAPCF construction	27
5.3	Silent secure computation with function-dependent preprocessing	29
6	Garbling PCF from a TAPCF	29
6.1	Concrete parameters and efficiency estimates	32
7	Secret-Shared Input Garbling	33
7.1	Secret-shared input garbling with HSS	35
8	Applications	37
8.1	Efficient HSS for T-shaped circuits	37
8.1.1	Applications of HSS for T-shaped circuits	39
8.2	Efficient zero-knowledge proofs for HSS	40
8.2.1	Application to verifying properties over secret-shared inputs	44
8.3	Non-interactive secure computation	44
8.3.1	Towards malicious security	46
A	Extended Preliminaries	59
A.1	Correlation-robust hash functions	59
B	Garbling and Secret-Sharing a Circuit	59
B.1	How to garble a circuit	59
B.2	How to secret-share a garbled circuit	60
C	Secret-Sharing-Friendly Garbled Circuits	62
D	PCG and PCFs with Scaled Outputs	64
E	Towards Authenticated Garbling	67
F	Deferred Proofs	68
F.1	Proof of Theorem 23	68
F.2	Proof of Proposition 29	69

1 Introduction

Cryptographic protocols often benefit from an access to a trusted source of correlated randomness. Traditional approaches for securely distributing correlated randomness are based on oblivious transfer extension [83] or somewhat homomorphic encryption [56], and induce a large communication overhead. To address this inefficiency, starting with the work of Boyle, Couteau, Gilboa, and Ishai [29], much research effort has been dedicated towards building primitives that allow parties to *silently* generate large amounts of correlated (pseudo)randomness from short seeds, i.e., without any interaction. These primitives are known as pseudorandom correlation generators (PCGs) [30, 33, 104] and pseudorandom correlation functions (PCFs) [34, 35, 51], and are designed for “simple” forms of correlated randomness. Specifically, the correlations considered in prior work target oblivious transfers and (vector) oblivious linear evaluations, which have already led to many applications, including efficient secure computation [37, 60, 82], threshold cryptography [2, 64, 90], private set intersection [39, 99, 102], zero-knowledge proofs [11–13, 61, 109, 110] and more.

Recently, primitives for silently generating correlated randomness were used to design improved protocols based on garbling schemes. A garbling scheme [16, 112] $\mathcal{GC}$ maps a Boolean circuit $\mathcal{C}$ to a *garbled circuit* $\hat{\mathcal{C}}$ and a set of labels $(L_{i,0}, L_{i,1})_{i \in [s]}$, such that, knowing $\hat{\mathcal{C}}$ and the labels L_{i,x_i} corresponding to an input $x = (x_1, \dots, x_s)$ allows anyone to recover $\mathcal{C}(x)$, without learning anything else about x . Garbling schemes are at the heart of secure computation protocols with minimal interaction [60, 84, 88, 107, 108], and efficient protocols for generating garbled circuits have both theoretical and practical applications. The recent work of Dittmer, Ishai, Lu, and Ostrovsky [60] (Eurocrypt 2022) showed that the silent generation of correlated randomness could be used to improve the distributed generation of garbled circuits. Inspired, in part, by their approach, and motivated by the many applications of minimally-interactive secure computation, we ask a related but different question: Is it possible to silently generate garbled circuits directly? In other words, we ask:

Can two parties non-interactively generate a large number of additive shares of pseudorandom garbled circuits from short seeds?

In this work, we answer the question in the affirmative by constructing PCGs and PCFs for garbling correlations. Towards answering this question, we initiate the study of efficient generation for pseudorandom *garbled circuit* correlations, where two parties obtain secret-shares of the garbled circuit and labels. Moreover, our constructions are potentially implementable and we estimate them to be concretely practical using existing building blocks.

Contributions. Our main contributions are threefold:

- **New Definitions.** We introduce the notion of *garbling pseudorandom correlation generators* (GPCGs) and *garbling pseudorandom correlation functions* (GPCFs). Intuitively, a GPCG is parametrized by a garbled circuit scheme $\mathcal{GC}$. Given GPCG keys (k^A, k^B) for a circuit $\mathcal{C}$, two parties can locally generate many pairs $(\hat{\mathcal{C}}_i^A, \hat{\mathcal{C}}_i^B)$ such that $\hat{\mathcal{C}}_i^A + \hat{\mathcal{C}}_i^B = \hat{\mathcal{C}}_i$ is a pseudorandom garbling of $\mathcal{C}$ under the garbling scheme $\mathcal{GC}$. A GPCF strengthens a GPCG on two fronts: (1) it allows generating the pairs $(\hat{\mathcal{C}}_i^A, \hat{\mathcal{C}}_i^B)$ on-demand, rather than expanding all correlations in one monolithic expansion, and (2) it allows the parties to *choose the circuit $\mathcal{C}_i$ to garble on the fly*. That is, the parties can generate pseudorandom shares $(\hat{\mathcal{C}}_i^A, \hat{\mathcal{C}}_i^B)$ of the garbled circuit $\hat{\mathcal{C}}_i$, encoding the circuit $\mathcal{C}_i$ chosen by the parties at evaluation time.
- **New Constructions.** We provide constructions of GPCGs and GPCFs, for a suitable *secret-sharing-friendly* garbling scheme, that is in essence a “BMR-style” multiparty garbling [15, 60, 107, 108]. Our construction of GPCG builds upon *programmable* PCGs for

Beaver triple correlations,⁵ for which efficient and practical [104] constructions are known from various flavors of the LPN assumption [21, 33]. Towards constructing GPCFs, we introduce the notion of *topology-adaptive* PCFs (TAPCFs), which are, informally, PCFs where the correlation can be specified by the parties on the fly, at evaluation time. We show that previous constructions of PCFs from LPN variants [34, 35, 51] can actually be directly cast as TAPCFs, uncovering a useful property that has so far not been exploited and is likely to be of independent interest. We build GPCFs for our secret-sharing-friendly garbling scheme GC from TAPCFs for degree-2 correlations.⁶

- **Applications.** We show that GPCGs and GPCFs, in combination with other cryptographic primitives, enable several new applications in the realm of low-interaction secure computation. We show that GPCGs and GPCFs:
 - (1) Result in a new framework for non-interactively computing over databases that are secret-shared between two servers, which, for certain types of queries, is much more practical compared to homomorphic secret sharing (HSS) [25, 27, 32, 98];
 - (2) Can be used to obtain *zero-knowledge proofs over secret-shared statements*, where two non-colluding designated verifiers can, given suitable shares of a witness, efficiently check the truth value of statement, in zero-knowledge; and
 - (3) Provide a new approach for semi-honest reusable, two-sided non-interactive secure computation (NISC) protocol [85] with a “non-interactive public disclosure” property.

In addition, we observe that TAPCFs enjoy applications of their own right; we describe one such important application to silent secure computation in Section 5.3. We provide an overview of our GPCG and GPCF applications in the following section.

1.1 Overview of applications

Here, we describe the three main applications for our constructions in more detail.

Efficient HSS for T-shaped circuits. Homomorphic secret sharing (HSS) [25] is a powerful distributed alternative to fully-homomorphic encryption, and has many uses in performing computations over secret-shared databases. However, existing HSS constructions only support *low-depth* computations, making them highly inefficient for computing complex functions over the database, or high-depth circuits more generally.

As a conceptual contribution, we identify a common computational pattern present in applications involving database queries: The computation can often be represented as a “T-shaped” circuit, where a low-degree pre-processing computation is applied to the entire database (the top of the T), followed by a high-depth post-processing computation over a small subset of items selected in the pre-processing phase (the leg of the T). This type of computation captures the case where queries first extract a number of matches from a database and *then* compute some potentially complex function on a selected subset of matches. Examples of such queries include threshold counting, fuzzy matching, queries to hash-based data structures, and more.

Existing HSS constructions require paying an *exponential* cost in the depth of the entire circuit. In contrast, we show that using our GPCG or GPCF construction, we can dramatically improve the efficiency of HSS applied to T-shaped computations. We achieve this by

⁵ Programmable PCGs enable partially specifying the correlation; see Section 3.3.

⁶ A degree-2 correlation consists of a triple $(a, b, P(a, b))$ where P is a polynomial of degree-2 in two variables. Beaver triples [14] are an example of a degree-2 correlation.

constructing a “compiler” that computes the low-degree pre-processing with the original HSS scheme, generates shares of a pseudorandom garbling $\widehat{\mathcal{C}}$ of the post-processing using a GPCF, and “connects” the outputs of the HSS to the secret-shared labels of $\widehat{\mathcal{C}}$. This yields a *new* type of HSS scheme that sacrifices additive output reconstruction, but confines the computational blowup to only being exponential in the depth of the *pre-processing* rather than the entire circuit. For database queries where a small subset of items is first selected and then a post-processing computation is applied over the selected items, our approach can exponentially improve the server-side efficiency over existing HSS techniques.

Zero-knowledge proofs for HSS. Zero-knowledge proofs on secret-shared data [22, 45, 48] provides a framework for parties to efficiently check properties over secret-shared data provided to them by an untrusted prover. While some existing techniques are practical [48] and generic (they can be applied to any P or NP statements), we identify two downsides that we address in the two-party context. Existing solutions are limited in that they:

- (1) require two sequential messages between the parties to check the proof (or, alternatively, require working in the random oracle model);
- (2) are tailored to languages that are efficiently verifiable using *arithmetic* circuits, which makes them cumbersome to use with Boolean circuits.

Using secret-shared garbled circuits, we construct a two-party proof system for HSS-shared statements (and witnesses) that (1) only requires sending one *message* between verifiers to check the proof, (2) works in the standard model (i.e., does not assume a random oracle), and (3) is tailored to languages with Boolean verification circuits. To the best of our knowledge, this results in the first zero-knowledge proof system for HSS without assuming a random oracle and supporting general verification predicates.

Semi-honest Two-party NISC with non-interactive disclosure. Non-interactive Secure Computation (NISC) allows two parties, after committing to their inputs, to securely evaluate multiple functions on their committed input, while using only a single additional round of communication per function. It is known that two-party NISC in the semi-honest model can be constructed from garbled circuits and oblivious transfers [84]. We show that our GPCF construction gives rise to a new construction of two-sided, two-party NISC [85] where, following a setup phase, two parties can commit to their respective inputs x and y in the first round, and later compute functions $f(x, y)$ in a single round, for any function f of their choice. We show that our protocol achieves a desirable “non-interactive disclosure” property: Two parties can compute $f(x, y)$ but also choose to reveal $f(x, y)$ to either of them or to any other party by making the result public in the second round.

1.2 Organization

In [Section 2](#), we provide a technical overview of our results and contributions, and describe the necessary preliminaries to PCGs and PCFs in [Section 3](#). Due to space limitations, foundations for generating “secret-sharing friendly” garbled circuits and our GPCG construction are deferred to [Appendices B and C](#). In [Section 4](#), we presents a construction of a *garbling pseudorandom correlation generator* (GPCG) using any suitable *pseudorandom correlation generator* (PCG) for degree-2 correlations [30, 33]. In [Section 5](#), we define and construct a *topology-adaptive pseudorandom correlation function* (TAPCF). In [Section 6](#) we construct our Garbling PCF using our TAPCF construction. In [Section 7](#), we show how efficiently encode a secret-shared input for a secret-shared circuit. Finally, in [Section 8](#), we describe the applications of our constructions.

2 Technical Overview

In this section, we give a technical overview of our results. We start by describing a natural strawman scheme that fails to meet the required security guarantees we are after but serves as an introduction to the technical notation and ideas. Afterwards, we show how to resolve the failures of the strawman scheme using degree-2 correlations which can be realized using a PCG. We then introduce the notion of a topology-adaptive PCF and overview the construction from the expand-accumulate LPN assumption [35].

2.1 A strawman approach

Let f be a function computed by a Boolean circuit $\mathcal{C}$ taking s bits as input. A garbled circuit $\hat{\mathcal{C}}$ is an encoding of the circuit $\mathcal{C}$ that can be efficiently evaluated given a garbled input $\hat{X}$ (encoding the input x) to reveal $\mathcal{C}(x)$. However, given $\hat{\mathcal{C}}$ and $\hat{X}$, no other information beyond $\mathcal{C}$ and $\mathcal{C}(x)$ is revealed to the evaluator.

In garbled circuit constructions, the garbled input $\hat{X}$, encoding $x = x_1 \parallel \dots \parallel x_s$, consists of s labels (which can be thought of as *keys*) that correspond to the input bits $x_1, \dots, x_s$. More concretely, the garbled circuit $\hat{\mathcal{C}}$ is defined with $2s$ input labels:

$$(L_{1,0}, L_{1,1}, \dots, L_{s,0}, L_{s,1}) \in (\mathbb{F}_{2^\lambda})^{2s},$$

which are used to encrypt the input bits to the circuit $\mathcal{C}$. The garbling of x consists of the labels $(L_{1,x_1}, \dots, L_{s,x_s})$ selected by the bits of x . At a high level, given the labels selected by x , the evaluator can decrypt the corresponding truth table entries for a given garbled input and proceed to get a new set of labels. This process is then repeated for all layers of the circuit until the last layer is reached. The final truth table outputs the value $\mathcal{C}(x)$. We provide a more in-depth overview of garbled circuits in [Appendix B](#).

Garbled circuits traditionally require one party (the garbler) to generate the garbled circuit and input labels. However, if two parties want to generate a garbled circuit and garble a secret-shared input, things become more challenging.

Suppose that the two parties holding additive shares of $x \in \{0, 1\}^s$ would like to produce secret shares of a garbled circuit $\hat{\mathcal{C}}$ and garbled input $\hat{X}$. Because the circuit $\mathcal{C}$ is itself known and independent of the input x , the parties can easily generate the garbled circuit $\hat{\mathcal{C}}$ “in the clear,” putting aside the fact that we would ultimately like to hide the labels of the circuit from both parties. However, even so, the parties still need a way to obtain secret shares of the labels that correspond to their secret-shared input x .

One way to achieve this trivially and non-interactively is by exploiting additive secret shares as follows. Denote the secret-shares of x held by the two parties A and B as $\langle x \rangle^A$ and $\langle x \rangle^B$, such that $\langle x \rangle^A \oplus \langle x \rangle^B = x$. Each share $\langle x \rangle^\sigma$ (for $\sigma \in \{A, B\}$) can be viewed as s shares $(\langle x_1 \rangle^\sigma, \dots, \langle x_s \rangle^\sigma)$ of the individual input bits. For each $i \in [s]$, both parties can obtain a secret share of L_{i,x_i} by computing:

$$\langle L_{i,x_i} \rangle^\sigma := L_{i,0} \cdot (\langle 1 \rangle^\sigma \oplus \langle x_i \rangle^\sigma) \oplus L_{i,1} \cdot \langle x_i \rangle^\sigma, \quad (1)$$

where $\cdot$ represents an element in $\mathbb{F}_{2^\lambda}$ multiplied by a scalar. Note that:

$$\langle L_{i,x_i} \rangle^A \oplus \langle L_{i,x_i} \rangle^B = L_{i,0} \cdot \underbrace{(1 \oplus \langle x_i \rangle^A \oplus \langle x_i \rangle^B)}_{= \neg x_i} \oplus L_{i,1} \cdot \underbrace{(\langle x_i \rangle^A \oplus \langle x_i \rangle^B)}_{= x_i} = L_{i,x_i}.$$

From this, the parties obtain *secret shares* of $\hat{X}$ (the garbled input), since they have secret-shares of the input labels L_{i,x_i} , for all $i \in [s]$. While at first glance it appears that we have

successfully constructed secret-shares of a garbling (by obtaining secret-shares of $\hat{X}$) this approach is unfortunately very limited.

Limitation of the strawman approach. In many cases, it is not sufficient to simply obtain secret-shares of $\hat{X}$. For example, consider the two-party computation application from [Section 1](#), where parties A and B have inputs a and b and would like to privately compute $\mathcal{C}(a, b)$ over their inputs. If they simply compute shares of $\hat{X}$ as in the strawman scheme, and swap their shares of $\hat{X}$ to learn $\hat{X}$, they would indeed be able to evaluate $\hat{\mathcal{C}}$ and learn $\mathcal{C}(a, b)$. However, this approach would be completely insecure: both parties *know the entire set of $2s$ labels* used to generate the garbling $\hat{X}$, which would reveal (a, b) to both parties! The problem here is that both parties have the entire set of input labels in the clear, which immediately reveals the inputs. While the strawman solution could work in a three-party setting, where the two parties send the resulting garbled circuit $\hat{\mathcal{C}}$ and shares of their garbled input $\hat{X}$ to a non-colluding third party, it does not naturally apply to other settings, and has limited applications.

With the above limitation in mind, we identify the following requirements for a secret-shared garbling scheme that works in a two-party setting.

Requirement I: A secret-shared garbled circuit. To effectively garble a secret-shared input in a way that would not reveal the input to either party (even in the presence of the garbled input $\hat{X}$), the labels need to be *secret-shared* between the two parties. Secret-sharing the labels ensures that neither party has access to the labels in the clear when generating the garbling. However, if the labels are secret-shared, then the garbled circuit $\hat{\mathcal{C}}$ *also* needs to be secret-shared. To wit, our first requirement is a way for the parties to non-interactively generate a secret-shared garbled circuit *and* select the labels corresponding to their secret-shared input *without* having the labels in the clear.

Requirement II: A way to generate fresh garblings. Even with a way to generate a secret-shared garbled circuit, the parties would only be able to use $\hat{\mathcal{C}}$ for *one* computation, after which they would need to regenerate a fresh garbled circuit $\hat{\mathcal{C}}$ (and the corresponding labels). While it was easy to do so in the strawman solution since the garbled circuit (and labels) were known by both parties, if the labels are secret-shared, then refreshing the labels becomes necessary. Thus, our second requirement is a way for the parties to generate “fresh” instances of the garbled circuit $\hat{\mathcal{C}}$ that can be used with new inputs.

With the above requirements laid out for generating secret shared garbled circuits, we identify the following two roadblocks in achieving them efficiently.

Challenge I: Efficiently refreshing garbled circuits. The second challenge is that it is unclear how to generate “new” secret shares of a garbled circuit efficiently (and non-interactively). While similar problems have been faced in prior work [\[43, 108\]](#), their solutions resort to interaction between parties to generate each garbled circuit. Therefore, at first glance, it may appear impossible to have a (fully non-interactive) way to refresh secret-shared garbled circuits, without resorting to heavy machinery (e.g., reusable garbled circuits [\[72\]](#)).

Challenge II: Secret-shared input garbling. The first challenge is that if the parties only have secret-shares of the input labels, then “selecting” the labels corresponding to the input x can no longer be done using [Eq. \(1\)](#), necessitating a different approach. (Computing the multiplication between two secret shared values would require interaction between parties, which we want to avoid.)

Organization of technical overview. In the rest of this overview, we focus on the first challenge since it is the more difficult one to resolve. In [Section 2.2](#) we overview our PCG-based approach and in [Sections 2.3 to 2.5](#) we describe our TAPCF-based approach. We then

overview a simple method for computing an input garbling using homomorphic secret sharing (HSS) in [Section 2.6](#), with an extended overview in [Appendix B](#) (a formal construction is presented in [Section 7.1](#)). This enables resolving the second challenge of connecting inputs to the pseudorandom garbled circuit instances.

2.2 How to refresh a secret-shared garbled circuit with a PCG

We now turn to explaining our first key idea and contribution: developing a PCG to generate many “disposable” secret-shared instances of the garbled circuit. Taking a step back, we will separate the random values that the parties can trivially generate themselves from the ones that require “structure” (and which we will generate using a PCG for a suitable correlation).

In modern garbled circuit schemes, labels are defined using the “free-xor” trick [\[89\]](#) which, roughly speaking, defines L_1 in terms of L_0 and a global offset Δ . In particular, $L_1 := L_0 \oplus \Delta$. Then, the i -th wire value in the garbled circuit is masked with a random bit $r_i \in \{0, 1\}$. Each AND gate taking as input wires i and j has an output wire mask $r_i \wedge r_j$ and output label $L_{k,0} \oplus (r_i \wedge r_j) \cdot \Delta$. The output wire masks are additionally scaled by the free-xor offset Δ^σ .

In a *secret-shared* garbled circuit [\[60, 107, 108\]](#), each party obtains its own global offset $\Delta^\sigma \in \{\Delta^A, \Delta^B\}$, which can be viewed as secret shares of an unknown Δ . We point the reader to [Appendices B](#) and [C](#) for additional background on garbled circuits and secret-sharing friendly constructions. We will denote secrets known to party A in [red](#) and those known to B in [blue](#).

Our first observation is that the parties can reuse the same free-xor offsets Δ^A and Δ^B each time they generate a new garbled circuit, since this value is independent of the circuit inputs and never revealed. Additionally, party $\sigma \in \{A, B\}$ can generate its labels $L_{i,0}^\sigma$, for all wires i , by sampling them locally. From that, each party can derive $L_{i,1}^\sigma := L_{i,0}^\sigma \oplus \Delta^\sigma$ using the sampled offset Δ^σ . The non-trivial part is generating shares of the i -th wire label and output mask. Specifically, the parties need to generate “fresh” masks r_i and r_j , and secret shares of

$$r_i, r_j, (r_i \wedge r_j), r_i \cdot \Delta^A, r_i \cdot \Delta^B, r_j \cdot \Delta^A, r_j \cdot \Delta^B, (r_i \wedge r_j) \cdot \Delta^A, (r_i \wedge r_j) \cdot \Delta^B,$$

which then allows them to compute shares of the output wire labels:

$$L_{k,r_i \wedge r_j} := L_{k,0} \oplus (r_i \wedge r_j) \cdot \Delta = L_{k,0}^A \oplus (r_i \wedge r_j) \cdot \Delta^A \oplus L_{k,0}^B \oplus (r_i \wedge r_j) \cdot \Delta^B.$$

Specifically, $(r_i \wedge r_j) \cdot \Delta^\sigma = \langle (r_i \wedge r_j) \cdot \Delta^\sigma \rangle^A \oplus \langle (r_i \wedge r_j) \cdot \Delta^\sigma \rangle^B$, for $\sigma \in \{A, B\}$, which allows the parties to derive secret shares of the above expression. Moreover, we can view the above equations as the output of a degree-2 polynomial P in two variables r_i, r_j with fixed, private global scaling constants Δ^A and Δ^B . It is known how to construct PCGs for degree-2 correlations [\[29, 30, 33, 34, 51, 104\]](#), which immediately makes it possible for the dealer to distribute short seeds s^A and s^B such that each party can locally apply the PCG to derive secret shares of $r_i, r_j, r_i \wedge r_j$. However, degree-2 correlations do not enable computing a share of the degree-3 expression with the global scaling constants. Fortunately, however, nearly all existing PCG constructions can be adapted to output shares scaled by a (secret) global constant at a negligible performance cost, which we formalize in [Appendix D](#). With that, in [Section 4](#), we show how to construct a garbling PCG for generating secret-shared garbled circuits, from the Ring-LPN assumption by plugging in either the practical PCG construction of Boyle et al. [\[33\]](#) or Schoppmann et al. [\[104\]](#).

Finally, the output wire mask is defined as a linear function of the next gate, as follows: Given $r_i, r_j, (r_i \wedge r_j)$, the set of four possible output wire masks (corresponding to the four possible inputs to the AND gate) are computed as:

$$\rho_{00} := (r_i \wedge r_j) \oplus r_k, \rho_{01} := \rho_{00} \oplus r_i, \rho_{10} := \rho_{00} \oplus r_j, \rho_{11} := \rho_{00} \oplus r_i \oplus r_j \oplus 1,$$

which are all linear functions of the shares held to both parties and the mask r_k of the next gate. More details on this are provided in [Appendix B.2](#).

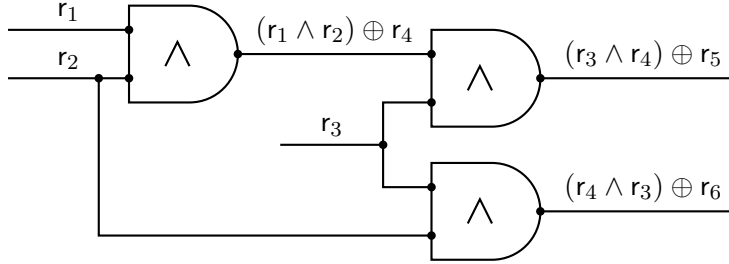


Fig. 1: Example circuit and mask values in the case where several wires are duplicated across multiple gates.

Dealing with duplicate wires. A subtle challenge we need to address is that the above method of secret sharing the garbled circuit implicitly assumes that no two gates in the circuit take the same inputs (i.e., that the circuit structure can be described as a binary tree). In the general case, circuits can have two or more gates that share the same input wire. A simple example is illustrated in [Fig. 1](#). If the PCG outputs random and independent masks for each gate, then the same wire would have different masks at the two input gates it is a part of, violating correctness. To address this, we require a *programmability* property from the PCG, which, at a high level, allows the dealer to distribute PCG seeds that share a common random mask r_i . In the example of [Fig. 1](#), the wire masks r_2 and r_3 are reused across two different gates. Therefore, the PCG seeds associated with those gates need to be defined so as to share the same seed, making the output consistent. In this way, we can “program” each wire mask into the gate PCG, so that all masks for the same wire are identical across gates, ensuring correctness.

Limitations of the PCG approach. With our Garbling PCG construction, the parties can efficiently generate many fresh instances of the same secret-shared garbled circuits that they can then use for computations involving different inputs. However, while the PCG approach is simple, it has some practical downsides:

- (1) PCGs, by definition, generate all the correlations in one monolithic expansion. This requires the parties to expand the PCG seed into *all* secret-shared random instances of the garbled circuit they will ever need to use. This is only practical if the total number of instances required is small.
- (2) Under the hood, our PCG-based construction for garbling correlations requires one PCG for degree-2 correlation *per* AND gate. This leads to a large concrete blowup in the size of the PCG seed given to the parties (e.g., a Garbling PCG for circuits with 10,000 AND gates has a seed size of 30 GB).
- (3) The circuit topology (i.e., the function computed by the garbled circuit) must be fixed at setup time. The parties cannot “rewire” the circuit gates on-demand since all instances of the garbled circuit output by the PCG must be identical. While this is sufficient for applications where the parties need to garble the same function repeatedly, it is not a versatile solution and has obvious limitations (e.g., it does not imply reusable two-party computation).

To address all three limitations simultaneously, we introduce the notion of a *topology-adaptive* PCF (TAPCF), which is likely to have applications outside of secret-shared garbled circuits

and is a natural object of study. In a nutshell, as already explained in [Section 1](#), a TAPCF allows all *correlations* to be specified *on-the-fly* and depend on common randomness when evaluating the PCF, which, among other things, makes it possible in our construction for the parties to specify the garbling correlations on-demand and generate any secret-shared garbled circuit of their choosing.

2.3 How to rewire a secret-shared garbled circuit with a TAPCF

Abstractly, a TAPCF is defined with respect to an (implicit) PRF F_K and allows the evaluators to efficiently compute secret shares of correlations of the form $P(F_K(a_1), \dots, F_K(a_m))$, where P is some correlation function and $(a_1, \dots, a_m)$ is a vector of PRF inputs. For example, if we choose P to be a degree-2 multiplication correlation, i.e., $(a, b) \mapsto (a, b, ab)$, computing $P(F_K(a_1), F_K(a_2))$ produces secret shares of a correlation with the form $(F_K(a_1), F_K(a_2), F_K(a_1) \cdot F_K(a_2))$ —i.e., a pseudorandom Beaver triple [14].

The power of the TAPCF abstraction is that the evaluators can specify which inputs to the function they want to compute correlations of *at evaluation time*. This stands in contrast to the standard PCF definition, where it is not possible to “mix-and-match” inputs to the correlation function P . Surprisingly, this powerful feature of TAPCFs can be obtained from a simple yet—to the best of our knowledge, overlooked—property implicit in several existing PCF constructions [34, 35], where the evaluation algorithm *already* allows specifying the correlation on-demand, even if the standard definition of PCFs does not explicitly require this to be the case! We believe that by shining a light on this surprising fact, and formally defining the notion of TAPCFs, new applications will emerge.

In [Section 6](#), we show that using a TAPCF, the parties can generate the necessary garbling correlations by letting P be a suitable degree-2 correlation, as described in [Section 2.2](#). At a high level, the TAPCF is used to generate pseudorandom masks for each wire in the circuit. Let r_i and r_j be two such masks for wires i and j , respectively. Then, for an AND gate that takes as input wires i and j , the TAPCF is used to generate a pseudorandom output mask r_k by specifying inputs i and j . Specifically, the parties compute shares of the degree-2 correlation $P(F_K(i), F_K(j))$, which generates shares of the output wire mask r_k . The TAPCF, guarantees that all gates that take as input wire i have an output mask generated as $P(F_K(i), F_K(\cdot))$, which, at a high level, guarantees consistency across all wire masks in the circuit. Using this idea, we obtain: (1) on-demand garbled circuit generation, (2) practical key sizes of around 1 GB that are essentially independent of the generated garbled circuit size, and (3) efficient “on-the-fly” garbling, allowing the parties to specify which circuit they wish to garble at evaluation time by appropriately specifying the wire topology.

2.4 How to define a TAPCF

The standard PCF definition specifies that each PCF evaluation appears indistinguishable to a fresh sample from the target correlation. Similarly, a TAPCF specifies that each evaluation should yield a random-looking sample from the target correlation, except that (1) some of the random coins used to generate one correlation can also be used to generate *other* correlations, and (2) the evaluators can specify a cross-sample correlation at evaluation time.

We provide a conceptually simple definition to capture this notion for correlations defined over Abelian groups, which automatically makes the target correlation reverse-sampleable. At a high level, a target correlation specified by a function $P: \mathbb{G}_2^m \rightarrow \mathbb{G}_3$ outputs *additive shares* (in the output group $\mathbb{G}_3$) of $P(F_K(a_1), \dots, F_K(a_m))$, for random inputs $(a_1, \dots, a_m) \in \mathbb{G}_1^m$, where $F_K: \mathbb{G}_1 \rightarrow \mathbb{G}_2$ is the implicit (weak) PRF.

Our definition of TAPCF is much closer in spirit to the traditional notion of pseudorandom functions: A TAPCF (TAPCF.Gen, TAPCF.Eval) is a pair of algorithm where TAPCF.Gen

generates a pair of keys (k^A, k^B) , and TAPCF.Eval takes as input a key, a function P specifying the correlation, and a set of inputs $(a_1, \dots, a_m)$. Correctness states that evaluation shares, denoted $y^\sigma \leftarrow \text{TAPCF.Eval}(\sigma, k^\sigma, P, (a_1, \dots, a_m))$, are computationally indistinguishable from shares (y^A, y^B) of $P(R(a_1), \dots, R(a_m))$, where R is a truly random function. Similarly, security states that even given $k^{\bar{\sigma}}$, evaluations $y^\sigma \leftarrow \text{TAPCF.Eval}(\sigma, k^\sigma, P, (a_1, \dots, a_m))$ are computationally indistinguishable from $P(R(a_1), \dots, R(a_m)) - y^{\bar{\sigma}}$. In particular, our notion of TAPCF recovers the standard definition PCFs *when no two inputs a_i are reused across different evaluations*. We find that our definition of TAPCFs sidesteps some of the complexities present in the standard PCF definition (e.g., the need for defining a reverse-sampler for the target correlation), while also making it clear that all PCFs must contain a (weak/strong) PRF “under the hood.”

2.5 How to build a TAPCF

A pseudorandom correlation function outputs pseudorandom samples of a correlation, where a typical choice of correlation deals shares of a low-degree polynomial. For example, a PCF for Beaver triples yields, on input i , shares of $(a_i, b_i, a_i \cdot b_i)$ for pseudorandom values a_i, b_i . However, in general, the definition of a PCF does not permit correlations *across* the samples generated using the PCF key (for example, it does not permit generating triples that share a common element, such as $(a_i, b_i, a_i \cdot b_i)$ and $(a_i, b_j, a_i \cdot b_j)$). The notion of *programmable* PCFs [35] does not capture this feature either: a programmable PCF allows to program *different* PCF keys such that evaluating both keys on the same input yield correlated outputs. However, that results in the cross-output correlation being fixed at key generation time, making the correlation hardcoded in the keys.

However, a closer look at existing constructions of PCFs reveals that they are much more flexible than the generic notion allows: they typically permit specifying, *on the fly*, a cross-output correlation between distinct PCF outputs produced by the same key!

The *general idea* for how we can achieve this cross-correlation property in existing PCF constructions is the following. At a high level, the PCF constructions of Boyle et al. [34, 35] can be made to share pseudorandom, degree- d monomials (in our Beaver triple example, $d = 2$), by first “virtually” generating a d -dimensional hypercube of monomials $\mathbf{a}_1 \otimes \mathbf{a}_2 \otimes \dots \otimes \mathbf{a}_d$, where $(\mathbf{a}_1, \dots, \mathbf{a}_d)$ are exponentially-long pseudorandom vectors, and “accessing” an entry $(i_1, \dots, i_d)$ of the hypercube using $O(c^d)$ evaluations of a suitable function secret sharing (FSS) scheme [24, 26], where c is a sparsity parameter that depends on the code used by the underlying LPN variant. Because chosen positions $(i_1, \dots, i_d)$ can be accessed on-demand, these schemes allow specifying, at evaluation time, which pseudorandom monomials should be used when generating the correlation.

To illustrate, we focus on degree-2 correlations (sufficient for our application), and use the construction of Boyle et al. [35] that is based on the expand-accumulate LPN (EA-LPN) assumption (Definition 2), which can be restated as follows.

EA-LPN assumption (intuition). Fix a length parameter N (typically very large, e.g., 2^{40}), a sparsity parameter c , and a noise parameter t (c, t are typically small, e.g., 80). Define the “threshold vector” as a vector of the form $(0, \dots, 0, 1, \dots, 1)$, where a length- ℓ threshold has the first “1” at some position i . Equivalently, we can view this vector as the truth-table of the greater-than predicate $g_i(x) = 0$, for $x < i$, and $g_i(x) = 1$, for $x \geq i$. Now, let $\mathbf{e}$ denote the vertical concatenation of t random threshold vectors of length $5N/t$, and let H be a random $N \times 5N$ sparse matrix, whose entries are Bernoulli samples with parameter $c/5N$ (i.e., H has c ones per row on average). Then, $\text{EALPN}(N, t, c)$ states that $H \cdot \mathbf{e}$ is computationally indistinguishable from a random vector.

TAPCF construction. To build a topology-adaptive PCF for degree-2 correlations, we sample two vectors $\mathbf{e}_a, \mathbf{e}_b$ as the concatenation of t random threshold vectors. Observe that under the EALPN assumption, $\mathbf{a} = H \cdot \mathbf{e}_a$ and $\mathbf{b} = H \cdot \mathbf{e}_b$ are pseudorandom. The 2-dimensional hypercube of monomials is then $\mathbf{a} \otimes \mathbf{b} = \mathbf{a} \cdot \mathbf{b}^\top = H \cdot (\mathbf{e}_a \cdot \mathbf{e}_b^\top) \cdot H^\top$. By construction, the matrix $\mathbf{e}_a \cdot \mathbf{e}_b^\top$ is divided into t^2 squares of side length $5N/t$, where each square is the truth-table of a 2-dimensional rectangle predicate (i.e., a predicate of the form $g_{i,j}(u, v) = 1$ if $u \geq i$ and $v \geq j$, and 0 else). Using FSS for 2-dimensional rectangle predicates [26] yields a way to provide “compressed” shares of such truth tables, where the shares of $g_{i,j}$ have size $O(\log^2(5N/t))$ when using the FSS scheme of Boyle et al. [26], and enable evaluating $g_{i,j}$ on any input x . Hence, by distributing t^2 FSS keys for 2-dimensional rectangle predicates, the parties can generate additive shares of any entry of the matrix $\mathbf{e}_a \cdot \mathbf{e}_b^\top$ from $O(t^2 \cdot \log^2(5N/t))$ -sized FSS keys. Now, since the rows of H are c -sparse (on average), computing shares of $a_i \cdot b_j$ (the (i, j) -th entry of the hypercube $\mathbf{a} \otimes \mathbf{b}$) can be done as follows. First, identify the c^2 entries of $\mathbf{e}_a \cdot \mathbf{e}_b^\top$ at all positions (u, v) where both the u -th entry of the i -th row of H and the v -th entry of the j -th row of H are equal to 1. Evaluate the FSS for 2-dimensional rectangles at all these positions, and sum the outcomes. See Fig. 2 for an illustration and Section 5 for the full construction.

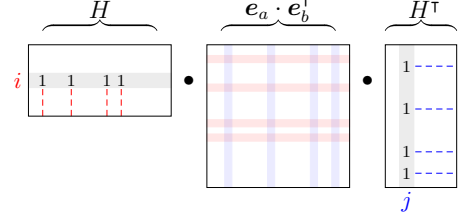


Fig. 2: Computation of the monomial $a_i \cdot b_j$ using the EALPN-based PCF of Boyle et al. [35], where $\mathbf{a} = H \cdot \mathbf{e}_a$ and $\mathbf{b} = H \cdot \mathbf{e}_b$. The c^2 position of $\mathbf{e}_a \cdot \mathbf{e}_b^\top$ at the intersection of the red and blue lines can be computed using FSS for 2-dimensional rectangle predicates.

2.6 How to garble secret shared inputs

At this point, we have two ways of producing pseudorandom secret shares of a pseudorandom garbled circuit. More concretely, the techniques described thus far produce a pair of offsets Δ^A and Δ^B and a set of input labels $L_{1,0}^A, \dots, L_{s,0}^A$ and $L_{1,0}^B, \dots, L_{s,0}^B$, which are distributed to each party. Additionally, both parties obtain secret shares of pseudorandom input masks $r_1, \dots, r_s$ (along with shares of the masks for each wire in the circuit, which we ignore for the time being).

The challenge now is having the parties obtain pseudorandom shares of their private inputs x and y . For the purposes of this overview, we focus on a single party (possibly an external third party) sharing their input x . The main idea is to have the trusted setup (which generates the PCG seeds or TAPCF keys) additionally distribute the fixed global offsets Δ^A and Δ^B to the two parties in the form of input shares under a public-key HSS scheme.

In a nutshell, HSS enables evaluating low-degree polynomials over input shares between two parties. Therefore, to secret share an input, each party (or an external party) computes an HSS input encoding of its private input $x = (x_1, \dots, x_s)$, which are also distributed to the two parties. Each party then holds an HSS input share Δ^A , Δ^B , and $(x_1, \dots, x_s)$.

The two parties then compute the degree-2 HSS program that computes $\Delta^A \cdot x_i$, $\Delta^B \cdot x_i$, which allows them to compute shares of the labels as follows. First, the parties compute shares of $x_i \cdot \Delta^A$ and $x_i \cdot \Delta^B$ using the HSS homomorphic evaluation. Second, the parties use shares of r , $r \cdot \Delta^A$, $r \cdot \Delta^B$ (which are part of the pseudorandom secret shared garbled circuit) to compute additive shares of

$$L_{i,\rho_i}^A := L_{i,0}^A \oplus (x_i \oplus r_i) \cdot \Delta^A \quad \text{and} \quad L_{i,\rho_i}^B := L_{i,0}^B \oplus (x_i \oplus r_i) \cdot \Delta^B.$$

These labels now correspond to the correct (masked) input bits, enabling the evaluation of the garbled circuit on input x . In particular, even though each party learns the value of the label (after recovering the share from the other party), it does not learn the input value x_i because it does not learn r_i (which masks the true value of the wire).

3 Preliminaries

Here, we describe notation and the background on pseudorandom correlations.

Notation. We let $\mathbb{N}$ denote the set of natural numbers and $\mathbb{F}_p$ denote a finite field of order p . We denote a vector as $\mathbf{v} = (v_1, \dots, v_n)$ and will sometimes write $(v_i)_{i=1}^n$ as a shorthand. We denote by $\text{poly}(\cdot)$ any polynomial and by $\text{negl}(\cdot)$ any negligible function. We let $x \leftarrow_R S$ denote a uniformly random sample drawn from S . We let $x \leftarrow \mathcal{A}$ denote assignment and $x := y$ denote initialization of x to the value of y . By an *efficient* algorithm $\mathcal{A}$ we mean that $\mathcal{A}$ is modeled by a (possibly non-uniform) Turing Machine that runs in probabilistic polynomial time. We write $D_0 \approx_c D_1$ to mean that two distributions D_0 and D_1 are *computationally* indistinguishable to all efficient distinguishers $\mathcal{D}$ and $D_0 \approx_s D_1$ to mean that D_0 and D_1 are *statistically* indistinguishable.

Secret sharing. We denote two-party secret shares of a value x as the tuple of shares $(\langle x \rangle^A, \langle x \rangle^B)$ such that $\langle x \rangle^A \oplus \langle x \rangle^B = x$.

3.1 Cryptographic assumptions

Definition 1 (Ring-LPN, verbatim from [33]). Let $\mathcal{R} = \mathbb{Z}_p[X]/F(X)$ for a prime p and a degree- N polynomial $F(X) \in \mathbb{Z}_p[X]$. For $t \in \mathbb{N}$ let $\mathcal{HW}_{\mathcal{R},t}$ be the distribution of “sparse polynomials” over $\mathcal{R}$ obtained by sampling t noise positions $\mathbf{A} \leftarrow [0, \dots, N]^t$ and t payloads $\mathbf{b} \leftarrow (\mathbb{Z}_p^*)^t$ uniformly at random, and outputting $e(X) := \sum_{j=0}^{t-1} \mathbf{b}[j] \cdot X^{\mathbf{A}[j]}$. For $\mathcal{R} = \mathcal{R}(\lambda)$, $m = m(\lambda)$, $t = t(\lambda)$, we say that the ring-LPN problem $\mathcal{R}\text{-LPN}_{\mathcal{R},m,t}$ is hard if for every non-uniform polynomial-time distinguisher $\mathcal{A}$, it holds that

$$\left| \Pr[\mathcal{A}((a^{(i)}, a^{(i)} \cdot e + f^{(i)})_{i=1}^m) = 1] - \Pr[\mathcal{A}((a^{(i)}, u^{(i)})_{i=1}^m) = 1] \right| \leq \text{negl}(\lambda),$$

where the probabilities are taken over $a^{(1)}, \dots, a^{(m)}, u^{(1)}, \dots, u^{(i)} \leftarrow_R \mathcal{R}$ and $e, f^{(1)}, \dots, f^{(m)} \leftarrow \mathcal{HW}_{\mathcal{R},t}$. Moreover, the regular ring-LPN problem $\mathcal{R}\text{-regLPN}_{m,t}$ is defined in the same way, except that it uses $\text{reg}\mathcal{HW}_{\mathcal{R},t}$, the distribution obtained by sampling the t noise positions $\mathbf{A} \leftarrow [0, \dots, N]^t$, such that each block of size N/t contains exactly one position. (Here, we assume that t divides N .)

Definition 2 (EALPN(N, t, c), cf. [35]). The expand-accumulate syndrome assumption with noise weight t , matrix sparsity c , and number of samples N , denoted $\text{EALPN}(N, t, c)$, states that

$$\left\{ (H, \mathbf{b}) \left| \begin{array}{l} H \leftarrow_R \mathcal{M}_{\text{ea}}(\text{pp}) \\ \mathbf{e} \leftarrow_R \mathcal{N}_{\text{ea}}(\text{pp}) \\ \mathbf{b} := H \cdot \mathbf{e} \end{array} \right. \right\} \approx_c \left\{ (H, \mathbf{b}) \left| \begin{array}{l} H \leftarrow_R \mathcal{M}_{\text{ea}}(\text{pp}) \\ \mathbf{b} \leftarrow_R \mathbb{F}_2^N \end{array} \right. \right\}.$$

3.2 Pseudorandom correlation generators

In this section, we provide the necessary background on pseudorandom correlation generators (PCGs). Our formalization closely follows prior work [30, 33].

Definition 3 (Correlation Generator). Let $(\text{Setup}, \mathcal{Y})$ be two probabilistic algorithms, where on input 1^λ , Setup generates a random pair $\mathbf{y}\mathbf{k} = (\mathbf{y}\mathbf{k}^A, \mathbf{y}\mathbf{k}^B)$. Moreover, $\mathcal{Y}$ takes as input the security parameter, a correlation key $\mathbf{y}\mathbf{k}$ and returns a pair $(y^A, y^B) \in \{0, 1\}^n \times \{0, 1\}^n$, defining a correlation on the outputs. Then, $(\text{Setup}, \mathcal{Y})$ is a correlation generator with setup. If Setup is trivial, we call it a correlation generator and omit the algorithm.

PCGs require the correlation to be “reverse sampleable” which, roughly speaking, means that given any (possibly adversarially generated) share of the target correlation, the other (honest) share can be efficiently sampled. In turn, this ensures the existence of an efficient simulator for producing the correct distribution of the target correlation.

Definition 4 (Reverse-Sampleable Correlation Generator). A correlation generator with setup $(\text{Setup}, \mathcal{Y})$ is said to be reverse-sampleable, if there exists an efficient algorithm RSample , such that for $\sigma, \bar{\sigma} \in \{A, B\}$ with $\sigma \neq \bar{\sigma}$ and for $\text{yk} = (\text{yk}^A, \text{yk}^B) \leftarrow \text{Setup}(1^\lambda)$, the pair

$$\left\{ (R'^A, R'^B) \mid (R^A, R^B) \leftarrow \mathcal{Y}(1^\lambda, \text{yk}), R'^\sigma := R^\sigma, R'^{\bar{\sigma}} \leftarrow \text{RSample}(1^\lambda, \sigma, \text{yk}^{\bar{\sigma}}, R^\sigma) \right\}$$

is computationally indistinguishable from $\mathcal{Y}(1^\lambda, \text{yk})$. In words, the correlation is reverse-sampleable if given one element of the pair, it is possible to efficiently sample the other element from a computationally-indistinguishable distribution.

Definition 5 (Pseudorandom Correlation Generator [30, 104]). Let λ be a security parameter and $(\text{Setup}, \mathcal{Y})$ be a reverse-sampleable correlation generator (with setup) with length parameter $n(\lambda)$. A Pseudorandom Correlation Generator (PCG) for $\mathcal{Y}$ is a pair of algorithms $(\text{PCG.Gen}, \text{PCG.Expand})$ with the following functionality:

- $\text{PCG.Gen}(1^\lambda, \text{yk}, \text{aux}) \rightarrow (\text{s}^A, \text{s}^B)$. The randomized seed generation algorithm takes as input the security parameter λ and (optionally) a correlation key yk , and (optional) auxiliary inputs aux . It outputs a pair of seeds (s^A, s^B) .
- $\text{PCG.Expand}(\sigma, \text{s}^\sigma) \rightarrow R^\sigma$. The deterministic expansion algorithm takes as input an index $\sigma \in \{A, B\}$ and a seed s^σ . It outputs a string $R^\sigma \in \{0, 1\}^n$.

The above functionality must satisfy correctness and security.

- **Correctness.** For all $\lambda \in \mathbb{N}$ and correlation key $\text{yk} \leftarrow \text{Setup}(1^\lambda)$ and aux , it holds that

$$\left\{ (R^A, R^B) \mid \begin{array}{l} (\text{s}^A, \text{s}^B) \leftarrow \text{Gen}(1^\lambda, \text{yk}, \text{aux}) \\ R^A \leftarrow \text{Expand}(A, \text{s}^A) \\ R^B \leftarrow \text{Expand}(B, \text{s}^B) \end{array} \right\} \approx_c \mathcal{Y}(1^\lambda, \text{yk}).$$

- **Security.** For $\bar{\sigma} \in \{A, B\}, \sigma \in \{A, B\} \setminus \{\bar{\sigma}\}$ and correlation key $\text{yk} = (\text{yk}^A, \text{yk}^B) \leftarrow \text{Setup}(1^\lambda)$, it holds that:

$$\left\{ (\text{s}^\sigma, R^{\bar{\sigma}}) \mid \begin{array}{l} (\text{s}^A, \text{s}^B) \leftarrow \text{Gen}(1^\lambda, \text{yk}, \text{aux}) \\ R^\sigma \leftarrow \text{Expand}(\sigma, \text{s}^\sigma) \\ R^{\bar{\sigma}} \leftarrow \text{RSample}(\sigma, \text{yk}^{\bar{\sigma}}, R^\sigma) \end{array} \right\} \approx_c \left\{ (\text{s}^\sigma, R^{\bar{\sigma}}) \mid \begin{array}{l} (\text{s}^A, \text{s}^B) \leftarrow \text{Gen}(1^\lambda, \text{yk}, \text{aux}) \\ R^{\bar{\sigma}} \leftarrow \text{Expand}(\bar{\sigma}, \text{s}^{\bar{\sigma}}) \end{array} \right\}$$

We will require the notion of a *programmable* PCG which, at a high level, allows one to specify part of the “seed” for one part of the output correlation. The definition is given as [Definition 7](#) in [Section 3.3](#).

3.3 Programmable PCGs

Following Boyle et al. [30], we define programmable PCGs for *bilinear* correlations, as it is the most natural setting for programmability.

Definition 6 (Simple Bilinear Correlation [30, Def. 10.2]). Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be Abelian groups and $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a bilinear map. Let $\mathcal{Y}_e$ be the efficient algorithm that on input 1^λ samples from $(\mathbb{G}_1 \times \mathbb{G}_T) \times (\mathbb{G}_2 \times \mathbb{G}_T)$ via:

$$R^A \leftarrow_R \mathbb{G}_1, \quad R^B \leftarrow_R \mathbb{G}_2, \quad S^A \leftarrow_R \mathbb{G}_T, \quad S^B := e(R^A, R^B) + S^A.$$

Definition 7 (Programmable PCG [30, 34]). Let λ be a security parameter and $n(\lambda) \in \text{poly}(\lambda)$. A PCG $\text{PCG} = (\text{PCG.Gen}, \text{PCG.Expand})$ is a programmable PCG for a simple bilinear correlation $\mathcal{Y}_e$ (cf. Definition 6), if PCG.Gen admits random inputs $\tau^0, \tau^1 \in \{0, 1\}^\lambda$ such that the following properties holds:

Correctness. For all $\lambda \in \mathbb{N}$, it holds that

$$\left\{ ((R^A, S^A), (R^B, S^B)) \left| \begin{array}{l} \tau^0, \tau^1 \leftarrow_R \{0, 1\}^\lambda \\ (s^A, s^B) \leftarrow \text{Gen}(1^\lambda, \tau^0, \tau^1, \text{aux}) \\ (R^A, S^A) \leftarrow \text{Expand}(A, s_A) \\ (R^B, S^B) \leftarrow \text{Expand}(B, s_B) \end{array} \right. \right\} \approx_c \mathcal{Y}_e(1^\lambda).$$

Programmability. There exist public the following efficiently computable deterministic functions $\varphi_A: \{0, 1\}^\lambda \rightarrow \mathbb{G}_1$, $\varphi_B: \{0, 1\}^\lambda \rightarrow \mathbb{G}_2$ such that

$$\Pr \left[R^A = \varphi_A(\tau^0) \wedge R^B = \varphi_B(\tau^1) : \begin{array}{l} \tau^0, \tau^1 \leftarrow_R \{0, 1\}^\lambda \\ (s^A, s^B) \leftarrow \text{Gen}(1^\lambda, \tau^0, \tau^1, \text{aux}) \\ (R^A, S^A) \leftarrow \text{Expand}(A, s_A) \\ (R^B, S^B) \leftarrow \text{Expand}(B, s_B) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Programmable security. For all $\lambda \in \mathbb{N}$, it holds that

$$\begin{aligned} (1) \quad & \left\{ (s^A, (\tau^0, \tau^1)) \left| \begin{array}{l} \tau^0, \tau^1 \leftarrow_R \{0, 1\}^\lambda \\ (s^A, s^B) \leftarrow \text{Gen}(1^\lambda, \tau^0, \tau^1, \text{aux}) \end{array} \right. \right\} \\ & \approx_c \left\{ (s^A, (\tau^0, \tau^1)) \left| \begin{array}{l} \tau^0, \tau^1, \tilde{\tau}^1 \leftarrow_R \{0, 1\}^\lambda \\ (s^A, s^B) \leftarrow \text{Gen}(1^\lambda, \tau^0, \tilde{\tau}^1, \text{aux}) \end{array} \right. \right\} \text{ and} \\ (2) \quad & \left\{ (s^B, (\tau^0, \tau^1)) \left| \begin{array}{l} \tau^0, \tau^1 \leftarrow_R \{0, 1\}^\lambda \\ (s^A, s^B) \leftarrow \text{Gen}(1^\lambda, \tau^0, \tau^1, \text{aux}) \end{array} \right. \right\} \\ & \approx_c \left\{ (s^B, (\tau^0, \tau^1)) \left| \begin{array}{l} \tau^0, \tau^1, \tilde{\tau}^0 \leftarrow_R \{0, 1\}^\lambda \\ (s^A, s^B) \leftarrow \text{Gen}(1^\lambda, \tilde{\tau}^0, \tau^1, \text{aux}) \end{array} \right. \right\}. \end{aligned}$$

3.4 Pseudorandom correlation functions

Here, we provide a slight generalization of the standard definition of PCFs. The standard definition requires the correlation of the PCF to be reverse sampleable, similarly to the definition of PCGs. This is a necessary technicality associated with the standard PCF (and PCG) definition, but abstracted away in our TAPCF definition. The generalization consists in allowing the correlation to take an additional public input in, with the corresponding changes to the pseudorandomness and security definitions (allowing the adversary to choose these inputs adaptively).

Definition 8 (Reverse-sampleable correlation with setup). Let λ be a security parameter and $n = n(\lambda) \in \text{poly}(\lambda)$ be an output length. Let $(\text{Setup}, \mathcal{Y})$ be two probabilistic algorithms, where on input 1^λ , Setup generates a random pair $\text{yk} = (\text{yk}^A, \text{yk}^B)$. Moreover, $\mathcal{Y}$ takes as input the security parameter, a correlation key yk and an input $\text{in} \in \{0, 1\}^*$, returns a

pair $(y^A, y^B) \in \{0, 1\}^n \times \{0, 1\}^n$, defining a correlation on the outputs. We say that $\mathcal{Y}$ defines a reverse-sampleable correlation with setup, if there exists a probabilistic polynomial time algorithm RSample that takes as input the security parameter, $\text{yk}^{\bar{\sigma}}$, $\text{in} \in \{0, 1\}^*$, $\sigma \in \{A, B\}$, and $y^\sigma \in \{0, 1\}^n$, and outputs $y^{\bar{\sigma}} \in \{0, 1\}^n$, such that for all $\text{in} \in \{0, 1\}^*$, all $\text{yk} = (\text{yk}^A, \text{yk}^B)$ in the image of Setup and all $\bar{\sigma} \in \{A, B\} \setminus \{\sigma\}$, it holds that:

$$\{(y^A, y^B) \mid (y^A, y^B) \leftarrow \mathcal{Y}(1^\lambda, \text{yk}, \text{in})\} \approx_s \left\{ (y^A, y^B) \left| \begin{array}{l} (y'^A, y'^B) \leftarrow \mathcal{Y}(1^\lambda, \text{yk}, \text{in}) \\ y^\sigma := y'^\sigma \\ y^{\bar{\sigma}} \leftarrow \text{RSample}(1^\lambda, \text{yk}^{\bar{\sigma}}, \text{in}, \sigma, y^\sigma) \end{array} \right. \right\}.$$

$\text{Exp}_{\mathcal{A}, N=N(\lambda), 0}^{\text{pr}}(\lambda):$ $\text{yk} \leftarrow \text{Setup}(1^\lambda)$ $(\text{st}_0, (x_i)_{i \in [N]}) \leftarrow \mathcal{A}(1^\lambda),$ s.t. $x_i \in \{0, 1\}^n$ are pairwise distinct $y_0^A := \perp, y_i^B := \perp$ for $i = 1$ to $N(\lambda):$ $(\text{in}_i, \text{st}_i) \leftarrow \mathcal{A}(1^\lambda, \text{st}_{i-1}, (y_{i-1}^A, y_{i-1}^B))$ $(y_i^A, y_i^B) \leftarrow \mathcal{Y}(1^\lambda, \text{yk}, \text{in}_i)$ $b' \leftarrow \mathcal{A}(1^\lambda, \text{st}_N)$ return b'	$\text{Exp}_{\mathcal{A}, N=N(\lambda), 1}^{\text{pr}}(\lambda):$ $\text{yk} \leftarrow \text{Setup}(1^\lambda)$ $(\text{st}_0, (x_i)_{i \in [N]}) \leftarrow \mathcal{A}(1^\lambda),$ s.t. $x_i \in \{0, 1\}^n$ are pairwise distinct $(k^A, k^B) \leftarrow \text{PCF.Gen}(1^\lambda, \text{yk})$ $y_0^A := \perp, y_i^B := \perp$ for $i = 1$ to $N(\lambda):$ $(\text{in}_i, \text{st}_i) \leftarrow \mathcal{A}(1^\lambda, \text{st}_{i-1}, (y_{i-1}^A, y_{i-1}^B))$ for $\sigma \in \{A, B\}:$ $y_i^\sigma \leftarrow \text{PCF.Eval}(\sigma, k^\sigma, \text{in}_i, x_i)$ $b' \leftarrow \mathcal{A}(1^\lambda, \text{st}_N)$ return b'
---	---

Fig. 3: (Strong) Pseudorandom $\mathcal{Y}$ -correlated outputs of a PCF. Here, valid adversaries choose the $x_i \in \{0, 1\}^n$ as nonces, i.e. pairwise distinct.

$\text{Exp}_{\mathcal{A}, N=N(\lambda), \sigma, \bar{\sigma}, 0}^{\text{sec}}(\lambda):$ $\text{yk} = (\text{yk}^A, \text{yk}^B) \leftarrow \text{Setup}(1^\lambda)$ $(k^A, k^B) \leftarrow \text{PCF.Gen}(1^\lambda, \text{yk})$ $(\text{st}_0, (x_i)_{i \in [N]}) \leftarrow \mathcal{A}(1^\lambda, \sigma, k^\sigma, \text{yk}^\sigma),$ s.t. $x_i \in \{0, 1\}^n$ are pairwise distinct $y_0^{\bar{\sigma}} := \perp$ for $i = 1$ to $N:$ $(\text{in}_i, \text{st}_i) \leftarrow \mathcal{A}(1^\lambda, \text{st}_{i-1}, y_{i-1}^{\bar{\sigma}})$ $y_i^\sigma \leftarrow \text{PCF.Eval}(\sigma, k^\sigma, \text{in}_i, x_i)$ $y_i^{\bar{\sigma}} \leftarrow \text{RSample}(1^\lambda, \text{yk}^{\bar{\sigma}}, \text{in}_i, \sigma, y_i^\sigma)$ $b' \leftarrow \mathcal{A}(1^\lambda, \text{st}_N)$ return b'	$\text{Exp}_{\mathcal{A}, N=N(\lambda), \sigma, \bar{\sigma}, 1}^{\text{sec}}(\lambda):$ $\text{yk} = (\text{yk}^A, \text{yk}^B) \leftarrow \text{Setup}(1^\lambda)$ $(k^A, k^B) \leftarrow \text{PCF.Gen}(1^\lambda, \text{yk})$ $(\text{st}_0, (x_i)_{i \in [N]}) \leftarrow \mathcal{A}(1^\lambda, \sigma, k^\sigma, \text{yk}^\sigma),$ s.t. $x_i \in \{0, 1\}^n$ are pairwise distinct $y_0^{\bar{\sigma}} := \perp$ for $i = 1$ to $N:$ $(\text{in}_i, \text{st}_i) \leftarrow \mathcal{A}(1^\lambda, \text{st}_{i-1}, y_{i-1}^{\bar{\sigma}})$ $y_i^{\bar{\sigma}} \leftarrow \text{PCF.Eval}(\bar{\sigma}, k^{\bar{\sigma}}, \text{in}_i, x_i)$ $b' \leftarrow \mathcal{A}(1^\lambda, \text{st}_N)$ return b'
--	--

Fig. 4: (Strong) Security of game for a PCF. Here, RSample is the algorithm for reverse sampling the correlation $\mathcal{Y}$, as defined in Definition 8. Moreover, valid adversaries choose the $x_i \in \{0, 1\}^n$ as nonces, i.e. pairwise distinct.

Definition 9 (Pseudorandom Correlation Function [34, 35]). Let λ be a security parameter, $\mathcal{Y}$ be a reverse-sampleable correlation with setup with output length $n(\lambda) \in \text{poly}(\lambda)$, and $\lambda \leq m = m(\lambda) \in \text{poly}(\lambda)$ be an input length. A pseudorandom correlation function (PCF) for $\mathcal{Y}$ is defined by a pair of algorithms $(\text{PCF.Gen}, \text{PCF.Eval})$ with the following functionality:

- $\text{PCF.Gen}(1^\lambda, \text{yk}) \rightarrow (k^A, k^B)$. The randomized generation algorithm takes as input the security parameter λ and the correlation key yk . It outputs a pair of keys (k^A, k^B) .

- $\text{PCF.Eval}(\sigma, k^\sigma, \text{in}, x) \rightarrow y^\sigma$. The deterministic evaluation algorithm takes as input $\sigma \in \{A, B\}$, a key k^σ , a correlation input $\text{in} \in \{0, 1\}^*$ and a (PCF) input string $x \in \{0, 1\}^m$. It outputs a string $y^\sigma \in \{0, 1\}^n$.

A (strong) PCF for the correlation $\mathcal{Y}$ must the following properties:

- **Correctness / Pseudorandom $\mathcal{Y}$ -correlated outputs.** For every $\sigma \in \{A, B\}$, all efficient (valid) adversaries $\mathcal{A}$, and all $N \in \text{poly}(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for all sufficiently large λ ,

$$\left| \Pr[\text{Exp}_{\mathcal{A}, N, 0}^{\text{pr}}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{A}, N, 1}^{\text{pr}}(\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\mathcal{A}, N, b}^{\text{pr}}(\lambda)$, for $b \in \{0, 1\}$, is as defined in Figure 3. In particular, the adversary is given access to $N(\lambda)$ samples.

- **Security.** For all distinct $\sigma, \bar{\sigma} \in \{A, B\}$, and all (valid) efficient adversaries $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all sufficiently large λ ,

$$\left| \Pr[\text{Exp}_{\mathcal{A}, N, \sigma, \bar{\sigma}, 0}^{\text{sec}}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{A}, N, \sigma, \bar{\sigma}, 1}^{\text{sec}}(\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\mathcal{A}, N, \sigma, \bar{\sigma}, b}^{\text{sec}}(\lambda)$, for $b \in \{0, 1\}$, is as defined in Figure 4 (again, with the adversary given $N(\lambda)$ samples).

Remark 10 (Strong PCFs in the standard model). Our PCF constructions assume that there is a mapping H that maps any PCF input a to a sample from $\text{Ber}_{c/2N}^{5N}$, i.e., a mapping to a random row of M (as specified above). To get a weak PCF, it suffices to let H be a sampler for $\text{Ber}_{c/2N}^{5N}$, which uses a as randomness; in the strong PCF setting, one can model H as a random oracle with outputs sampled from $\text{Ber}_{c/2N}^{5N}$. Alternatively, a recent work [38] showed how to instantiate H in the standard model using extremely lossy functions [113], at the cost of assuming a strengthened EALPN assumption. While we describe our constructions assuming a strong PCF to simplify notation, in practice, for our applications, it is enough for the parties to have a shared source of randomness to avoid this requirement.

3.5 Homomorphic secret sharing

Homomorphic secret sharing (HSS) [25] enables distributing secret shares of an input x between two (or more [59]) parties in a way that allows them to *non-interactively* compute some function over x . For the purposes of this work, we only require a degree-2 HSS scheme with perfect correctness, which can be instantiated from a range of assumptions [1].

Definition 11 (Homomorphic Secret Sharing [25]). Let λ be a security parameter and $\mathcal{R}$ be a finite ring. A homomorphic secret sharing (HSS) scheme for a class of programs $\mathcal{P}$ with input space $\mathcal{I} \subseteq \mathcal{R}$, consists of three efficient algorithms $\text{HSS} = (\text{Setup}, \text{Input}, \text{Eval})$ with the following syntax:

- $\text{Setup}(1^\lambda) \rightarrow (\text{pk}, (\text{ek}^A, \text{ek}^B))$. The randomized setup algorithm takes as input the security parameter λ and outputs a public key pk and a pair of evaluation keys $(\text{ek}^A, \text{ek}^B)$.
- $\text{Input}(\text{pk}, x) \rightarrow ([x]^A, [x]^B)$. The randomized input sharing algorithm takes as input the public key pk and an input value $x \in \mathcal{I}$. It outputs a pair of HSS input shares $([x]^A, [x]^B)$.
- $\text{Eval}(\text{ek}^\sigma, ([x_1]^\sigma, \dots, [x_n]^\sigma), P) \rightarrow \langle y \rangle^\sigma$. The deterministic evaluation algorithm takes as input an evaluation key ek^σ , a vector of n HSS input shares, and an n -input program $P \in \mathcal{P}$. It outputs an additive share of the output $y = P(x_1, \dots, x_n)$.

The above functionality must satisfy the following properties:

- **Correctness.** For all security parameters $\lambda \in \mathbb{N}$, any $n \in \mathbb{N}$, and any n -input program $P \in \mathcal{P}$ with input space $\mathcal{I} \subseteq \mathcal{R}$, it holds that for any $x_1, \dots, x_n \in \mathcal{I}$:

$$\Pr \left[\begin{array}{l} (pk, (ek^A, ek^B)) \leftarrow \text{Setup}(1^\lambda) \\ ([x_i]^A, [x_i]^B) \leftarrow \text{Input}(pk, x_i) \ \forall i \in [n] \\ \langle y \rangle^A \leftarrow \text{Eval}(A, ek^A, ([x_i]^A)_{i \in [n]}, P) \\ \langle y \rangle^B \leftarrow \text{Eval}(B, ek^B, ([x_i]^B)_{i \in [n]}, P) \\ y := \langle y \rangle^A + \langle y \rangle^B \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

- **Security.** For all efficient adversaries $\mathcal{A}$, and any $\sigma \in \{A, B\}$, there exists a negligible function $\text{negl}(\cdot)$ such that:

$$\Pr \left[\begin{array}{l} (x_0, x_1, st) \leftarrow \mathcal{A}(1^\lambda) \\ (pk, (ek^A, ek^B)) \leftarrow \text{Setup}(1^\lambda) \\ b \leftarrow_R \{0, 1\} \\ ([x_b]^A, [x_b]^B) \leftarrow \text{Input}(pk, x_b) \\ b' \leftarrow \mathcal{A}(st, pk, ek^\sigma, [x_b]^\sigma) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

HSS memory values. Let $\langle\langle x \rangle\rangle^\sigma$ denote an intermediate “memory share” of an HSS evaluation held by party σ . In existing HSS schemes (e.g., [25, 54, 59, 65]) memory shares can be added together and multiplied with an HSS input share to produce a new memory share.

HSS vector inputs. We will slightly abuse notation and write $\llbracket x \rrbracket^\sigma$ and $\langle\langle x \rangle\rangle^\sigma$ to denote $(\llbracket x_1 \rrbracket^\sigma, \dots, \llbracket x_n \rrbracket^\sigma)$ and $(\langle\langle x_1 \rangle\rangle^\sigma, \dots, \langle\langle x_n \rangle\rangle^\sigma)$, respectively.

Auxiliary algorithms. For convenience, we define the following auxiliary HSS algorithms, which are implicit in HSS constructions for branching programs (see e.g., Boyle et al. [25]) and help us to make the semantics simpler in some of our constructions. In particular, we define **HSS.Mult** that can either take as input two HSS input shares *or* one HSS input share and one HSS memory share, and produces a memory share of the product. **HSS.Mult** can be viewed as a minimal version of **HSS.Eval**. We also define **HSS.Convert** which facilitates converting between HSS input (resp. memory) shares to additive shares. We define **HSS.Eval*** to be the evaluation algorithm that outputs an HSS memory share instead of additive shares.

Definition 12 (Auxiliary HSS Algorithms). We define the following deterministic efficient algorithms (**HSS.Mult**, **HSS.Convert**, **HSS.Eval***, **HSS.Decode**) with the following syntax:

- **HSS.Mult**($ek^\sigma, \llbracket a \rrbracket^\sigma, \llbracket b \rrbracket^\sigma$ *or* $\langle\langle b \rangle\rangle^\sigma$) $\rightarrow \langle ab \rangle^\sigma$. The *mult* algorithm takes as input an HSS evaluation key ek^σ , an HSS input share $\llbracket a \rrbracket^\sigma$, and either an input share $\llbracket b \rrbracket^\sigma$ or a memory share $\langle\langle b \rangle\rangle^\sigma$. It outputs additive secret shares of the product ab .
- **HSS.Convert**($ek^\sigma, \llbracket x \rrbracket^\sigma$ *or* $\langle\langle x \rangle\rangle^\sigma$) $\rightarrow \langle\langle x \rangle\rangle^\sigma$ *or* $\langle x \rangle^\sigma$. The *conversion* algorithm takes as input an evaluation key ek^σ and either an input share $\llbracket x \rrbracket^\sigma$ or a memory share $\langle\langle x \rangle\rangle^\sigma$ of a value x . If given an input share, it outputs a memory share $\langle\langle x \rangle\rangle^\sigma$; otherwise, if given a memory share as input, it outputs an additive share $\langle x \rangle^\sigma$.
- **HSS.Eval***($ek^\sigma, (\llbracket x_1 \rrbracket^\sigma, \dots, \llbracket x_n \rrbracket^\sigma), P$) $\rightarrow \langle\langle y \rangle\rangle^\sigma$. The *special evaluation* algorithm takes as input an evaluation key ek^σ , a vector of n encoded HSS inputs, and an n -input program $P \in \mathcal{P}$. It outputs a memory share of the output $y = P(x_1, \dots, x_n)$.
- **HSS.Decode**($ek^A, ek^B, (\llbracket x \rrbracket^A, \llbracket x \rrbracket^B)$) $\rightarrow x$. The *decoding* algorithm takes as input the pair of HSS evaluation keys both input shares of a value x . It outputs the decoded value x .

3.6 Function secret sharing

Function secret sharing (FSS) is the “dual” notion of HSS. Rather than secret sharing an input that can be homomorphically evaluated on by two parties computing a public function over the secret input, FSS secret shares a *function* which can be evaluated on a public input. FSS is a core building block in existing PCG and PCF constructions.

Definition 13 (Function Secret Sharing (FSS) [24]). *Let $2 \leq p$ be an integer and $\mathcal{F} := \{f_i : \{0,1\}^\nu \rightarrow \mathbb{G}\}$ be a family of functions, where $\mathbb{G}$ is an Abelian group. A p -party FSS scheme for $\mathcal{F}$ consists of efficiently computable (possibly randomized) algorithms **Gen** and **Eval** with the following syntax:*

- **FSS.Gen** $(1^\lambda, f) \rightarrow (\kappa_1, \dots, \kappa_p)$. *The randomized key generation algorithm takes as input a security parameter 1^λ and function $f \in \mathcal{F}$. It outputs p evaluation keys $\kappa_1, \dots, \kappa_p$.*
- **FSS.Eval** $(\sigma, \kappa^\sigma, x) \rightarrow \langle y \rangle^\sigma$. *The deterministic evaluation algorithm takes as input a party index σ , an evaluation key κ^σ and $x \in \{0,1\}^\nu$. It outputs additive secret share $\langle y \rangle^\sigma$.*

The functionality must satisfy the following properties:

- **Correctness.** *A p -party FSS scheme is correct if for all $f \in \mathcal{F}$*

$$\Pr \left[(\kappa_1, \dots, \kappa_p) \leftarrow \text{FSS.Gen}(1^\lambda, f) : \text{FSS.Eval}(1, \kappa^1, x) + \dots + \text{FSS.Eval}(p, \kappa^p, x) = f(x) \right] = 1.$$

- **Security.** *A p -party FSS scheme $(\text{Gen}, \text{Eval})$ is secure if there exists an efficient simulator $\mathcal{S}$ such that for all subsets $I \subset [p]$ of indices, such that $|I| < p$, and D_I is the distribution over $\{\kappa^\sigma \mid \sigma \in I\}$, where κ^σ is sampled according to $\text{Gen}(1^\lambda, f)$, it holds that $D_I \approx_c \mathcal{S}(1^\lambda, I)$.*
- **Efficiency.** *A p -party FSS scheme is efficient if (1) each key κ^σ is at most $O(\lambda \cdot |\mathcal{F}|^\epsilon)$ in size, for any $\epsilon < 1$ (possibly dependent on ν).*

4 Building Garbling PCGs

In this section, we show how to construct a GPCG from any programmable PCG for degree-2 correlations supporting scalable outputs (see [Appendix D](#) for the formalization of scalable outputs—a standard property achieved by all existing PCG constructions).

4.1 PCG for secret-shared garbling correlations

In this section, we define and construct a Garbling PCG. Our construction can be realized from any *programmable* PCG ([Definition 7](#)) for degree-2 correlations satisfying [Definition 43](#).

Definition 14 (Garbling correlation). *Let λ be a security parameter and let*

$$\text{GC} = (\text{GC.SampLabels}_\sigma, \text{GC.Garble}, \text{GC.Encode}, \text{GC.Decode})$$

*be a secret-sharing-friendly garbled circuit scheme (as per [Definition 40](#)). Let $\text{Setup}(1^\lambda)$ output a randomly sampled $\text{yk} =: (\Delta^A, \Delta^B)$. Together with **Setup**, a probabilistic algorithm $\mathcal{Y} = \mathcal{Y}_{\text{GC}, \mathcal{C}}$ is a garbling correlation for **GC** with hard-wired circuit $\mathcal{C}$ if on input $(1^\lambda, \text{yk})$, it proceeds as follows:*

- 1: $\mathbf{L}^A \leftarrow \text{GC.SampLabels}_A(1^\lambda, \mathcal{C})$.
- 2: $\mathbf{L}^B \leftarrow \text{GC.SampLabels}_B(1^\lambda, \mathcal{C})$.
- 3: $(\widehat{\mathcal{C}}, \mathbf{r}) \leftarrow \text{GC.Garble}(1^\lambda, \mathcal{C}, \Delta^A, \mathbf{L}^A, \Delta^B, \mathbf{L}^B)$.
- 4: *Secret share* $\widehat{\mathcal{C}}, \mathbf{r}$ s.t. $\widehat{\mathcal{C}} = \langle \widehat{\mathcal{C}} \rangle^A \oplus \langle \widehat{\mathcal{C}} \rangle^B$, $\mathbf{r} = \langle \mathbf{r} \rangle^A \oplus \langle \mathbf{r} \rangle^B$.
- 5: *For* $\sigma \in \{A, B\}$, *secret share* $\mathbf{r}\Delta^\sigma$, s.t. $\mathbf{r}\Delta^\sigma = \langle \mathbf{r}\Delta^\sigma \rangle^A \oplus \langle \mathbf{r}\Delta^\sigma \rangle^B$.
- 6: *Set* $y^A = (\Delta^A, \mathbf{L}^A, \langle \widehat{\mathcal{C}} \rangle^A, \langle \mathbf{r} \rangle^A, \langle \mathbf{r}\Delta^A \rangle^A, \langle \mathbf{r}\Delta^B \rangle^A)$.
- 7: *Set* $y^B = (\Delta^B, \mathbf{L}^B, \langle \widehat{\mathcal{C}} \rangle^B, \langle \mathbf{r} \rangle^B, \langle \mathbf{r}\Delta^A \rangle^B, \langle \mathbf{r}\Delta^B \rangle^B)$.
- 8: *Output* (y^A, y^B) .

In the slightly more general case of an *input-dependent* garbling correlation with setup (cf. [Definition 26](#)), we show that this correlation is reverse-sampleable, see [Lemma 27](#).

Definition 15 (Garbling PCG). A Garbling PCG is a PCG ([Definition 5](#)) for the garbling correlations defined in [Definition 14](#).

Notation. To simplify syntax and notation in our construction, we define an algorithm $\text{PCG.Eval}^\bullet$ that simulates “on-demand” correlations by outputting the ℓ -th block from the full PCG output. Specifically,

- $\text{PCG.Eval}^\bullet(\sigma, \mathbf{s}^\sigma, \ell) \rightarrow \langle c_\ell \rangle^\sigma$. Takes as input a party identifier σ , seed $\mathbf{s}^\sigma$, and index ℓ . Outputs the ℓ -th instance of the correlation.

$\text{PCG.Eval}^\bullet$ can be constructed by first running PCG.Expand and then outputting the ℓ -th correlation, and shares the same interface as PCF.Eval . (Conversely, PCG.Expand can be constructed by running $\text{PCG.Eval}^\bullet$ on each ℓ .)

4.2 Secret-shared gate garbling

Before we describe our Garbling PCG construction, we first provide a construction for garbling a single AND gate using secret-shares of appropriately correlated randomness. This will greatly simplify the construction of our GPCG in the next subsection.

Proposition 16 (Correctness). *Construction 1 satisfies the correctness property required of secret-shared gate garbling.*

Proof. At a high level, [Construction 1](#) generates *secret shares* of a garbled gate from [Construction 9](#). To see this, note that $(\langle g_{00} \rangle^\sigma, \langle g_{01} \rangle^\sigma, \langle g_{10} \rangle^\sigma, \langle g_{11} \rangle^\sigma)$ are secret shares of the “double encryption” of the output mask and labels:

$$\begin{aligned}
\langle g_{00} \rangle^A \oplus \langle g_{00} \rangle^B &= \mathbf{H}(L_{i,0}^A \| L_{j,0}^A \| k \| 0) \oplus \mathbf{H}(L_{i,0}^B \| L_{j,0}^B \| k \| 0) \oplus (\rho_{00} \| L_{k,\rho_{00}}^A \| L_{k,\rho_{00}}^B), \\
\langle g_{01} \rangle^A \oplus \langle g_{01} \rangle^B &= \mathbf{H}(L_{i,0}^A \| L_{j,1}^A \| k \| 1) \oplus \mathbf{H}(L_{i,0}^B \| L_{j,1}^B \| k \| 1) \oplus (\rho_{01} \| L_{k,\rho_{01}}^A \| L_{k,\rho_{01}}^B), \\
\langle g_{10} \rangle^A \oplus \langle g_{10} \rangle^B &= \mathbf{H}(L_{i,1}^A \| L_{j,0}^A \| k \| 2) \oplus \mathbf{H}(L_{i,1}^B \| L_{j,0}^B \| k \| 2) \oplus (\rho_{10} \| L_{k,\rho_{10}}^A \| L_{k,\rho_{10}}^B), \\
\langle g_{11} \rangle^A \oplus \langle g_{11} \rangle^B &= \mathbf{H}(L_{i,1}^A \| L_{j,1}^A \| k \| 3) \oplus \mathbf{H}(L_{i,1}^B \| L_{j,1}^B \| k \| 3) \oplus (\rho_{11} \| L_{k,\rho_{11}}^A \| L_{k,\rho_{11}}^B).
\end{aligned}$$

With some simple case analysis (see [Footnote 12](#) of [Appendix B.2](#)), it is possible to verify that $\rho_{00} = (r_i \wedge r_j) \oplus r_k$, $\rho_{01} = (r_i \wedge \neg r_j) \oplus r_k$, $\rho_{10} = (\neg r_i \wedge r_j) \oplus r_k$, and $\rho_{11} = (\neg r_i \wedge \neg r_j) \oplus r_k$.

It is then easy to see that $L_{k,\rho}^A = L_{k,0}^A \oplus \rho \Delta^A$ and $L_{k,\rho}^B = L_{k,0}^B \oplus \rho \Delta^B$, for all $\rho \in \{\rho_{00}, \rho_{01}, \rho_{10}, \rho_{11}\}$. It then follows that given $(L_{i,\alpha}^A, L_{j,\beta}^A)$ and $(L_{i,\alpha}^B, L_{j,\beta}^B)$, corresponding to wire values α and β , it is possible to recover $(\rho_{\alpha\beta} \| L_{k,\rho_{\alpha\beta}}^A \| L_{k,\rho_{\alpha\beta}}^B)$. This matches the requirements of [Construction 9](#) (secret-shared garbled circuits) in [Appendix C](#). Correctness then immediately follows from the correctness of [Construction 9](#). $\square$

Construction 1 (Secret-shared Gate Garbling).

Parameters. Let $H : \{0, 1\}^* \rightarrow \{0, 1\} \times \mathbb{F}_{2^\lambda}$ be a suitable correlation-robust hash function. Let $(\langle 1 \rangle^A, \langle 1 \rangle^B)$ be fixed secret-shares of $1 \in \mathbb{F}_2$. Let $\bar{\sigma} := \{A, B\} \setminus \{\sigma\}$ denote the other party's identifier.

GarbleGate $(\sigma, k, \text{labels}, \text{masks}, \langle r_k \rangle^\sigma, \langle r_k \Delta^A \rangle^\sigma, \langle r_k \Delta^B \rangle^\sigma, \Delta^\sigma)$:

```

1 : parse labels =  $(L_{i,0}^\sigma, L_{j,0}^\sigma, L_{k,0}^\sigma)$ ,
      masks =  $(\langle r_i \rangle^\sigma, \langle r_j \rangle^\sigma, \langle r_i r_j \rangle^\sigma, \langle r_i \Delta^A \rangle^\sigma, \langle r_j \Delta^A \rangle^\sigma, \langle (r_i r_j) \Delta^A \rangle^\sigma, \langle r_i \Delta^B \rangle^\sigma, \langle r_j \Delta^B \rangle^\sigma, \langle (r_i r_j) \Delta^B \rangle^\sigma)$ 
2 :  $\langle \rho_{00} \rangle^\sigma := \langle r_i r_j \rangle^\sigma \oplus \langle r_k \rangle^\sigma$   $\triangleright \rho_{00} = (r_i \wedge r_j) \oplus r_k$ 
3 :  $\langle \rho_{00}^A \rangle^\sigma := \langle (r_i r_j) \Delta^A \rangle^\sigma \oplus \langle r_k \Delta^A \rangle^\sigma$ 
4 :  $\langle \rho_{00}^B \rangle^\sigma := \langle (r_i r_j) \Delta^B \rangle^\sigma \oplus \langle r_k \Delta^B \rangle^\sigma$ 
5 :  $\langle \rho_{01} \rangle^\sigma := \langle \rho_{00} \rangle^\sigma \oplus \langle r_i \rangle^\sigma$   $\triangleright \rho_{01} = (r_i \wedge \neg r_j) \oplus r_k$ 
6 :  $\langle \rho_{01}^A \rangle^\sigma := \langle \rho_{00}^A \rangle^\sigma \oplus \langle r_j \Delta^A \rangle^\sigma$ 
7 :  $\langle \rho_{01}^B \rangle^\sigma := \langle \rho_{00}^B \rangle^\sigma \oplus \langle r_j \Delta^B \rangle^\sigma$ 
8 :  $\langle \rho_{10} \rangle^\sigma := \langle \rho_{00} \rangle^\sigma \oplus \langle r_j \rangle^\sigma$   $\triangleright \rho_{10} = (\neg r_i \wedge r_j) \oplus r_k$ 
9 :  $\langle \rho_{10}^A \rangle^\sigma := \langle \rho_{00}^A \rangle^\sigma \oplus \langle r_i \Delta^A \rangle^\sigma$ 
10 :  $\langle \rho_{10}^B \rangle^\sigma := \langle \rho_{00}^B \rangle^\sigma \oplus \langle r_i \Delta^B \rangle^\sigma$ 
11 :  $\langle \rho_{11} \rangle^\sigma := \langle \rho_{00} \rangle^\sigma \oplus \langle r_i \rangle^\sigma \oplus \langle r_j \rangle^\sigma \oplus \langle 1 \rangle^\sigma$   $\triangleright \rho_{11} = (\neg r_i \wedge \neg r_j) \oplus r_k$ 
12 :  $\langle \rho_{11}^A \rangle^\sigma := \langle \rho_{00}^A \rangle^\sigma \oplus \langle r_i \Delta^A \rangle^\sigma \oplus \langle r_j \Delta^A \rangle^\sigma \oplus \Delta^\sigma$ 
13 :  $\langle \rho_{11}^B \rangle^\sigma := \langle \rho_{00}^B \rangle^\sigma \oplus \langle r_i \Delta^B \rangle^\sigma \oplus \langle r_j \Delta^B \rangle^\sigma$ 
14 :  $L_{i,1}^\sigma := L_{i,0}^\sigma \oplus \Delta^\sigma, \quad L_{j,1}^\sigma := L_{j,0}^\sigma \oplus \Delta^\sigma, \quad L_{k,1}^\sigma := L_{k,0}^\sigma \oplus \Delta^\sigma$ 
15 :  $\langle L_{k,\rho_{00}}^\sigma \rangle^\sigma := L_{k,0}^\sigma \oplus \langle \rho_{00} \rangle^\sigma, \quad \langle \bar{L}_{k,\rho_{00}}^\sigma \rangle^\sigma := \langle \bar{\rho}_{00} \rangle^\sigma$ 
16 :  $\langle L_{k,\rho_{01}}^\sigma \rangle^\sigma := L_{k,0}^\sigma \oplus \langle \rho_{01} \rangle^\sigma, \quad \langle \bar{L}_{k,\rho_{01}}^\sigma \rangle^\sigma := \langle \bar{\rho}_{01} \rangle^\sigma$ 
17 :  $\langle L_{k,\rho_{10}}^\sigma \rangle^\sigma := L_{k,0}^\sigma \oplus \langle \rho_{10} \rangle^\sigma, \quad \langle \bar{L}_{k,\rho_{10}}^\sigma \rangle^\sigma := \langle \bar{\rho}_{10} \rangle^\sigma$ 
18 :  $\langle L_{k,\rho_{11}}^\sigma \rangle^\sigma := L_{k,0}^\sigma \oplus \langle \rho_{11} \rangle^\sigma, \quad \langle \bar{L}_{k,\rho_{11}}^\sigma \rangle^\sigma := \langle \bar{\rho}_{11} \rangle^\sigma$ 
19 :  $\langle g_{00} \rangle^\sigma \leftarrow H(L_{i,0}^\sigma \| L_{j,0}^\sigma \| k \| 0) \oplus (\langle \rho_{00} \rangle^\sigma \| \langle L_{k,\rho_{00}}^A \rangle^\sigma \| \langle L_{k,\rho_{00}}^B \rangle^\sigma)$ 
20 :  $\langle g_{01} \rangle^\sigma \leftarrow H(L_{i,0}^\sigma \| L_{j,1}^\sigma \| k \| 1) \oplus (\langle \rho_{01} \rangle^\sigma \| \langle L_{k,\rho_{01}}^A \rangle^\sigma \| \langle L_{k,\rho_{01}}^B \rangle^\sigma)$ 
21 :  $\langle g_{10} \rangle^\sigma \leftarrow H(L_{i,1}^\sigma \| L_{j,0}^\sigma \| k \| 2) \oplus (\langle \rho_{10} \rangle^\sigma \| \langle L_{k,\rho_{10}}^A \rangle^\sigma \| \langle L_{k,\rho_{10}}^B \rangle^\sigma)$ 
22 :  $\langle g_{11} \rangle^\sigma \leftarrow H(L_{i,1}^\sigma \| L_{j,1}^\sigma \| k \| 3) \oplus (\langle \rho_{11} \rangle^\sigma \| \langle L_{k,\rho_{11}}^A \rangle^\sigma \| \langle L_{k,\rho_{11}}^B \rangle^\sigma)$ 
23 : return  $(\langle g_{00} \rangle^\sigma, \langle g_{01} \rangle^\sigma, \langle g_{10} \rangle^\sigma, \langle g_{11} \rangle^\sigma)$ 

```

Construction 1: Secret-shared gate garbling construction

4.3 Garbling PCG construction

We provide the construction of our Garbling PCG in [Construction 2](#). It uses secret-shared gate garbling ([Construction 1](#)) as a subroutine to garble each gate in the circuit.

Proposition 17. *Under the Ring-LPN assumption, [Construction 2](#) satisfies the correctness and security properties required of a Garbling PCG ([Definition 15](#)).*

Proof. We need to show correctness and security, which we prove in turn.

Correctness. To see correctness, note that for each AND gate, $\text{PCG.Eval}^\bullet$ outputs the necessary mask relations required for **GarbleGate**. Then, it follows from [Proposition 16](#) (correctness of **GarbleGate**) and the correctness of the PCG, that the resulting secret-shared garbling of each gate are correct.

Next, consider the output $y^\sigma = (\Delta^\sigma, L^\sigma, \langle \hat{C} \rangle^\sigma, (\langle r_1 \rangle^\sigma, \dots, \langle r_s \rangle^\sigma), \langle r \Delta^A \rangle^\sigma, \langle r \Delta^B \rangle^\sigma)$ for $\sigma \in \{A, B\}$ obtained by evaluating GPCG.Eval , using seeds s^A and s^B that were sampled for a

Construction 2 (Garbling PCG).

Parameters. Let $\text{PCG} = (\text{PCG.Gen}, \text{PCG.Expand})$ be a *programmable* PCG for degree-2 correlations over $\mathbb{F}_2$, satisfying scalable outputs (Definition 43) with two scalars Δ^A, Δ^B (given as input to GPCG.Gen), and outputting degree-2 correlations of the form: $(a, b, c, a\Delta^A, b\Delta^A, c\Delta^A, a\Delta^B, b\Delta^B, c\Delta^B)$, with $c = ab$. Let the input wires of an s -input circuit $\mathcal{C}$ be indexed by $1 \dots s$. Finally, let $\text{PRF} = (\text{PRF.KeyGen}, \text{PRF.Eval})$ be a PRF with outputs in $\mathbb{F}_{2^\lambda}$, which we will use for generating the pseudorandom wire labels $L_{i,0}^\sigma \in \mathbb{F}_{2^\lambda}$.

$\text{GPCG.Gen}(1^\lambda, \text{yk}, \mathcal{C})$:

```

1 : parse  $\text{yk} = (\Delta^A, \Delta^B)$ 
2 : foreach wire  $i \in \mathcal{C}$ 
3 :    $\triangleright$  Note: two output wires from the same gate are identical.
4 :    $\tau_{r_i} \leftarrow_R \{0, 1\}^\lambda$   $\triangleright$  Sample a seed for wire mask correlations.
5 : foreach AND gate  $G_k \in \mathcal{C}$  with input wires  $(i, j)$ 
6 :    $(s_k^A, s_k^B) \leftarrow \text{PCG.Gen}(1^\lambda, \tau_{r_i}, \tau_{r_j}, \Delta^A, \Delta^B)$ 
7 :  $K^A \leftarrow \text{PRF.KeyGen}(1^\lambda), K^B \leftarrow \text{PRF.KeyGen}(1^\lambda)$ 
8 :  $s^A \leftarrow \left( \{s_k^A \mid \text{for all AND gates } G_k \in \mathcal{C}\}, \Delta^A, \mathcal{C}, K^A \right)$ 
9 :  $s^B \leftarrow \left( \{s_k^B \mid \text{for all AND gates } G_k \in \mathcal{C}\}, \Delta^B, \mathcal{C}, K^B \right)$ 
10 : return  $(s^A, s^B)$ 

```

$\text{GPCG.Eval}^\bullet(\sigma, s^\sigma, \ell)$:

```

1 : parse  $s^\sigma = (\{s_k^\sigma \mid \text{for all AND gates } G_k \in \mathcal{C}\}, \Delta^\sigma, \mathcal{C}, K^\sigma)$ 
2 : foreach wire  $i \in \mathcal{C}$ 
3 :   if wire  $i$  is an output wire of  $\mathcal{C}$  then  $\langle r_i \rangle^\sigma := 0$ 
4 :    $L^\sigma \leftarrow \text{SampLabels}_\sigma(1^\lambda, \mathcal{C}; \text{PRF.Eval}(K^\sigma, \ell))$ 
5 : Let  $S = \emptyset$   $\triangleright$  Set to collect input masks with  $\Delta$ -factors
6 : foreach AND gate  $G_k =: (i, j, k) \in \mathcal{C}$   $\triangleright$  Expand all necessary correlations
7 :    $\text{masks}_k \leftarrow \text{PCG.Eval}^\bullet(\sigma, s_k^\sigma, \ell)$ , where  $\text{masks}_k =$ 
      $\left( \langle r_i \rangle^\sigma, \langle r_j \rangle^\sigma, \langle r_i r_j \rangle^\sigma, \langle r_i \Delta^A \rangle^\sigma, \langle r_j \Delta^A \rangle^\sigma, \langle (r_i r_j) \Delta^A \rangle^\sigma, \langle r_i \Delta^B \rangle^\sigma, \langle r_j \Delta^B \rangle^\sigma, \langle (r_i r_j) \Delta^B \rangle^\sigma \right)$ 
8 :   if  $i$  is an input wire:  $S := S \cup \{(i, \langle r_i \Delta^A \rangle^\sigma, \langle r_i \Delta^B \rangle^\sigma)\}$ 
9 :   if  $j$  is an input wire:  $S := S \cup \{(j, \langle r_j \Delta^A \rangle^\sigma, \langle r_j \Delta^B \rangle^\sigma)\}$ 
10 : foreach AND gate  $G_k =: (i, j, k) \in \mathcal{C}$   $\triangleright$  Compute garbling using expanded correlations
11 :    $\text{labels} := (L_{i,0}^\sigma, L_{j,0}^\sigma, L_{k,0}^\sigma)$ 
12 :    $\langle \hat{G}_k \rangle^\sigma \leftarrow \text{GarbleGate}(k, \text{labels}, \text{masks}_k, \langle r_k \rangle^\sigma, \langle r_k \Delta^A \rangle^\sigma, \langle r_k \Delta^B \rangle^\sigma, \Delta^\sigma)$ 
13 : Set  $\langle \mathbf{r} \Delta^A \rangle^\sigma := (\langle r_i \Delta^A \rangle^\sigma)_{i \in [s]}, \langle \mathbf{r} \Delta^B \rangle^\sigma := (\langle r_i \Delta^B \rangle^\sigma)_{i \in [s]}$  with entries from  $S$ 
14 : return  $\left( \Delta^\sigma, L^\sigma, (\langle \hat{G}_k \rangle^\sigma)_{k \in [\nu]}, (\langle r_1 \rangle^\sigma, \dots, \langle r_s \rangle^\sigma), \langle \mathbf{r} \Delta^A \rangle^\sigma, \langle \mathbf{r} \Delta^B \rangle^\sigma \right)$ 

```

Construction 2: Garbling PCG construction.

fixed circuit $\mathcal{C}$. Expanding the shares $(\langle \hat{\mathcal{C}} \rangle^A, \langle \hat{\mathcal{C}} \rangle^B)$, we have that for $\sigma \in \{A, B\}$:

$$\langle \hat{\mathcal{C}} \rangle^\sigma = (\langle \hat{G}_1 \rangle^\sigma, \dots, \langle \hat{G}_\nu \rangle^\sigma), \quad L^\sigma = (L_{1,0}^\sigma, \dots, L_{s,0}^\sigma), \quad \langle \mathbf{r} \rangle^\sigma = (\langle r_1 \rangle^\sigma, \dots, \langle r_s \rangle^\sigma), \langle \mathbf{r} \Delta^A \rangle^\sigma, \langle \mathbf{r} \Delta^B \rangle^\sigma$$

where ν is the number of AND gates in $\mathcal{C}$. Now, if we reconstruct, we get

$$\hat{\mathcal{C}} := (\hat{G}_1, \dots, \hat{G}_\nu), \quad \mathbf{L}^A, \mathbf{L}^B, \quad \mathbf{r} = (r_1, \dots, r_s), \mathbf{r} \Delta^A, \mathbf{r} \Delta^B$$

which is *almost* the identical to the normal garbling except that there the construction outputs *two* sets of labels

$$(\textcolor{red}{L}_{1,0}^A, \textcolor{red}{L}_{1,1}^A, \dots, \textcolor{red}{L}_{s,0}^A, \textcolor{red}{L}_{s,1}^A, L_{1,0}^B, L_{1,1}^B, \dots, L_{s,0}^B, L_{s,1}^B).$$

However, by noticing this output is equivalent to outputting the *shares* of the some implicit label $L_{i,0} := \textcolor{red}{L}_{i,0}^A \oplus L_{i,0}^B$, and by, furthermore ignoring the shares of $L_{i,1}$ (which we do not need), we obtain correctness. The wire masks $\mathbf{r}$ are scaled by Δ^A and Δ^B to enable garbled gate evaluation by maintaining the required invariants (see [Appendix B.2](#)).

We note that similar minor syntactic issues appear in previous works on secret sharing and pseudorandom correlation generators, since it is notationally inconvenient to define a “trivial sharing” where the pseudorandom secret share *itself* is the output.

Finally, note that for all output wires indexed by some i , the corresponding mask r_i share is set to zero (which is a trivial “share” of zero). In turn, we have that all output wires have *no* mask, which guarantees that the output wires reveal the result, as required for evaluation correctness (see [Remark 41](#)).

Security. We prove security via a hybrid argument. Let ν be the total number of AND gates in the circuit $\mathcal{C}$. Define the set $\{\mathbf{s}_\ell^\sigma \mid \ell \in [\nu]\}$ to have a canonical ordering and where $\mathbf{s}_\ell^\sigma$ is sampled according to [Construction 2](#). Without loss of generality, we assume that all AND gates in $\mathcal{C}$ take a unique pair of input wires (which avoids dealing with duplicate copies of $\mathbf{s}_\ell^\sigma$). Additionally, notice that we can divide the generation of $\mathbf{s}_i^A$ into two types of PCG seeds:

Type I. For all pairs $(\mathbf{s}_i^A, \mathbf{s}_j^A)$ for $i \neq j$, (τ_i^0, τ_i^1) and (τ_j^0, τ_j^1) are generated uniformly at random and are independent of each other. This case corresponds to AND gates that do not share input wires with any other AND gates in $\mathcal{C}$.

Type II. There exist at least one pair of $(\mathbf{s}_i^A, \mathbf{s}_j^A)$ for $i \neq j$ such that $\{\tau_i^0, \tau_i^1\} \cap \{\tau_j^0, \tau_j^1\} \neq \emptyset$ (i.e., they have one programmed correlation in common). This case corresponds to AND gates that share input wires with at least one other AND gate in the circuit $\mathcal{C}$.

We now proceed with the hybrid argument:

- *Hybrid $\mathcal{H}_0$.* This hybrid is defined as the view of party A in [Construction 2](#) (the hybrid can be defined symmetrically for party B), where the view consists of both the PCG seed and the programmed correlations. We define

$$\mathcal{H}_0 := \{(\hat{\mathbf{s}}_1^A, (\tau_1^0, \tau_1^1)), \dots, (\hat{\mathbf{s}}_\nu^A, (\tau_\nu^0, \tau_\nu^1))\},$$

where $\hat{\mathbf{s}}_i^A \leftarrow \text{PCG.Gen}(1^\lambda, \tau_i^0, \tau_i^1)$, for all $i \in [\nu]$.

- *Hybrid $\mathcal{H}_1$.* In this hybrid, we replace each τ_i^1 with a random $\tilde{\tau}_i^1$ for all $\mathbf{s}_i^A$ of Type I (i.e., there does not exist another $\mathbf{s}_j^A$ that depends on either τ_i^0 or τ_i^1 , for all $i \neq j$). We define

$$\mathcal{H}_1 := \{(\hat{\mathbf{s}}_1^A, (\tau_1^0, \tilde{\tau}_1^1)), \dots, (\hat{\mathbf{s}}_\nu^A, (\tau_\nu^0, \tilde{\tau}_\nu^1))\},$$

where $\tilde{\tau}_i^1 \leftarrow_R \{0, 1\}^\lambda$ for all indices i where $\mathbf{s}_i^A$ is of Type I, and $\tilde{\tau}_i^1 := \tau_i^1$ for all indices j where $\mathbf{s}_j^A$ is of Type II.

Claim. $\mathcal{H}_0 \approx_c \mathcal{H}_1$. The claim follows directly from a standard hybrid argument and invoking the standard (non programmable) PCG security.

- *Hybrid $\mathcal{H}_2$.* In this hybrid, we replace each τ_i^1 with a random $\tilde{\tau}_i^1$ for all $\mathbf{s}_i^A$ of Type II. We define

$$\mathcal{H}_1 := \{(\hat{\mathbf{s}}_1^A, (\tau_1^0, \tilde{\tau}_1^1)), \dots, (\hat{\mathbf{s}}_\nu^A, (\tau_\nu^0, \tilde{\tau}_\nu^1))\},$$

where $\tilde{\tau}_i^1 \leftarrow_R \{0, 1\}^\lambda$, for all indices $i \in [\nu]$.

To prove the computational indistinguishability claim, we will need the following lemma which, at a high level, states that we can program many PCGs with the same seed, without impacting the programmable PCG security.

Lemma 18 ([34, Lemma 7.6 from the full version]). *Let $\text{PCG} = (\text{PCG.Gen}, \text{PCG.Expand})$ be a programmable PCG as in Definition 7. Then, for any $d = \text{poly}(\lambda)$, it holds that*

$$\left\{ (s_i^B, (\tau^0, \tau_i^1))_{i=1}^d \mid \begin{array}{l} \forall i \in [d], \tau^0, \tau_i^1 \leftarrow_R \{0, 1\}^\lambda, \\ (s_i^A, s_i^B) \leftarrow \text{Gen}(1^\lambda, \tau^0, \tau_i^1, \text{aux}) \end{array} \right\} \\ \approx_c \left\{ (s_i^B, (\tau^0, \tau_i^1))_{i=1}^d \mid \begin{array}{l} \forall i \in [d], \tau^0, \tilde{\tau}^0, \tau_i^1 \leftarrow_R \{0, 1\}^\lambda, \\ (s_i^A, s_i^B) \leftarrow \text{Gen}(1^\lambda, \tilde{\tau}^0, \tau_i^1, \text{aux}) \end{array} \right\}.$$

A symmetric property holds for s^A .

Claim. $\mathcal{H}_1 \approx_c \mathcal{H}_2$. Suppose, towards contradiction, that there exists an efficient adversary $\mathcal{A}$ such that distinguishes between $\mathcal{H}_1$ and $\mathcal{H}_2$ with non-negligible advantage. Then, $\mathcal{A}$ must also be a distinguisher for Lemma 18. To see why, notice that all s_i^A of Type I are distributed identically between $\mathcal{H}_1$ and $\mathcal{H}_2$. Therefore, the only difference in distributions comes from the *interdependent* seeds s_j^A that are of Type II. However, in this case, the set of all dependent s_j^A 's form an instance of the left-hand-side of Lemma 18 in $\mathcal{H}_1$ and an instance of the right-hand-side of Lemma 18 in $\mathcal{H}_2$. Computational indistinguishability follows.

Claim. $\mathcal{H}_2$ satisfies Definition 15. The final thing left to show is that $\mathcal{H}_2$ (where we explicitly define τ^0, τ^1 as part of the distribution) satisfies Definition 15 where only the PCG seed and *output correlation* are part of the distribution. Fortunately, this has already been proven by Boyle et al. [33]. We restate their result in the following lemma:

Lemma 19 ([33, Lemma 7.4 of the full version]). *Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a bilinear map. Then, a programmable PCG $\text{PCG} = (\text{PCG.Gen}, \text{PCG.Expand})$ for the bilinear 2-party correlation $\mathcal{Y}_e$ is also a PCG for $\mathcal{Y}_e$ in the sense of Definition 5 (where the sampling of τ^0, τ^1 happens inside PCG.Gen).*

The claim then follows immediately from Lemma 19, concluding the proof. $\square$

4.4 Concrete parameters and efficiency estimates

We now give a rough estimate of concrete efficiency we can expect from Construction 2 when using the state-of-the-art programmable PCGs of Boyle et al. [33] based on the Ring-LPN assumption. First, because Construction 2 must output *all* the garbled circuits in a single expansion step, the parties need to store all the generated circuits which puts a practical limit on the number of garbled circuit instances they can reasonably generate. If we assume a 1 TB storage capacity and circuits of 10,000 AND gates, then each garbled gate is roughly $4 \cdot (2\lambda + 1)$ bits = 1,028 bits when using $\lambda = 128$. This makes a garbled circuit with 10,000 AND gates roughly 1.3 MB in size, and the parties can therefore be expected to store upwards of 750,000 garbled circuit instances.

Communication costs. Using the programmable PCG construction for the correlation required in Construction 2 (which can be viewed as being equivalent to *authenticated* Beaver triples) requires a seed size of approximately 3 MB [33, Table 4 in the full version]. Since we require *one* PCG seed per AND gate, this results in a seed size of 30 GB. This is roughly a 30 $\times$ improvement over naively secret-sharing the 750,000 garbled circuits. These communication costs can be improved by using more practical security parameters (e.g., $\lambda = 80$),

which reduce the communication size by roughly half [33] or using improved constructions of programmable PCGs in future work.

Computational costs. Using the parameters that fix the communication costs as described above, the estimated performance is 50,000 expansions per second [33, Table 4 in the full version]. We can therefore expect an (amortized) computational cost of roughly five garbled circuits per second. Of course, these performance numbers are for a single core. A parallel machine can improve performance by a linear factor in the number of cores, since all operations are parallelizable.

4.5 Practical optimizations

We highlight some practical considerations that can be applied to [Construction 2](#).

More efficient garbling for Boolean formulas. If the circuit $\mathcal{C}$ is a Boolean formula (i.e., the circuit is structured as a binary tree, where each input wire to a gate is not shared by any other gate), then we can use a *non-programmable* PCG in [Construction 2](#), which would improve the seed size by roughly factor of $100\times$ and computation by roughly $200\times$ [30, 100].

Replacing the PCG with a PCF. The way we defined [Construction 2](#), we can easily replace the programmable PCG with any programmable PCF for authenticated degree-2 correlations to enable *efficient* on-demand correlation generation. However, this approach still requires providing one PCF key per gate in the circuit, leading to a rather large encoding size. In [Section 6](#), we provide a much more efficient construction based on PCFs that satisfy an additional property.

Removing redundant correlations. Observe that if an AND gate in the circuit takes two inputs that are themselves outputs from two previous gates, the PCG output provides *redundant* secret shares of the input masks (since those masks are already available from the gate outputs). While the PCG is still required for the gate to compute the product of the masks, it need not output *all* the correlated shares. Instead, it suffices for the PCG to output $(r_i r_j, r_i r_j \cdot \Delta^A, r_i r_j \cdot \Delta^B)$, which for certain constructions of programmable PCGs (e.g., [30, 33]), can save a factor of approximately $2\times$ in computational overhead.

4.6 Reconfigurable garbled circuits via universal circuits

Here, we briefly describe a blackbox technique to generate “reconfigurable” secret-shared garbled circuits from our GPCG construction. In a nutshell, we can use GPCG.Gen from [Construction 2](#) as-is to generate PCG seeds for a *universal circuit* [106] $\mathcal{C}$ that takes as input a description of circuit $\tilde{\mathcal{C}}$ and input x and outputs $\tilde{\mathcal{C}}(x)$. Then, GPCG.Eval[•] can be defined to take an additional input $\tilde{\mathcal{C}}$. Because $\tilde{\mathcal{C}}$ can be known by both servers, selecting garbled-circuit labels corresponding to the description of $\tilde{\mathcal{C}}$ can be done using the approach described in [Section 2.1](#). Specifically, let $b_1, \dots, b_\ell$ be the bits describing $\tilde{\mathcal{C}}$. Then, then each server can non-interactively select the correct label by computing $L_{i,b_i}^\sigma := L_{i,0}^\sigma \oplus b_i \cdot \Delta^\sigma$. (The labels for the secret input x is still computed using [Construction 5](#).)

While this approach to decoupling the computation from the garbled circuit topology is immediate, it does not lead to practical parameters. For example, using state-of-the-art constructions for universal circuits, the number of gates blows up by a factor of $O(\log B)$ (where the constant factor is roughly 4) for computing circuits of size B [75, 87, 92, 114]. Therefore, if we let $B = 10,000$, then the seed size for our Garbling PCG would blow up to roughly 350 GB!

$\begin{array}{l} \text{Exp}_{\mathcal{A},0}^{\text{pr}}(\lambda): \\ R \leftarrow_R \mathcal{R} \\ b' \leftarrow \mathcal{A}^{\text{Eval}^0}(1^\lambda) \\ \text{return } b' \\ \\ \text{Oracle Eval}^0(P, (a_1, \dots, a_m)): \\ y := P(R(a_1), \dots, R(a_m)) \\ r \leftarrow_R \mathbb{G}_3, (y^A, y^B) := (y + r, -r) \\ \text{return } (y^A, y^B) \end{array}$	$\begin{array}{l} \text{Exp}_{\mathcal{A},1}^{\text{pr}}(\lambda): \\ (k^A, k^B) \leftarrow \text{TAPCF.Gen}(1^\lambda) \\ b' \leftarrow \mathcal{A}^{\text{Eval}^1}(1^\lambda) \\ \text{return } b' \\ \\ \text{Oracle Eval}^1(P, (a_1, \dots, a_m)): \\ \text{for } \sigma \in \{A, B\}: \\ y^\sigma \leftarrow \text{TAPCF.Eval}(\sigma, k^\sigma, P, (a_1, \dots, a_m)) \\ \text{return } (y^A, y^B) \end{array}$
--	---

Fig. 5: Pseudorandom $\mathcal{P}$ -correlated outputs for a TAPCF.

5 Topology-Adaptive PCFs

Here, we define the notion of a topology-adaptive PCF (TAPCF) and provide a construction from the EA-LPN assumption from [35], cf. [Definition 2](#). While we introduce TAPCF as a tool to achieve our main goal (garbling PCFs), we also observe that TAPCF are of independent interest and enjoy applications of their own right. We describe one such application, to silent secure computation with function-dependent preprocessing.

5.1 Definition

Definition 20 (Topology-Adaptive PCF (TAPCF)). *Let λ be a security parameter, let $m = m(\lambda), n = n(\lambda) \in \text{poly}(\lambda)$, let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3$ be Abelian groups of order $\text{poly}(\lambda)$ and define a set of m -input functions $\mathcal{P}$ of type $P: \mathbb{G}_2^m \rightarrow \mathbb{G}_3$. A TAPCF consists of a tuple of efficient algorithms $(\text{TAPCF.Gen}, \text{TAPCF.Eval})$ with the following functionality:*

- $\text{TAPCF.Gen}(1^\lambda) \rightarrow (k^A, k^B)$. *The randomized generation algorithm takes as input the security parameter λ and outputs a pair of keys (k^A, k^B) .*
- $\text{TAPCF.Eval}(\sigma, k^\sigma, P, (a_1, \dots, a_m)) \rightarrow y^\sigma$. *The deterministic evaluation algorithm takes as input $\sigma \in \{A, B\}$, a key k^σ , a description of a function $P \in \mathcal{P}$, and a vector $(a_1, \dots, a_m) \in \mathbb{G}_1^m$. It outputs an element $y^\sigma \in \mathbb{G}_3$.*

Let $\mathcal{R} = \{R: \mathbb{G}_1 \rightarrow \mathbb{G}_2\}$ be the set of all functions from $\mathbb{G}_1$ to $\mathbb{G}_2$. Then, TAPCF is a (strong) TAPCF, if the following two properties hold:

- **Correctness / Pseudorandom $\mathcal{P}$ -correlated outputs.** *For every $\sigma \in \{A, B\}$, and all efficient adversaries $\mathcal{A}$, there exists a negligible function negl such that for all sufficiently large λ ,*

$$\left| \Pr[\text{Exp}_{\mathcal{A},0}^{\text{pr}}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{A},1}^{\text{pr}}(\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\mathcal{A},b}^{\text{pr}}(\lambda)$, for $b \in \{0, 1\}$, is as defined in [Figure 5](#).

- **Security.** *For all distinct $\sigma, \bar{\sigma} \in \{A, B\}$, and all efficient adversaries $\mathcal{A}$, there exists a negligible function negl such that for all sufficiently large λ ,*

$$\left| \Pr[\text{Exp}_{\mathcal{A},\sigma,\bar{\sigma},0}^{\text{sec}}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{A},\sigma,\bar{\sigma},1}^{\text{sec}}(\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\mathcal{A},\sigma,\bar{\sigma},b}^{\text{sec}}(\lambda)$, for $b \in \{0, 1\}$, is as defined in [Figure 6](#).

$\text{Exp}_{\mathcal{A}, \sigma, \bar{\sigma}, 0}^{\text{sec}}(\lambda):$ $R \leftarrow_R \mathcal{R}$ $(k^A, k^B) \leftarrow \text{TAPCF.Gen}(1^\lambda)$ $b' \leftarrow \mathcal{A}^{\text{Eval}^0}(1^\lambda, \sigma, k^\sigma)$ $\text{return } b'$ $\text{Oracle Eval}^0(P, (a_1, \dots, a_m)):$ $y^\sigma \leftarrow \text{TAPCF.Eval}(\sigma, k^\sigma, P, (a_1, \dots, a_m))$ $y := P(R(a_1), \dots, R(a_m))$ $\text{if } \sigma = A: y^{\bar{\sigma}} := y + y^\sigma \text{ else: } y^{\bar{\sigma}} := y - y^\sigma$ $\text{return } y^{\bar{\sigma}}$	$\text{Exp}_{\mathcal{A}, \sigma, \bar{\sigma}, 1}^{\text{sec}}(\lambda):$ $(k^A, k^B) \leftarrow \text{TAPCF.Gen}(1^\lambda)$ $b' \leftarrow \mathcal{A}^{\text{Eval}^1}(1^\lambda, \sigma, k^\sigma)$ $\text{return } b'$ $\text{Oracle Eval}^1(P, (a_1, \dots, a_m)):$ $y^{\bar{\sigma}} \leftarrow \text{TAPCF.Eval}(\bar{\sigma}, k^{\bar{\sigma}}, P, (a_1, \dots, a_m))$ $\text{return } y^{\bar{\sigma}}$
---	--

Fig. 6: Security of game for a TAPCF.

Remark 21. Note that [Definition 20](#) has an “implicit” PRF $F_K: \mathbb{G}_1 \rightarrow \mathbb{G}_2$ applied to each input before the correlation function P is applied over the PRF outputs. Hence, we define security with respect to a truly random function $R: \mathbb{G}_1 \rightarrow \mathbb{G}_2$. Also, we remark on the choice of defining this PCF in the strong variant and how to obtain it in the standard model in [Remark 10](#).

5.2 TAPCF construction

We briefly recall the construction of PCF for degree- d correlations from [\[35\]](#), whose security reduces to the expand-accumulate LPN assumption ([Definition 2](#)). Let $\text{FSS} = (\text{FSS.Gen}, \text{FSS.Eval})$ be a function secret sharing scheme for d -dimensional rectangle predicates, defined as follows:

Definition 22 (d -Dimensional Rectangle Predicate). Let $\mathbb{G}$ be an Abelian group. A d -dimensional rectangle predicate $f_{(\alpha, \beta), m}$ for $\alpha, \beta \in (\{0, 1\}^\nu)^d$, and $m \in \mathbb{G}$, is defined as the function $f(\mathbf{x}) = m$ iff $\alpha_i \leq x_i \leq \beta_i$ for all $i \in [d]$, and $f(\mathbf{x}) = 0$ otherwise. When $\beta_i = \mathbf{1}$ for all $i \in [d]$, we denote it as $f_{\alpha, m}$. A d -dimensional distributed rectangle function is an FSS scheme for the family of d -dimensional rectangle predicates with domain $(\{0, 1\}^\nu)^d$ and range $\mathbb{G}$.

Fix parameters $\text{pp} = (N, t, c)$. N is the number of samples, c is a matrix sparsity parameter (typically $c = \Theta(\log N)$ or $\omega(\log N)$), and t is the Hamming weight of the noise. Let Δ_N denote a $5N$ -by- $5N$ lower triangular matrix filled with ones. We let $\mathcal{M}_{\text{ea}}(\text{pp})$ denote the distribution obtained by sampling an N -by- $5N$ matrix M whose entries are independent Bernoulli sample equal to 1 with probability $c/2N$, and outputting $H = M \cdot \Delta_N$. Let $\text{Ber}_{c/2N}^{5N}$ denote the distribution of the rows of M . We denote by $\mathcal{N}_{\text{ea}}(\text{pp})$ the distribution obtained by concatenating t random unit vectors of length $5N/t$.

The PCF construction of [\[35\]](#), which we restate here in the language of TAPCF, is given in [Construction 3](#). As an independent contribution, in proving security of our TAPCF construction, we provide a proof of security for the d -dimensional generalization of the PCF construction of Boyle et al. [\[35\]](#), which was not present in their original work.

Theorem 23. Assume that the $\text{EALPN}(N, t, c)$ assumption holds, and that $\text{FSS} = (\text{FSS.Gen}, \text{FSS.Eval})$ is a secure FSS for the class of d -dimensional rectangles. Then, [Construction 3](#) is a TAPCF for the class of all degree- d , m -variate polynomials over $\mathbb{F}_2$. Using the FSS scheme for d -dimensional rectangles of Boyle et al. [\[26\]](#), the scheme has the following complexities:

- each party’s key is of size $\approx 4\lambda(\log(5N/t)t)^d$ bits,
- the cost of TAPCF.Gen is dominated by t^d invocations of FSS.Gen ,

Construction 3 (Topology-Adaptive PCF).

Parameters. Let $(\text{FSS.Gen}, \text{FSS.Eval})$ be an FSS scheme for the class of d -dimensional rectangle predicates (Definition 22). Let $\text{pp} = (N, t, c)$ be the parameters of the EALPN-assumption. Let H be a hash function, modeled as a random oracle, that outputs sparse vectors over $\{0, 1\}^{5N}$ sampled from the Bernoulli distribution with parameter $c/5N$ (resulting in vectors of Hamming weight approximately c).

$\text{TAPCF.Gen}(1^\lambda, \text{aux})$:

- 1: $(\mathbf{e}_1, \dots, \mathbf{e}_d) \leftarrow_R \mathcal{N}_{\text{ea}}^d(\text{pp})$ $\triangleright$ Sample regular-LPN noise.
- 2: **foreach** $i \leq d$ **and** $j \leq t$:
- 3: Let $\alpha_j^{(i)} \in [5N/t]$ denote the position of the first 1 in the j -th block of $\mathbf{e}_i$.
- 4: **foreach** $\mathbf{i} = (i_1, i_2, \dots, i_d) \in [t]^d$:
- 5: Let $\boldsymbol{\alpha}_{\mathbf{i}}$ denote the d -tuple $\boldsymbol{\alpha}_{\mathbf{i}} = (\alpha_{i_1}^{(1)}, \dots, \alpha_{i_d}^{(d)})$
- 6: $(\kappa_{\mathbf{i}}^A, \kappa_{\mathbf{i}}^B) \leftarrow \text{FSS.Gen}(f_{\boldsymbol{\alpha}_{\mathbf{i}}, 1})$ $\triangleright f_{\boldsymbol{\alpha}_{\mathbf{i}}, 1}$ is a d -dimensional rectangle (Def. 22).
- 7: $(\mathbf{k}^A, \mathbf{k}^B) := ((\kappa_{\mathbf{i}}^A)_{\mathbf{i} \in [t]^d}, (\kappa_{\mathbf{i}}^B)_{\mathbf{i} \in [t]^d})$
- 8: **return** $(\mathbf{k}^A, \mathbf{k}^B)$

$\text{TAPCF.Eval}(\sigma, \mathbf{k}^\sigma, P, (a_1, \dots, a_m))$:

- 1: **parse** $\mathbf{k}^\sigma = (\kappa_{\mathbf{i}}^\sigma)_{\mathbf{i} \in [t]^d}$
- 2: Define a_i as (i, a_i) for each $a_i \in (a_1, \dots, a_m)$
- 3: **parse** P as an m -variate sum of degree- d monomials $\prod_{i \in \mathcal{I}} X_{a_i}$
where $\mathcal{I}$ is a subset of d elements from $\{a_1, \dots, a_m\}$ defining the monomial.
- 4: $y^\sigma := 0$
- 5: **foreach** monomial $(\prod_{i \in \mathcal{I}} X_{a_i}) \in P$
- 6: **foreach** $i \leq d$:
- 7: $\mathbf{h}_{a_i} \leftarrow \text{H}(a_i)$ and let c_i denote the Hamming weight of $\mathbf{h}_{a_i}$
 $\triangleright$ Note that $c_i \approx c$ due to sampling from the Bernoulli distribution.
- 8: **foreach** nonzero entry of $\mathbf{h}_{a_i}$:
- 9: Let $j_i^k \leq t$ denote the index of the corresponding noise block
 and α_i^k denote the position of the k -th 1 within this block.
 $\triangleright$ Note that each block is a unit vector because of the sampling from
 $\triangleright$ the regular noise distribution.
- 10: **foreach** tuple $(k_1, \dots, k_d) \in [c_1] \times \dots \times [c_d]$:
- 11: $\mathbf{j} := (j_1^{k_1}, \dots, j_d^{k_d})$
- 12: $\boldsymbol{\alpha} := (\alpha_1^{k_1}, \dots, \alpha_d^{k_d})$
- 13: $y^\sigma := y^\sigma + \text{FSS.Eval}(\sigma, \kappa_{\mathbf{j}}^\sigma, \boldsymbol{\alpha})$
- 14: **return** y^σ

Construction 3: Topology Adaptive PCF construction.

- the cost of TAPCF.Eval is dominated by $\approx (c \cdot \log(5N/t))^d$ invocations of a length-doubling pseudorandom generator.

Proof. The proof is given in [Appendix F.1](#). □

In particular, when $d = 2$, the construction needs t^2 FSS keys for a 2-dimensional interval function on a $2 \log(5N/t)$ -bit domain, which gives a main storage cost of around $4(\log(5N/t) \cdot t)^2 \lambda$ bits. The computation involves c^2 calls to FSS.Eval , which costs a total of $\approx (c \cdot \log(5N/t))^2$ PRG operations [26, 35, 77].

Remark 24 (TAPCF from variable-density LPN). Note that the PCF from Boyle et al. [34] also yields a TAPCF, whose security reduces to the variable-density LPN assumption. Since it yields a significantly less efficient construction, we focus on the EALPN-based construction.

5.3 Silent secure computation with function-dependent preprocessing

The primary motivation for PCFs is to enable *silent* generation of useful forms of correlated randomness. State-of-the-art PCF constructions [34–36, 40, 51] typically generate many 2-party Beaver triples over $\mathbb{F}_2$. In turn, these triples can be used within the celebrated GMW protocol [71]. This results in a silent 2-party secure computation protocol that communicates 4 bits per AND gate in the online phase.

Interestingly, it is known that 4 bits per AND gate is not the limit in the preprocessing model if one allows the preprocessing to be *function-dependent* [17, 31, 50, 58]. In particular, the TinyTable protocol [58] uses “Beaver-style” (degree-2) function-dependent correlated randomness to securely compute any Boolean circuits using only 2 bits per AND gate. This strongly motivates designing pseudorandom correlation generators (and functions) for function-dependent correlated randomness. This problem has received significant attention: it was first explicitly put forth in [33], and further pursued in [21, 91]. Yet, all known protocols operate in the *batch setting*: they require fixing the function ahead of time and are only efficient when many copies of the same function are computed on different inputs. Indeed, all known solutions have a seed size of the form $|f| \cdot \text{polylog}(n) \cdot \text{poly}(\lambda)$, where $|f|$ denotes the size of the target function and n the number of evaluations. This only becomes efficient when f is evaluated on a number of inputs that far exceeds its circuit size, which severely limits the applicability of these results.

We observe that TAPCFs yield the first fully satisfactory solution to the problem of silent secure computation with function-dependent preprocessing, resolving a five-year-old open problem in the field: a TAPCF enables generating function-dependent correlated randomness with a key size *independent of f* , and even allows specifying the function f on the fly, or changing the target function across different evaluations. Combining this observation with Theorem 23 and TinyTable, we get:

Theorem 25. *Assume that the $\text{EALPN}(N, t, c)$ assumption holds. Then, there exists a silent 2-party secure computation protocol for Boolean circuits in the semi-honest model with the following characteristics:*

- *Function-independent preprocessing: The dealer deals to each party a key of size*

$$\approx 4\lambda(\log 5N/t) \cdot t^2 \text{ bits,}$$

where N denotes a (possibly exponential) bound on the total number of AND gates of all functions to be securely evaluated in the future.

- *Function-dependent fully-silent preprocessing: Upon deciding on a function f to evaluate, the parties locally generate one TinyTable correlation per AND gate of f . The per-AND-gate cost of the silent preprocessing phase is dominated by $\approx (c \cdot \log(5N/t))^2$ invocations of a length-doubling PRG.*
- *Online phase: If f has n AND gates and m outputs, the parties communicate at most $2(n + m)$ bits.*

6 Garbling PCF from a TAPCF

In this section, we provide a construction for *on-demand* garbling correlations from TAPCFs. Our construction (presented in Construction 4) follows directly from our TAPCF abstraction.

Specifically, the TAPCF allows the evaluators to specify which wire masks are used as input to the AND gates. We begin by defining the notion of a garbling correlation and subsequently that of a Garbling PCF.

Definition 26 (Garbling Correlation). *Let λ be a security parameter and let*

$$\text{GC} = (\text{GC.SampLabels}_\sigma, \text{GC.Garble}, \text{GC.Encode}, \text{GC.Decode})$$

be a secret-sharing friendly garbled circuit scheme (cf. Definition 40). We define the correlation with setup $(\text{Setup}, \mathcal{Y}_{\text{GC}})$, where Setup takes λ as input and samples a correlation key $\text{yk} := (\text{yk}^A, \text{yk}^B) := (\Delta^A, \Delta^B) \in \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda}$. Then, $\mathcal{Y}_{\text{GC}}: (1^\lambda, \text{yk}, \mathcal{C}) \mapsto (y^A, y^B)$ proceeds as:

- 1: *Parse $\text{yk} = (\Delta^A, \Delta^B)$*
- 2: *$\mathbf{L}^A \leftarrow \text{GC.SampLabels}_A(1^\lambda, \mathcal{C})$.*
- 3: *$\mathbf{L}^B \leftarrow \text{GC.SampLabels}_B(1^\lambda, \mathcal{C})$.*
- 4: *$(\hat{\mathcal{C}}, \mathbf{r}) \leftarrow \text{GC.Garble}(1^\lambda, \mathcal{C}, \Delta^A, \mathbf{L}^A, \Delta^B, \mathbf{L}^B)$.*
- 5: *Secret share $\hat{\mathcal{C}}$ as $(\langle \hat{\mathcal{C}} \rangle^A, \langle \hat{\mathcal{C}} \rangle^B)$, s.t. $\hat{\mathcal{C}} = \langle \hat{\mathcal{C}} \rangle^A \oplus \langle \hat{\mathcal{C}} \rangle^B$.*
- 6: *Secret share $\mathbf{r}$ as $(\langle \mathbf{r} \rangle^A, \langle \mathbf{r} \rangle^B)$ s.t. $\mathbf{r} = \langle \mathbf{r} \rangle^A \oplus \langle \mathbf{r} \rangle^B$.*
- 7: *For $\sigma \in \{A, B\}$, secret share $\mathbf{r}\Delta^\sigma$ as $(\langle \mathbf{r}\Delta^\sigma \rangle^A, \langle \mathbf{r}\Delta^\sigma \rangle^B)$, s.t. $\mathbf{r}\Delta^\sigma = \langle \mathbf{r}\Delta^\sigma \rangle^A \oplus \langle \mathbf{r}\Delta^\sigma \rangle^B$.*
- 8: *Set $y^A = (\Delta^A, \mathbf{L}^A, \langle \hat{\mathcal{C}} \rangle^A, \langle \mathbf{r} \rangle^A, \langle \mathbf{r}\Delta^A \rangle^A, \langle \mathbf{r}\Delta^B \rangle^A)$.*
- 9: *Set $y^B = (\Delta^B, \mathbf{L}^B, \langle \hat{\mathcal{C}} \rangle^B, \langle \mathbf{r} \rangle^B, \langle \mathbf{r}\Delta^A \rangle^B, \langle \mathbf{r}\Delta^B \rangle^B)$.*
- 10: *Output (y^A, y^B) .*

Note that this correlation is reverse-sampleable (cf. Definition 8) for the secret-sharing-friendly garbling scheme of Construction 9:

Lemma 27. *Let GC be the secret-sharing-friendly garbling circuit scheme of Construction 9. Then, the garbling correlation for GC is reverse-sampleable.*

Proof. We prove for party $\sigma = A$, since the other case is symmetric. Consider the following reverse sampling algorithm RSample that takes as input λ , $\text{yk}^B = \Delta^B$, $\text{in} := \mathcal{C}$, and $y^A = (\Delta^A, \mathbf{L}^A, Y_{(\hat{\mathcal{C}})}, Y_{(\mathbf{r})}, Y_{(\mathbf{r}\Delta^A)}, Y_{(\mathbf{r}\Delta^B)})$ where y^A is sampled from $\mathcal{Y}(1^\lambda, \text{yk}, \mathcal{C})$:

- 1: *Sample $\mathbf{L}^B \leftarrow \text{SampLabels}_B(1^\lambda, \mathcal{C})$.*
- 2: *Compute a new garbling $(\hat{\mathcal{C}}, \mathbf{r}) \leftarrow \text{GC.Garble}(1^\lambda, \mathcal{C}, \Delta^A, \mathbf{L}^A, \Delta^B, \mathbf{L}^B)$.*
- 3: *Parse $\hat{\mathcal{C}} := (\hat{G}_1, \dots, \hat{G}_\nu)$ as a sequence of ν garbled gates.*
- 4: *Set $\langle \mathbf{r} \rangle^B := Y_{(\mathbf{r})} \oplus \mathbf{r}$. Then, $Y_{(\mathbf{r})} = \langle \mathbf{r} \rangle^A$ is still A 's share but now of the new $\mathbf{r}$.*
- 5: *Similarly, set $\langle \mathbf{r}\Delta^A \rangle^B := Y_{(\mathbf{r}\Delta^A)} \oplus \mathbf{r}\Delta^A$, $\langle \mathbf{r}\Delta^B \rangle^B := Y_{(\mathbf{r}\Delta^B)} \oplus \mathbf{r}\Delta^B$ to make $Y_{(\mathbf{r}\Delta^A)}$ and $Y_{(\mathbf{r}\Delta^B)}$ shares of the new $\mathbf{r}\Delta^A$ and $\mathbf{r}\Delta^B$, respectively.*
- 6: *For each $k \in [\nu]$, let $G_k = (i, j, k) \in \mathcal{C}$, and $\hat{G}_k = (g_{00}, g_{01}, g_{10}, g_{11})$.*
- 7: *Parse $Y_{(\hat{\mathcal{C}})} = (\hat{G}'_1, \dots, \hat{G}'_\nu)$ where $\hat{G}'_k = (g'_{00}, g'_{01}, g'_{10}, g'_{11})$ is a random by construction.*
- 8: *For all $k \in [\nu]$: Set $\hat{y}_k := (g'_{00} \oplus g_{00}, g'_{01} \oplus g_{01}, g'_{10} \oplus g_{10}, g'_{11} \oplus g_{11})$. Then this is a share of $\hat{G}_k$, together with $\hat{G}'_k$ as the other parties share.*
- 9: *Finally, output $y'^B = (\Delta^B, \mathbf{L}^B, (\hat{y}_k)_{k \in [\nu]}, \langle \mathbf{r} \rangle^B, \langle \mathbf{r}\Delta^A \rangle^B, \langle \mathbf{r}\Delta^B \rangle^B)$.*

Note that all random elements (labels, gates, and masks) have been generated as in the original correlation $\mathcal{Y}$ (using SampLabels_σ and Garble) and we have created fresh shares for B corresponding to A 's correlation values (the last four components of A 's correlation tuple are just random shares of “something” so they serve as fitting uniformly random masks).

Similarly, we have set all the secret shared garbled gates in the circuit to be of the freshly generated circuit, while maintaining that one of the corresponding shares remains $Y_{\hat{\mathcal{C}}}$. Now, it suffices to observe that for all $k \in [\nu]$, $\hat{y}_k \oplus y_k = g_k$, and y'^B is distributed identically to output y^B of the garbling correlation (conditioned on y^A). This concludes the proof of reverse samplability. $\square$

Definition 28 (Garbling PCF). Let λ be a security parameter and let GC be a secret-sharing-friendly garbled circuit scheme. A Garbling PCF for GC is a PCF (Definition 9) for the garbling correlation (Definition 26) with respect to GC.

Construction 4 (Garbling PCF).

Parameters. Let $\text{TAPCF} = (\text{TAPCF.Gen}, \text{TAPCF.Eval})$ be a TAPCF for bivariate, degree-2 correlations satisfying scalable outputs (Definition 43) with two scalars $\Delta^A, \Delta^B \in \mathbb{F}_{2^\lambda}$, and $P(a, b) = (a, b, c)$, with $c = ab \in \mathbb{F}_2$. Note that the output of TAPCF.Eval will also contain shares of the scaled versions $\Delta^A \cdot P(a, b)$ and $\Delta^B \cdot P(a, b)$. Let $H : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \mathbb{G}_1$ be a suitable hash function (see Remark 10), where $\mathbb{G}_1$ is the Abelian group as in Definition 20. Let the input wires of an s -input circuit $\mathcal{C}$ be indexed by $1, \dots, s$. Finally, let $\text{PRF} = (\text{PRF.KeyGen}, \text{PRF.Eval})$ be a PRF with outputs in $\{0, 1\}^{\text{poly}(\lambda)}$.

$\text{GPCF.Gen}(1^\lambda, yk)$:

- 1 : **parse** $yk = (\Delta^A, \Delta^B)$
- 2 : $(k_{\text{PCF}}^A, k_{\text{PCF}}^B) \leftarrow \text{TAPCF.Gen}(1^\lambda, \Delta^A, \Delta^B)$
- 3 : $K^A \leftarrow \text{PRF.KeyGen}(1^\lambda), K^B \leftarrow \text{PRF.KeyGen}(1^\lambda)$
- 4 : $k^A := (k_{\text{PCF}}^A, K^A, \Delta^A), k^B := (k_{\text{PCF}}^B, K^B, \Delta^B)$
- 5 : **return** (k^A, k^B)

$\text{GPCF.Eval}(\sigma, k^\sigma, \mathcal{C}, \text{nonce})$:

- 1 : **parse** $k^\sigma = (k_{\text{PCF}}^\sigma, K^\sigma, \Delta^\sigma)$
- 2 : **foreach** wire $i \in \mathcal{C}$
- 3 : **if** wire i is an output wire of $\mathcal{C}$ **then** $\langle r_i \rangle^\sigma := 0$
- 4 : $L^\sigma \leftarrow \text{SampleLabels}_\sigma(1^\lambda, \mathcal{C}; \text{PRF.Eval}(K^\sigma, \text{nonce}))$
- 5 : Let $S = \emptyset$ ▷ Set to collect input masks with Δ -factors
- 6 : **foreach** AND gate $G_k = (i, j, k) \in \mathcal{C}$ ▷ Expand all necessary correlations
- 7 : $\text{masks}_k \leftarrow \text{TAPCF.Eval}(\sigma, k_{\text{PCF}}^\sigma, P, H(i \parallel \text{nonce}), H(j \parallel \text{nonce}))$, where $\text{masks}_k =$

$$\left(\langle r_i \rangle^\sigma, \langle r_j \rangle^\sigma, \langle r_i r_j \rangle^\sigma, \langle r_i \Delta^A \rangle^\sigma, \langle r_j \Delta^A \rangle^\sigma, \langle (r_i r_j) \Delta^A \rangle^\sigma, \langle r_i \Delta^B \rangle^\sigma, \langle r_j \Delta^B \rangle^\sigma, \langle (r_i r_j) \Delta^B \rangle^\sigma \right)$$
- 8 : **if** i is an input wire: $S := S \cup \{ \langle r_i \Delta^A \rangle^\sigma, \langle r_i \Delta^B \rangle^\sigma \}$
- 9 : **if** j is an input wire: $S := S \cup \{ \langle r_j \Delta^A \rangle^\sigma, \langle r_j \Delta^B \rangle^\sigma \}$
- 10 : **foreach** AND gate $G_k = (i, j, k) \in \mathcal{C}$ ▷ Compute garbling
- 11 : $\text{labels} := (L_{i,0}^\sigma, L_{j,0}^\sigma, L_{k,0}^\sigma)$
- 12 : $\langle \hat{G}_k \rangle^\sigma \leftarrow \text{GarbleGate}(k, \text{labels}, \text{masks}_k, \langle r_k \rangle^\sigma, \langle r_k \Delta^A \rangle^\sigma, \langle r_k \Delta^B \rangle^\sigma, \Delta^\sigma)$
- 13 : ▷ as defined in Construction 1
- 14 : Let $\langle \hat{\mathcal{C}} \rangle^\sigma := (\langle \hat{G}_k \rangle^\sigma)_{k \in [\nu]}$
- 15 : Set $\langle r \Delta^A \rangle^\sigma := (\langle r_i \Delta^A \rangle^\sigma)_{i \in [s]}, \langle r \Delta^B \rangle^\sigma := (\langle r_i \Delta^B \rangle^\sigma)_{i \in [s]}$ with entries from S
- 16 : **return** $(\Delta^\sigma, L^\sigma, \langle \hat{\mathcal{C}} \rangle^\sigma, (\langle r_1 \rangle^\sigma, \dots, \langle r_s \rangle^\sigma), \langle r \Delta^A \rangle^\sigma, \langle r \Delta^B \rangle^\sigma)$

Construction 4: Garbling PCF construction using a TAPCF.

Proposition 29. *Under the expand-accumulate LPN assumption (cf. Definition 2), Construction 4 satisfies the correctness and privacy properties required of a Garbling PCF, defined in Definition 28. The expand-accumulate LPN assumption can be replaced with the variable-density LPN assumption using Remark 24.*

Proof. For each of the two properties (correctness and security), we show that for an efficient adversary on the property for the GPCF, we can construct an efficient adversary $\mathcal{B}$ on the respective property for the TAPCF with the same advantage in winning the game. See Appendix F.2 for the full proof. $\square$

6.1 Concrete parameters and efficiency estimates

We now give a rough estimate of concrete efficiency we can expect from Construction 4 when using the PCFs of Boyle et al. [35] based on expand-accumulate codes, (cf. Definition 2).

Parameters depend on the maximum number of correlations the parties might ever be expected to generate from their keys. We pick a very loose upper bound of $N = 2^{45}$ (roughly 35 trillion correlations, or four billion 10,000 gate circuits). Then, the PCF construction depends on two parameters: t and c which, respectively, determine the communication and computational efficiency of the resulting construction (see [35, Section 5.4 of the full version]).

On the security of EA-LPN. We stress that the Expand-Accumulate assumption is a recent assumption that is yet to receive significant cryptanalytic attention, and any choice of concrete parameter should be taken with a grain of salt. In particular, since the original publication, two papers [78, 100] have identified issues with the original formulation of EA-LPN and its choice of parameters, and suggested tweaks to the definition and updated parameters. We will rely on the state-of-the-art analysis provided by these papers in our parameter selection.

Very recently, Guo et al. [78] provided an attack on the EA-LPN assumption together with a fix that, essentially, slightly tweaks the noise distribution (with no impact on our TAPCF construction) and increases the key size (the number t of noisy coordinates) by a factor two. We note that [78] focuses only on the *aggressive* parameter sets from [35] (both in their attack and in their fix), while [35] also suggests more conservative parameters (backed up by more solid security arguments). For clarity, we state below the two possible key sizes when using the conservative parameters of [35], depending on whether the attack of [78] extends or not to these parameters.

Communication costs. The communication cost (in bits) is defined in terms of N , t , and c , and is given by $4\lambda \cdot (\log(5N/t) \cdot t)^2$ when using the state-of-the-art FSS schemes for 2-dimensional interval functions [26], where we will fix $\lambda = 128$ for the purposes of our estimates. Since we fix $N = 2^{45}$, we must carefully choose the parameters c and t . Boyle et al. [35] give both provable and heuristic parameter regimes. In the provable security regime, we can set $t = 85$ and $c = 3 \ln(5N)$. This results in a concrete key size of roughly 800 MB, or alternatively a noise parameter of $t = 170$ and a key size of 1.2 GB if the attack of [78] applies.

However, if we opt for heuristic parameters (optimized for concrete *computation* costs), we can set $t = 1328$ (updated from $t = 664$ in light of the attack and fix from [78]) and $c \in \{7, 9, 11\}$. These parameter choices are validated through simulation on the minimum distance of the code [35, 100]. We note that the more conservative choice of $c = 11$, compared to Boyle et al. [35], can be motivated in light of an improved minimum distance algorithm of Raghuraman et al. [100], affecting the security of the original heuristic parameters proposed in Boyle et al. [35].⁷ With these *compute-optimized* heuristic parameter choices, the key size

⁷ While the algorithm of Raghuraman et al. [100] does not directly result in an attack on the original parameter set of Boyle et al. [35], it invalidates the heuristic considerations on which they were based.

grows to 80 GB, but this significantly improves the concrete computation time, as we will explain next. We stress that these parameters might change in light of future cryptanalysis.

Computational costs. Computation costs are dominated by AES operations used to instantiate the 2-dimensional DPF [35]. The number of AES calls required, defined in terms of N , t , and c , and is given by $2 \cdot (c \cdot \log(5N/t))^2$ when using the state-of-the-art FSS schemes for 2-dimensional interval functions [26]. Using the AES-NI instruction set available on almost all modern CPUs, one byte of AES-128 can be computed in approximately 1.3 CPU cycles [5]. Therefore, on a 3.5 GHz processor, we can expect to process roughly 1.6×10^8 AES calls per second. Using the conservative parameters described above ($t = 170$, $c = 98$) and assuming that other computational costs are roughly 30% the cost of AES (an estimate obtained by examining state-of-the-art DPF implementations), we obtain roughly 4 PCF evaluations per second. However, when using the aggressive compute-optimized parameters ($t = 1328$, $c \in \{7, 9, 11\}$), we obtain throughputs ranging from roughly 370 (for $c = 11$) to 914 (for $c = 7$) PCF evaluations per second. Since we need one PCF evaluation *per gate*, a circuit with 10,000 AND gates would require roughly 12 seconds to compute *on one core* for $c = 7$. With the conservative parameters, on the other hand, generating a garbled circuit of 10,000 AND gates would require roughly 40 minutes. However, because this computation is again massively parallelizable, a 64-core machine should be able to generate one garbled circuit in under one minute. In the aggressive parameter regime, a 64-core machine should be able to generate one garbled circuit in only 200 ms of compute time.

Remark 30 (On efficient distributed key generation). Generating a TAPCF in a two-party computation protocol is bottlenecked by the generation of the DPF keys over a secret shared for a point function description [30, 33]. The most efficient protocol for generating one-dimensional DPF keys is the Doerner-shelat protocol [62], however, the protocol needs to be extended in the natural way to generating d -dimensional DPF keys (in our case $d = 2$).

7 Secret-Shared Input Garbling

In this section, we provide a construction for garbling inputs using the labels output by a Garbling PCG or PCF. In [Functionality 1](#), we describe the secret-shared input encoding functionality that we will instantiate later. While we define it to take inputs from two parties A and B , it can equivalently be defined to take the uploaded input x from a third party C (e.g., a client), and this is compatible with our instantiation as well. In [Definition 32](#), we define the syntax and interactive protocol structure that the parties run to instantiate [Functionality 1](#), which we later instantiate in [Section 7.1](#) using HSS.

Remark 31. We remark that the outputs of [Functionality 1](#) define secret shares of a garbled input $\hat{X}$ (as output by the secret-sharing-friendly garbled circuit scheme described in [Construction 9](#) in [Appendix C](#)).

Definition 32 (A secret-shared input garbling scheme). *Let HSS be any HSS scheme. A secret-shared input garbling scheme is a triple of efficient algorithms $\text{GI} = (\text{GI.Setup}, \text{GI.Encode}, \text{GI.SelectLabels})$, having the following syntax:*

- $\text{GI.Setup}(1^\lambda, \Delta^A, \Delta^B) \rightarrow (\text{pk}, \text{gik}^A, \text{gik}^B)$. *The randomized setup algorithm takes as input the security parameter λ , and two elements $\Delta^A, \Delta^B \in \mathbb{F}_{2^\lambda}$. It outputs keys $(\text{pk}, (\text{gik}^A, \text{gik}^B))$.*
- $\text{GI.Encode}(\text{pk}, x) \rightarrow ([x]^A, [x]^B)$. *The randomized input encoding algorithm takes as input a key pk and an input $x \in \mathcal{I}$. It outputs two input encodings of x with respect to HSS.*

Functionality 1 (Secret-shared input garbling).

Parameters. The functionality is parameterized by a security parameter λ , and two input lengths $s_A, s_B \in \mathbb{N}$ such that $s = s_A + s_B$. It interacts with two parties, denoted A and B , and an adversary $\mathcal{A}$. The functionality is split into **Setup**, **Encode** and **SelectLabels** procedures, defined as follows:

- **Setup:** Sample and store $(\Delta^A, \Delta^B) \leftarrow_R \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda}$.
- **Encode:**
 - 1: On input $x_A \in \{0, 1\}^{s_A}$ from party A and $x_B \in \{0, 1\}^{s_B}$ from party B .
 - 2: Store $x := x_A \parallel x_B = x_1 \parallel \dots \parallel x_s$ and output nothing.
- **SelectLabels:**
 - 1: If both parties are honest:
 - 2.1: Wait for $\text{In} = (\mathbf{L}_0^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \mathbf{r} \Delta^A \rangle^\sigma, \langle \mathbf{r} \Delta^B \rangle^\sigma)$ from σ , $\forall \sigma \in \{A, B\}$, where $\mathbf{L}_0^\sigma = (L_{1,0}^\sigma, \dots, L_{s,0}^\sigma)$, $\mathbf{r} = \langle \mathbf{r} \rangle^A \oplus \langle \mathbf{r} \rangle^B$, $\mathbf{r} \Delta^\sigma = \langle \mathbf{r} \Delta^\sigma \rangle^A \oplus \langle \mathbf{r} \Delta^\sigma \rangle^B$.
 - 2.2: Define $\rho_i := x_i \oplus \langle \mathbf{r}_i \rangle^A \oplus \langle \mathbf{r}_i \rangle^B$, for all $i \in [s]$.
 - 2.3: Sample $\langle x \rangle^A, \langle x \rangle^B \leftarrow_R \{0, 1\}^s$ subject to $\langle x \rangle^A \oplus \langle x \rangle^B = x$.
 - 2.4: Sample $\langle x \Delta^\sigma \rangle^A, \langle x \Delta^\sigma \rangle^B \leftarrow_R \mathbb{F}_{2^\lambda}^s$ subject to $\langle x \Delta^\sigma \rangle^A \oplus \langle x \Delta^\sigma \rangle^B = x \Delta^\sigma$, for all $\sigma \in \{A, B\}$.
 - 2.5: Compute $\langle \rho_i \rangle^\sigma := \langle x_i \rangle^\sigma \oplus \langle \mathbf{r}_i \rangle^\sigma$, for all $\sigma \in \{A, B\}$.
 - 2.6: Compute $\langle \rho_i \Delta^A \rangle^\sigma := \langle x_i \Delta^A \rangle^\sigma \oplus \langle \mathbf{r}_i \Delta^A \rangle^\sigma$, for all $\sigma \in \{A, B\}$.
 - 2.7: Compute $\langle \rho_i \Delta^B \rangle^\sigma := \langle x_i \Delta^B \rangle^\sigma \oplus \langle \mathbf{r}_i \Delta^B \rangle^\sigma$, for all $\sigma \in \{A, B\}$.
 - 2.8: For all $\sigma \neq \bar{\sigma} \in \{A, B\}$, compute:
 - 2.8.1: $\langle L_{i,\rho_i}^\sigma \rangle^\sigma := L_{i,0}^\sigma \oplus \langle \rho_i \Delta^\sigma \rangle^\sigma$, $\langle L_{i,\rho_i}^{\bar{\sigma}} \rangle^\sigma := \langle \rho_i \Delta^{\bar{\sigma}} \rangle^\sigma$ for all $i \in [s]$.
 - 2.8.2: $\langle \mathbf{L}_\rho^\sigma \rangle^\sigma := (\langle L_{1,\rho_1}^\sigma \rangle^\sigma, \dots, \langle L_{s,\rho_s}^\sigma \rangle^\sigma)$, $\langle \mathbf{L}_\rho^{\bar{\sigma}} \rangle^\sigma := (\langle L_{1,\rho_1}^{\bar{\sigma}} \rangle^\sigma, \dots, \langle L_{s,\rho_s}^{\bar{\sigma}} \rangle^\sigma)$.
 - 2: If party σ is honest and $\bar{\sigma} \in \{A, B\} \setminus \{\sigma\}$ is corrupted:
 - 3.1: Wait for $\text{In} = (\mathbf{L}_0^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \mathbf{r} \Delta^A \rangle^\sigma, \langle \mathbf{r} \Delta^B \rangle^\sigma)$ from party σ .
 - 3.2: Wait for (In, Out) from $\mathcal{A}$, where:
$$\text{In} = (\mathbf{L}_0^{\bar{\sigma}}, \langle \mathbf{r} \rangle^{\bar{\sigma}}, \langle \mathbf{r} \Delta^A \rangle^{\bar{\sigma}}, \langle \mathbf{r} \Delta^B \rangle^{\bar{\sigma}}), \quad \text{Out} = (\langle \mathbf{L}_\rho^A \rangle^{\bar{\sigma}}, \langle \mathbf{L}_\rho^B \rangle^{\bar{\sigma}}) \parallel (\langle \rho_1 \rangle^{\bar{\sigma}}, \dots, \langle \rho_s \rangle^{\bar{\sigma}})$$

where $\mathbf{L}_0^{\bar{\sigma}} = (L_{1,0}^{\bar{\sigma}}, \dots, L_{s,0}^{\bar{\sigma}})$, $\mathbf{r} = \langle \mathbf{r} \rangle^A \oplus \langle \mathbf{r} \rangle^B$, $\mathbf{r} \Delta^\sigma = \langle \mathbf{r} \Delta^\sigma \rangle^A \oplus \langle \mathbf{r} \Delta^\sigma \rangle^B$.
 - 3.3: Define $\rho_i := x_i \oplus \langle \mathbf{r}_i \rangle^A \oplus \langle \mathbf{r}_i \rangle^B$, for all $i \in [s]$.
 - 3.4: For all $i \in [s]$:
 - 3.4.1: Compute $\langle x_i \rangle^{\bar{\sigma}} := \langle \rho_i \rangle^{\bar{\sigma}} \oplus \langle \mathbf{r}_i \rangle^{\bar{\sigma}}$ and $\langle x_i \rangle^\sigma := x_i \oplus \langle x_i \rangle^{\bar{\sigma}}$.
 - 3.4.2: Compute $\langle \rho_i \rangle^\sigma := \langle x_i \rangle^\sigma \oplus \langle \mathbf{r}_i \rangle^\sigma$.
 - 3.4.3: Compute $L_{i,\rho_i}^A = L_{i,0}^A \oplus \rho_i \Delta^A$, and $L_{i,\rho_i}^B = L_{i,0}^B \oplus \rho_i \Delta^B$.
 - 3.4.4: Compute $\langle L_{i,\rho_i}^A \rangle^\sigma := L_{i,\rho_i}^A \oplus \langle L_{i,\rho_i}^A \rangle^{\bar{\sigma}}$ and $\langle L_{i,\rho_i}^B \rangle^\sigma := L_{i,\rho_i}^B \oplus \langle L_{i,\rho_i}^B \rangle^{\bar{\sigma}}$.
 - 3.5: Let $\langle \mathbf{L}_\rho^\sigma \rangle^\sigma := (\langle L_{1,\rho_1}^\sigma \rangle^\sigma, \dots, \langle L_{s,\rho_s}^\sigma \rangle^\sigma)$.
 - 3: Output $(\langle \mathbf{L}_\rho^A \rangle^A, \langle \mathbf{L}_\rho^B \rangle^A) \parallel (\langle \rho_1 \rangle^A, \dots, \langle \rho_s \rangle^A)$ to party A .
 - 4: Output $(\langle \mathbf{L}_\rho^A \rangle^B, \langle \mathbf{L}_\rho^B \rangle^B) \parallel (\langle \rho_1 \rangle^B, \dots, \langle \rho_s \rangle^B)$ to party B .

Functionality 1: Ideal functionality for secret shared input garbling.

- $\text{Gl.SelectLabels}(\sigma, \text{gik}^\sigma, \llbracket x \rrbracket^\sigma, \mathbf{L}_0^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \mathbf{r} \Delta^A \rangle^\sigma, \langle \mathbf{r} \Delta^B \rangle^\sigma) \rightarrow \langle \hat{X} \rangle^\sigma$. The deterministic label selection algorithm takes as input a party index σ , a key gik^σ , and σ 's encoding of input x , input labels $\mathbf{L}_0^\sigma$ and a share of masks $\langle \mathbf{r} \rangle^\sigma$, together with Δ -scaled versions, $\langle \mathbf{r} \Delta^A \rangle^\sigma$, $\langle \mathbf{r} \Delta^B \rangle^\sigma$. It outputs additive secret shares of the garbled input corresponding to x .

We say that Gl is correct and secure if it instantiates *Functionality 1* via the interactive protocol between parties A and B described in *Fig. 7*, where party $\sigma \in \{A, B\}$ has input $(x_\sigma, \mathbf{L}_0^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \mathbf{r} \Delta^A \rangle^\sigma, \langle \mathbf{r} \Delta^B \rangle^\sigma)$.

Setup:

- 1 : $\Delta^A, \Delta^B \leftarrow \mathbb{F}_{2^\lambda}$
- 2 : $(pk, (gik^A, gik^B)) \leftarrow \text{Gl.Setup}(1^\lambda, \Delta^A, \Delta^B)$
- 3 : **send** (pk, gik^A) to party A , **send** (pk, gik^B) to party B

Encode:

- 1 : Each party σ computes $(\llbracket x_\sigma \rrbracket^A, \llbracket x_\sigma \rrbracket^B)$ via $\text{Gl.Encode}(pk, x_\sigma)$
- 2 : **send** $\llbracket x_\sigma \rrbracket^{\bar{\sigma}}$ to party $\bar{\sigma} \in \{A, B\} \setminus \{\sigma\}$

SelectLabels:

- 1 : $\llbracket x \rrbracket^\sigma := (\llbracket x_{A,1} \rrbracket^\sigma, \dots, \llbracket x_{A,s_A} \rrbracket^\sigma, \llbracket x_{B,1} \rrbracket^\sigma, \dots, \llbracket x_{B,s_B} \rrbracket^\sigma)$
- 2 : Each party $\sigma \in \{A, B\}$ does the following:

$$\langle \hat{X} \rangle^\sigma \leftarrow \text{Gl.SelectLabels}(\sigma, gik^\sigma, \llbracket x \rrbracket^\sigma, L_0^\sigma, \langle r \rangle^\sigma, \langle r \Delta^A \rangle^\sigma, \langle r \Delta^B \rangle^\sigma)$$

Fig. 7: Interactive protocol utilizing the Gl algorithms to instantiate [Functionality 1](#).

7.1 Secret-shared input garbling with HSS

In [Construction 5](#), we instantiate [Functionality 1](#) using any public key HSS scheme [7, 25, 27, 32]. In this construction, the input x and the pair of global masks (Δ^A, Δ^B) are secret-shared via the public-key HSS scheme. Then, using the HSS scheme, the parties can non-interactively encode their inputs by multiplying the shares of x with the shares of Δ^A and Δ^B , inside HSS, to obtain additive secret shares of the resulting product.

Proposition 33. *Assuming the correctness and security of the homomorphic secret sharing scheme HSS, [Construction 5](#) securely instantiates [Functionality 1](#) using the interactive protocol defined in [Definition 32](#).*

Proof. We consider the honest and corrupted cases separately.

Case: Both parties are honest. If both parties are honest, note that by the correctness of the HSS.Mult algorithm, we have that $\langle y_i^A \rangle^\sigma$ is an additive secret share of $x_i \cdot \Delta^A \in \mathbb{F}_{2^\lambda}$ and $\langle y_i^B \rangle^\sigma$ is an additive secret share of $x_i \cdot \Delta^B \in \mathbb{F}_{2^\lambda}$. We then have that $\langle z_i^A \rangle^\sigma$ is an additive share of $(x_i \oplus r_i) \cdot \Delta^A$ and $\langle z_i^B \rangle^\sigma$ is an additive share of $(x_i \oplus r_i) \cdot \Delta^B$. This means that $\langle L_{i,\rho_i}^A \rangle^\sigma$ is an additive share of $L_{i,0}^A \oplus (x_i \oplus r_i) \cdot \Delta^A$ and $\langle L_{i,\rho_i}^B \rangle^\sigma$ is an additive share of $L_{i,0}^B \oplus (x_i \oplus r_i) \cdot \Delta^B$, which matches the output of [Functionality 1](#). By the correctness of the HSS.Convert algorithm, $\langle x_i \rangle^\sigma$ is an additive secret share of x_i deterministically computed from $\llbracket x_i \rrbracket^\sigma$. Then, by the additive homomorphism of additive secret shares, we have that $\langle x_i \rangle^\sigma \oplus \langle r_i \rangle^\sigma = \langle x_i \oplus r_i \rangle^\sigma$, which matches the output given to party σ .

Case: One of the parties is corrupted. We prove the case where $\mathcal{A}$ corrupts party A (the other case is perfectly symmetric). We first construct the following efficient simulator $\mathcal{S}$ for the view of party B : For each invocation of **Encode**, $\mathcal{S}$ plays the role of party B in the real world, and computes $(\llbracket 0 \rrbracket^A, \llbracket 0 \rrbracket^B) \leftarrow \text{HSS.Input}(pk, 0^{s_B})$, then sends $\llbracket 0 \rrbracket^A$ to $\mathcal{A}$, and stores $\llbracket 0 \rrbracket^B$ for future use. When it receives $\llbracket x_A \rrbracket^B$ from $\mathcal{A}$, it stores it.

For each invocation of **SelectLabels**, $\mathcal{S}$ plays the role of $\mathcal{A}$ in the ideal world and invokes the functionality on input (In, Out) , where $\mathcal{S}$ computes (In, Out) using the random coins of $\mathcal{A}$ (recall that we are in the semi-honest model where the simulator gets the coins of $\mathcal{A}$). It then computes Gl.SelectLabels in [Construction 5](#) using $\llbracket x_A \rrbracket^A$ for party A 's input (stored from **Encode**) and the simulated HSS input $\llbracket 0 \rrbracket^A$ for party B .

Construction 5 (Secret-shared input garbling with HSS).

Parameters. Let $\text{HSS} = (\text{HSS.Setup}, \text{HSS.Input}, \text{HSS.Eval})$ be any HSS scheme and let **Mult** and **Convert** be the auxiliary HSS functions defined in Definition 12.

Gl.Setup($1^\lambda, \Delta^A, \Delta^B$):

- 1 : $(\text{pk}, (\text{ek}^A, \text{ek}^B)) \leftarrow \text{HSS.Setup}(1^\lambda)$
- 2 : $(\llbracket \Delta^A \rrbracket^A, \llbracket \Delta^A \rrbracket^B) \leftarrow \text{HSS.Input}(\text{pk}, \Delta^A)$
- 3 : $(\llbracket \Delta^B \rrbracket^A, \llbracket \Delta^B \rrbracket^B) \leftarrow \text{HSS.Input}(\text{pk}, \Delta^B)$
- 4 : $\text{gik}^A := (\text{ek}^A, \llbracket \Delta^A \rrbracket^A, \llbracket \Delta^B \rrbracket^A), \quad \text{gik}^B := (\text{ek}^B, \llbracket \Delta^A \rrbracket^B, \llbracket \Delta^B \rrbracket^B)$
- 5 : **return** $(\text{pk}, \text{gik}^A, \text{gik}^B)$

Gl.Encode(pk, x):

- 1 : **return** $(\llbracket x \rrbracket^A, \llbracket x \rrbracket^B) \leftarrow \text{HSS.Input}(\text{pk}, x)$

▷ **Gl.SelectLabels** when defined to take an HSS memory share of x .

Gl.SelectLabels($\sigma, \text{gik}^\sigma, \langle x \rangle^\sigma, L_0^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \Delta^A \cdot \mathbf{r} \rangle^\sigma, \langle \Delta^B \cdot \mathbf{r} \rangle^\sigma$):

- 1 : **parse** $\text{gik}^\sigma = (\text{ek}^\sigma, \llbracket \Delta^A \rrbracket^\sigma, \llbracket \Delta^B \rrbracket^\sigma)$
- 2 : **parse** $L_0^\sigma = (L_{1,0}^\sigma, \dots, L_{s,0}^\sigma)$ **and** $\langle \mathbf{r} \rangle^\sigma = (\langle r_1 \rangle^\sigma, \dots, \langle r_s \rangle^\sigma)$
- 3 : **parse** $\langle x \rangle^\sigma = (\langle x_1 \rangle^\sigma, \dots, \langle x_s \rangle^\sigma)$
- 4 : **foreach** $i \in [s]$:
- 5 : $\langle x_i \rangle^\sigma \leftarrow \text{Convert}(\text{ek}^\sigma, \langle x_i \rangle^\sigma)$
- 6 : $\langle y_i^A \rangle^\sigma \leftarrow \text{Mult}(\text{ek}^\sigma, \llbracket \Delta^A \rrbracket^\sigma, \langle x_i \rangle^\sigma)$
- 7 : $\langle y_i^B \rangle^\sigma \leftarrow \text{Mult}(\text{ek}^\sigma, \llbracket \Delta^B \rrbracket^\sigma, \langle x_i \rangle^\sigma)$
- 8 : $\langle z_i^A \rangle^\sigma := \langle y_i^A \rangle^\sigma \oplus \langle \Delta^A \cdot \mathbf{r}_i \rangle^\sigma, \quad \langle z_i^B \rangle^\sigma := \langle y_i^B \rangle^\sigma \oplus \langle \Delta^B \cdot \mathbf{r}_i \rangle^\sigma,$
- 9 : **if** $\sigma = A$: $\langle L_{i,\rho_i}^A \rangle^A := L_{i,0}^A \oplus \langle z_i^A \rangle^A, \quad \langle L_{i,\rho_i}^B \rangle^A := \langle z_i^B \rangle^A$
- 10 : **if** $\sigma = B$: $\langle L_{i,\rho_i}^B \rangle^B := L_{i,0}^B \oplus \langle z_i^B \rangle^B, \quad \langle L_{i,\rho_i}^A \rangle^B := \langle z_i^A \rangle^B$
- 11 : $\langle L_\rho^A \rangle^\sigma := (\langle L_{1,x_1}^A \rangle^\sigma, \dots, \langle L_{s,\rho_s}^A \rangle^\sigma), \quad \langle L_\rho^B \rangle^\sigma := (\langle L_{1,x_1}^B \rangle^\sigma, \dots, \langle L_{s,\rho_s}^B \rangle^\sigma)$
- 12 : $\langle \hat{X} \rangle^\sigma := (\langle L_\rho^A \rangle^\sigma, \langle L_\rho^B \rangle^\sigma) \parallel (\langle x_1 \rangle^\sigma \oplus \langle r_1 \rangle^\sigma, \dots, \langle x_s \rangle^\sigma \oplus \langle r_s \rangle^\sigma)$
- 13 : **return** $\langle \hat{X} \rangle^\sigma$

▷ **Gl.SelectLabels** when defined to take an HSS input share of x .

Gl.SelectLabels($\sigma, \text{gik}^\sigma, \llbracket x \rrbracket^\sigma, L_0^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \Delta^A \cdot \mathbf{r} \rangle^\sigma, \langle \Delta^B \cdot \mathbf{r} \rangle^\sigma$):

- 1 : **parse** $\text{gik}^\sigma = (\text{ek}^\sigma, _, _)$
- 2 : $\langle x \rangle^\sigma \leftarrow \text{HSS.Convert}(\text{ek}^\sigma, \llbracket x \rrbracket^\sigma)$
- 3 : **return** **Gl.SelectLabels**($\sigma, \text{gik}^\sigma, \langle x \rangle^\sigma, L_0^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \Delta^A \cdot \mathbf{r} \rangle^\sigma, \langle \Delta^B \cdot \mathbf{r} \rangle^\sigma$)

Construction 5: Secret-shared input garbling with HSS construction.

To prove security, we argue that the joint distribution of the transcript and output of $\mathcal{A}$ in the real world are computationally indistinguishable to the ideal world where $\mathcal{S}$ plays the role of $\mathcal{A}$. To see this, it suffices to observe that the only difference in distributions is that, in the real world, Bob sends an HSS input share $\llbracket x_B \rrbracket^A$ of his input x_B to Alice, while in the ideal world $\mathcal{S}$ sends an HSS input share of zero. Indistinguishability between the real and ideal world therefore follows directly from the security of the HSS scheme. $\square$

8 Applications

In this section, we present three applications of our garbling PCF and PCG constructions.

8.1 Efficient HSS for T-shaped circuits

HSS constructions [7, 25, 27, 28, 32, 59, 65, 67, 98, 103] enable computations over secret shares of a database held by a set of servers (typically two), where one server is assumed to not collude with the other. By distributing the database across multiple servers, many clients can query the database and obtain the query results, without requiring the servers to communicate.

HSS constructions provide a compelling alternative to fully-homomorphic encryption, by providing concrete efficiency [7, 27, 32] and constructions from a diverse set of cryptographic assumptions [25, 52, 59, 65, 98, 103]. However, these constructions only work for computing *low-degree* polynomials, as the computational costs blow up *exponentially* in the degree. This limitation currently prevents HSS from being used in many natural and practical contexts.

	Basic HSS	Our framework
Comm.	$O(k)$	$O(k \cdot d)$
Comp.	$O(n \cdot 2^d)$	$O(n + k \cdot d)$

Table 1: Communication and computation comparison between basic HSS and our HSS framework when computing T-shaped circuits with n inputs in a constant-degree pre-processing phase and a degree- d post-processing applied to k inputs.

T-shaped computations. We make the observation that database and data structure queries are often represented by T-shaped circuits, where a wide-and-shallow pre-processing computation is applied to all items, followed by a narrow-and-deep post-processing computation applied to a small subset of selected items from the pre-processing step.

Examples of such T-shaped database computations include: threshold counting queries, approximate nearest neighbor search queries [6, 70], bloom filter membership testing [20], and hash-based data structure queries [44, 47, 74].

Our key insight is that we can build efficient HSS for T-shaped computations by combining (1) any HSS scheme for low-degree pre-processing with (2) a secret-shared garbled circuit to encode the post-processing. By “connecting” the HSS output with the garbled circuit, we guarantee the same level of privacy (the client cannot recover anything more than the output of the computation) while also gaining the best-of-both-worlds in terms of computational efficiency (see comparison in Table 1). We arrive at the following informal theorem.

Theorem 34 (HSS for T-shaped computation (informal)). *For all T-shaped computations with a constant-degree pre-processing applied to n inputs followed by a degree- d , width- k post-processing computation, there exists an HSS scheme with a non-additive reconstruction algorithm, achieving a download communication cost (i.e., the response share size) of $O(k \cdot d)$ and server-side computation cost of $O(n + k \cdot d)$.*

Construction. For a T-shaped computation (as in Theorem 34), let P_0 be an n -input, k -output computation with a multiplicative depth d_0 and P_1 be a k -input computation with a multiplicative depth d_1 and width $O(k)$. Because the computation is T-shaped, we have that $d_0 \ll d_1$ and $k \ll n$. The full construction is given in Construction 6. Here, we provide a high-level overview of the construction:

- **HSS computation:** Compute P_0 over the n HSS input values, which produces k HSS memory shares of the output bits $(y_1, \dots, y_k)$.
- **Garbled circuit:** Generate a secret-shared garbled circuit $\hat{C}$ computing the function P_1 .

- **Garbled inputs:** Garble input bits $(y_1, \dots, y_k)$ using [Construction 5](#) to produce $\hat{X}$.
- **Output:** $\mathcal{C}$ and the shares of $\hat{\mathcal{C}}$ and $\hat{X}$.

Construction 6 (HSS for T-Shaped Computation).

Parameters. Let $\text{HSS} = (\text{HSS.Setup}, \text{HSS.Input}, \text{HSS.Eval})$ be a HSS scheme for a class of programs $\mathcal{P}$, where $P_0 \in \mathcal{P}$ is an n -input, k -output computation with a multiplicative depth d_0 . Let $\text{GC} = (\text{GC.SampLabels}_\sigma, \text{GC.Garble}, \text{GC.Encode}, \text{GC.Decode})$ be a secret-sharing-friendly garbled circuit scheme, and $\text{GPCF} = (\text{GPCF.Gen}, \text{GPCF.Eval})$ be a Garbling PCF for GC . Let $\text{GI} = (\text{GI.Setup}, \text{GI.Encode}, \text{GI.SelectLabels})$ be the HSS-based secret-shared input garbling scheme from [Construction 5](#), using HSS .

$\text{HSS}^\top.\text{Setup}(1^\lambda)$:

- 1: $\Delta^A, \Delta^B \leftarrow_R \mathbb{F}_{2^\lambda}$
- 2: $(k^A, k^B) \leftarrow \text{GPCF.Gen}(1^\lambda, (\Delta^A, \Delta^B))$
- 3: $(pk, gik^A, gik^B) \leftarrow \text{GI.Setup}(1^\lambda, \Delta^A, \Delta^B)$
- 4: $ek^A := (k^A, gik^A), ek^B := (k^B, gik^B)$
- 5: **return** $(pk, (ek^A, ek^B))$

$\text{HSS}^\top.\text{Input}(pk, x)$:

- 1: **return** $(\llbracket x \rrbracket^A, \llbracket x \rrbracket^B) \leftarrow \text{HSS.Input}(pk, x)$

$\text{HSS}^\top.\text{Eval}(\sigma, ek^\sigma, \llbracket x \rrbracket^\sigma, P)$:

- 1: **parse** $P = (P_0, P_1)$, $ek^\sigma = (k^\sigma, gik^\sigma)$, and $gik^\sigma = (ek_{\text{HSS}}^\sigma, \llbracket \Delta^A \rrbracket^\sigma, \llbracket \Delta^B \rrbracket^\sigma)$
- 2: $\langle y_0 \rangle^\sigma \leftarrow \text{HSS.Eval}^*(\sigma, ek_{\text{HSS}}^\sigma, \llbracket x \rrbracket^\sigma, P_0)$
- 3: $(\Delta^\sigma, L_0^\sigma, \langle \hat{\mathcal{C}} \rangle^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \mathbf{r} \Delta^A \rangle^\sigma, \langle \mathbf{r} \Delta^B \rangle^\sigma) \leftarrow \text{GPCF.Eval}(\sigma, k^\sigma, \mathcal{C}, \text{ctr})$,
where $\mathcal{C}$ is the circuit encoding P_1 and ctr is a counter, incremented each time
- 4: $\langle \hat{X} \rangle^\sigma \leftarrow \text{GI.SelectLabels}(\sigma, gik^\sigma, \langle y_0 \rangle^\sigma, L_0^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \mathbf{r} \Delta^A \rangle^\sigma, \langle \mathbf{r} \Delta^B \rangle^\sigma)$
- 5: **return** $y^\sigma := (\mathcal{C}, \langle \hat{\mathcal{C}} \rangle^\sigma, \langle \hat{X} \rangle^\sigma)$

$\text{HSS}^\top.\text{Recover}(y^A, y^B)$:

- 1: **parse** $y_A = (\mathcal{C}, \langle \hat{\mathcal{C}} \rangle^A, \langle \hat{X} \rangle^A)$, $y_B = (\mathcal{C}', \langle \hat{\mathcal{C}} \rangle^B, \langle \hat{X} \rangle^B)$ $\triangleright$ **assert** $\mathcal{C} = \mathcal{C}'$
- 2: $\hat{\mathcal{C}} := \langle \hat{\mathcal{C}} \rangle^A + \langle \hat{\mathcal{C}} \rangle^B$, $\hat{X} := \langle \hat{X} \rangle^A + \langle \hat{X} \rangle^B$
- 3: **return** $\text{GC.Decode}(\mathcal{C}, \hat{\mathcal{C}}, \hat{X})$

Construction 6: HSS for T-shaped circuits via GPCFs.

The HSS computation of P_0 requires a computational cost of $O(n \cdot 2^{d_0})$, since the computational cost is exponential in the depth of P_0 . Generating the garbled circuit requires a computational cost of $O(k \cdot d_1)$ (we assume the width of the circuit is $O(k)$). Finally, the communication cost is just the share size of the garbled circuit and garbled input, which is $O(k \cdot d_1)$. In all of the above uses, $O(\cdot)$ hides a linear factor in the security parameter.

Remark 35 (Non-additive reconstruction). The exponential improvement in the computational overhead of the HSS computation for T-shaped circuits using [Theorem 34](#) comes at the cost of non-additive reconstruction. Additive reconstructions in HSS schemes provides two main benefits: (1) guaranteed function privacy and (2) optimal download rate [25, 28]. In database query applications, where a client obtains the output of the computation, additive

reconstruction not particularly important and only modestly increases the download rate by a factor of d .

8.1.1 Applications of HSS for T-shaped circuits Many database computations can be divided into two steps that can be viewed as being T-shaped computations. In the first step, a “filtering” function is applied over all database items to select a small subset of items. In the second step, a (typically, computationally expensive) function is applied to the filtered subset of items. For example, many hashing-based data structures [6, 20, 44, 47, 70, 74] work by first retrieving a subset of items from the data structure / database (using the the hash function as the “filtering” function), and then applying a post-processing operation on this retrieved subset of items.

A framework via syndrome decoding. We can efficiently instantiate an HSS for T-shaped database computation described above by combining a linear sketch based on any suitable linear error-correcting code [80, 94, 101] with an HSS scheme for low-degree polynomials. For a database of size n , where we assume the post-processing is applied to at most $k \ll n$ matches, we let H be an $\ell \times n$ parity-check matrix with an efficient syndrome decoder capable of decoding up to k errors. Note that $\ell = O(k)$, which implies that the the codeword $\mathbf{cw}$ is of size $O(k)$. Following the pre-processing performed in HSS (i.e., the filtering step), the servers obtain n additive secret shares, which we will write as the vector $\langle \mathbf{y} \rangle^\sigma := (\langle y_1 \rangle^\sigma \dots \langle y_n \rangle^\sigma)$, where $\mathbf{y}$ is a *sparse* vector (since, by assumption, at most k items are selected). Using the sketch, the parties can efficiently compute the codeword $\mathbf{cw} := H \cdot (\langle \mathbf{y} \rangle^\sigma)^\top$ over the secret shares, as it is a linear operation. The codeword $\mathbf{cw}$ can be efficiently decoded to the set of (up to) k items using the syndrome decoder of the error-correcting code.

Our idea is to offload the syndrome decoding and post-processing into the garbled circuit. Specifically, the garbled circuit first reconstructs the shares of $\mathbf{cw}$, then performs syndrome decoding over the codeword $\mathbf{cw}$ to recover the vector of items, and finally performs the post-processing over the selected items.

This simple framework combines three ingredients: (1) an HSS scheme, (2) any linear sketch based on syndrome decoding, and (3) a secret-shared garbled circuit. It immediately applies to several practical applications. We provide the following two concrete examples:

- **Threshold counting.** For a count query q specifying some predicate, the answer to the query is 0 if fewer than some threshold k items match the predicate, and 1 if more than k items match the predicate. Using our framework, we can easily instantiate a simple threshold counting scheme as follows. Let H be a parity check matrix for a linear code that corrects up to k errors and *detects* (i.e., output $\perp$) if $> k$ errors occur [94]. Then,
 - 1: Using HSS, compute the predicate specified in the query over each item in the database to obtain a secret share of an indicator bit b_i indicating whether the predicate was satisfied for the item or not.
 - 2: Compute the share of the codeword $\langle \mathbf{cw} \rangle^\sigma := H \cdot (\langle b_1 \rangle^\sigma, \dots, \langle b_n \rangle^\sigma)$.
 - 3: Output secret-shares of the garbled circuit that recovers the codeword $\mathbf{cw}$, decodes $\mathbf{cw}$, and outputs 0 if the decoding succeeds and 1 otherwise.

In this example, the garbled circuit outputs 1 if the syndrome decoding fails (indicating there were more than k items that matched the predicate) and 0 if the decoding succeeds (indicating there were fewer than k items that matched the predicate). The client learns nothing more than the output bit, thanks to the security property of the garbled circuit.

- **Hash-based data structures.** We can use our framework to query such hash-based data structures efficiently as follows:

- 1: Using HSS, compute an indicator bit b_i for each item in the database that matches the hash-based filtering predicate for the query.
- 2: Compute the share of the codeword $\langle \text{cw} \rangle^\sigma := H \cdot (\langle b_1 \rangle^\sigma, \dots, \langle b_n \rangle^\sigma)$.
- 3: Output secret-shares of the garbled circuit that recovers the codeword cw , decodes cw to get a vector of candidate items, and output the result of the post-processing function applied to the filtered subset of items.

In this example, H needs to be able to extract up to k “errors” where k is a bound on the number of items returned in the pre-processing step (e.g., in many applications, k is a small constant, e.g., 5 [47]). Once more, the client learns nothing more than the k decoded outputs, thanks to the security property of the garbled circuit.

8.2 Efficient zero-knowledge proofs for HSS

Recently, several techniques have been developed to allow multiple parties (called *verifiers*) to check properties over secret-shared data [22, 23, 45, 48]. In this setting, two (or more) verifiers have *secret shares* of the statement x and witness w , and would like to check that $(x, w) \in \mathcal{R}$, for some NP relation $\mathcal{R}$ corresponding to a language $\mathcal{L}$ (in the case where $\mathcal{L}$ is efficiently recognizable, the witness can be omitted). Using current techniques, the dealer must secret-share a separate proof π attesting to $(x, w) \in \mathcal{R}$ [22, 48].

In this section, we show how to instantiate a zero-knowledge proof for HSS input shares using a secret-shared garbled circuit. Using our proof system, two parties can efficiently check any property of their secret-shared data by exchanging only one message (shares of a corresponding garbled circuit and garbled input). Moreover, our approach can be applied to both efficiently recognizable languages and languages in NP (provided the dealer also shares a witness w in the latter case). Compared to simply using HSS itself to verify the relation homomorphically,⁸ our approach enables concretely efficient verification, does not require increasing the input share size, and is directly suitable for languages that are naturally represented by Boolean circuits.

To the best of our knowledge, we are the first to explore zero-knowledge proofs in the context of HSS in the standard model. All existing approaches for zero-knowledge proofs on secret-shared data (i.e., [22, 48]) (1) require exchanging two sequential messages between verifiers to verify the proof *or* working in the random oracle model and (2) are tailored towards verifying arithmetic languages. While it is possible to apply these proof systems to HSS (e.g., by first locally converting the input shares into additive shares and then checking the proof over the additive shares), the limitations outlined above still apply. Moreover, this approach is limited to verifying predicates that can be described by arithmetic branching programs (the computational class supported by HSS).

Our approach, on the other hand, requires minimal communication, since the verifiers only exchange one message (shares of the garbled circuit of the verification circuit) in the standard model, i.e., *without assuming a random oracle*,⁹ and are tailored to verifying *unstructured* languages. In particular, our approach is not limited to just verifying predicates that can be described by branching programs. Elaborating on the last point, as observed by Jawurek et al. [86] (and follow-up works, e.g., [41, 68, 69, 81]), using garbled circuits for zero-knowledge proofs removes the “algebraic” requirements associated with many zero-knowledge techniques and provides efficiency benefits when proving statements certain languages. For example, garbled circuits can enable better efficiency when proving statements

⁸ One trivial approach to verifying an HSS shared statement is to run the verification circuit inside HSS and outputting the resulting shares to reconstruct the verification result. This approach enables verifying any relation with a verification predicate that can be computed by a branching program.

⁹ A correlation-robust hash function suffices for constructing the garbled circuit.

such as “I know x such that $y = \text{Sha256}(x)$,” since the verification procedure (which involves recomputing Sha256) is more naturally expressed as a Boolean circuit [69].

Definition 36 (Zero-Knowledge proof over HSS inputs (zkHSS)). Let $\lambda \in \mathbb{N}$ be a security parameter. A zero-knowledge proof system for an NP relation $\mathcal{R} \subseteq \{0, 1\}^* \times \{0, 1\}^*$ instantiated between a prover and two verifiers who are given HSS input shares of the statement x , consists of four (possibly randomized) algorithms $\text{zkHSS} = (\text{Setup}, \text{Prove}, \text{Audit}, \text{Verify})$ with the following functionality:

- $\text{Setup}(1^\lambda) \rightarrow (\text{pk}, (\text{ak}_A, \text{ak}_B))$. The randomized setup algorithm takes as input the security parameter λ . It outputs a common public key pk and a pair of audit keys $(\text{ak}_A, \text{ak}_B)$.
- $\text{Prove}(\text{pk}, x, w) \rightarrow (\pi_A, \pi_B)$. The randomized prove algorithm takes as input the common public key pk , a statement $x \in \mathcal{L}$ and witness $w \in \{0, 1\}^*$. It outputs a pair of proof shares (π_A, π_B) .
- $\text{Audit}(\text{pk}, \text{ak}_\sigma, \llbracket x \rrbracket^\sigma, \pi_\sigma) \rightarrow \tau_\sigma$. The deterministic audit algorithm takes as input the common public key pk , an audit key ak_σ , an HSS input share $\llbracket x \rrbracket^\sigma$ of the statement x , and a proof share π_σ . It outputs a verification string τ_σ .
- $\text{Verify}(\text{pk}, \tau_A, \tau_B) \rightarrow 0/1$. The deterministic verification algorithm takes as input the common public key pk and a pair of verification strings. It outputs 1 for accept, or 0 for reject.

The above functionality must satisfy completeness, soundness, and zero-knowledge:

- **Completeness.** For all well-formed HSS input shares $\llbracket x \rrbracket := (\llbracket x \rrbracket^A, \llbracket x \rrbracket^B)$ of a statement $x \in \mathcal{L}$ and all witnesses $w \in \{0, 1\}^*$ such that $(x, w) \in \mathcal{R}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{pk}, \tau_A, \tau_B) = 1 : \\ \begin{array}{l} (\text{pk}, (\text{ak}_A, \text{ak}_B)) \leftarrow \text{Setup}(1^\lambda) \\ (\pi_A, \pi_B) \leftarrow \text{Prove}(\text{pk}, x, w) \\ \tau_A \leftarrow \text{Audit}(\text{pk}, \text{ak}_A, \llbracket x \rrbracket^A, \pi_A) \\ \tau_B \leftarrow \text{Audit}(\text{pk}, \text{ak}_B, \llbracket x \rrbracket^B, \pi_B) \end{array} \end{array} \right] = 1,$$

where the probability is taken over the randomness of Prove and Setup .

- **Soundness.** A zkHSS proof system is sound against a malicious prover and semi-honest verifiers, if for all efficient adversaries $\mathcal{A}$, and all $x \in \{0, 1\}^*$ such that $x \notin \mathcal{L}$, there exists a negligible function negl such that

$$\Pr \left[\begin{array}{l} \text{Decode}(\text{ek}^A, \text{ek}^B, (\llbracket x \rrbracket^A, \llbracket x \rrbracket^B)) = x \\ \wedge \\ \text{Verify}(\text{pk}, \tau_A, \tau_B) = 1 \end{array} : \begin{array}{l} (\text{pk}, (\text{ak}_A, \text{ak}_B)) \leftarrow \text{Setup}(1^\lambda) \\ (\llbracket x \rrbracket^A, \llbracket x \rrbracket^B, \pi_A, \pi_B) \leftarrow \mathcal{A}(\text{pk}, x) \\ \tau_A \leftarrow \text{Audit}(\text{pk}, \text{ak}_A, \llbracket x \rrbracket^A, \pi_A) \\ \tau_B \leftarrow \text{Audit}(\text{pk}, \text{ak}_B, \llbracket x \rrbracket^B, \pi_B) \end{array} \right] \leq \text{negl}(\lambda),$$

probability is taken over the randomness of $\mathcal{A}$ and Setup , and where ek^A and ek^B are the evaluation keys associated with the HSS scheme (Decode is as defined in Definition 12).

- **Zero-knowledge.** A zkHSS scheme is zero-knowledge if there exists an efficient simulator $\mathcal{S}$ such that for all distinct $\sigma, \bar{\sigma} \in \{A, B\}$, $\{\llbracket x \rrbracket^\sigma, \pi_\sigma, \text{ak}_\sigma, \tau_{\bar{\sigma}}\} \approx_c \mathcal{S}(1^\lambda, \text{pk})$. In words, the view of one verifier reveals no information on the HSS-shared statement x or witness w .

Construction 7 (Zero-knowledge proofs for HSS).

Parameters. Let $\text{GI} = (\text{GI.Setup}, \text{GI.Encode}, \text{GI.SelectLabels})$ be a secret-shared input garbling scheme with input space $\mathbb{F}_{2^\lambda}$, instantiated by [Construction 5](#) using an HSS scheme $\text{HSS} = (\text{HSS.Setup}, \text{HSS.Input}, \text{HSS.Eval})$. Let $\text{GPCF} = (\text{GPCF.Gen}, \text{GPCF.Eval})$ be a GPCF for a secret-sharing-friendly garbling scheme $\text{GC} = (\text{GC.SampLabels}_\sigma, \text{GC.Garble}, \text{GC.Encode}, \text{GC.Decode})$, and $\mathcal{R}$ be an NP relation with verification circuit $\mathcal{C}$, such that $\mathcal{C}(x, w) = 1$ iff $(x, w) \in \mathcal{R}$. Let ctr be a monotonically increasing counter.

$\text{zkHSS.Setup}(1^\lambda)$:

- 1: $\Delta^A, \Delta^B \leftarrow_R \mathbb{F}_{2^\lambda}$
- 2: $(k^A, k^B) \leftarrow \text{GPCF.Gen}(1^\lambda, (\Delta^A, \Delta^B))$
- 3: $(\text{pk}, (\text{gik}^A, \text{gik}^B)) \leftarrow \text{GI.Setup}(1^\lambda, \Delta^A, \Delta^B)$
- 4: $\text{ak}_A := (A, k^A, \text{gik}^A), \quad \text{ak}_B := (B, k^B, \text{gik}^B)$
- 5: **return** $(\text{pk}, (\text{ak}_A, \text{ak}_B))$

$\text{zkHSS.Prove}(\text{pk}, x, w)$:

- 1: $(\llbracket w \rrbracket^A, \llbracket w \rrbracket^B) \leftarrow \text{GI.Encode}(\text{pk}, w)$
- 2: **return** $(\llbracket w \rrbracket^A, \llbracket w \rrbracket^B)$

$\text{zkHSS.Audit}(\text{pk}, \text{ak}_\sigma, \llbracket x \rrbracket^\sigma, \pi_\sigma)$:

- 1: $\text{ak}_\sigma = (\sigma, k^\sigma, \text{gik}^\sigma), \quad \pi_\sigma = \llbracket w \rrbracket^\sigma$
- 2: $(\Delta^\sigma, \mathbf{L}_0^\sigma, \langle \hat{\mathcal{C}} \rangle^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \Delta^A \cdot \mathbf{r} \rangle^\sigma, \langle \Delta^B \cdot \mathbf{r} \rangle^\sigma) \leftarrow \text{GPCF.Eval}(\sigma, k^\sigma, \mathcal{C}, \text{ctr})$
- 3: $\llbracket (x, w) \rrbracket^\sigma := (\llbracket x_1 \rrbracket^\sigma, \dots, \llbracket x_{s_x} \rrbracket^\sigma, \llbracket w_1 \rrbracket^\sigma, \dots, \llbracket w_{s_w} \rrbracket^\sigma)$
- 4: $\langle \hat{X} \rangle^\sigma \leftarrow \text{GI.SelectLabels}(\sigma, \text{gik}^\sigma, \llbracket (x, w) \rrbracket^\sigma, \mathbf{L}_0^\sigma, \langle \mathbf{r} \rangle^\sigma, \langle \mathbf{r} \Delta^A \rangle^\sigma, \langle \mathbf{r} \Delta^B \rangle^\sigma)$
- 5: $\text{ctr} := \text{ctr} + 1$
- 6: **return** $\tau_\sigma := (\langle \hat{\mathcal{C}} \rangle^\sigma, \langle \hat{X} \rangle^\sigma)$

$\text{zkHSS.Verify}(\text{pk}, \tau_A, \tau_B)$:

- 1: **parse** $\tau_\sigma = (\langle \hat{\mathcal{C}} \rangle^\sigma, \langle \hat{X} \rangle^\sigma)$ for $\sigma \in \{A, B\}$
- 2: $\hat{\mathcal{C}} := \langle \hat{\mathcal{C}} \rangle^A + \langle \hat{\mathcal{C}} \rangle^B$
- 3: $\hat{X} := \langle \hat{X} \rangle^A + \langle \hat{X} \rangle^B$
- 4: $b \leftarrow \text{GC.Decode}(\mathcal{C}, \hat{\mathcal{C}}, \hat{X})$
- 5: **return** b

Construction 7: Zero-knowledge proofs for HSS.

Proposition 37. Assuming the correctness and security of: (1) [Construction 5](#) instantiated using a homomorphic secret sharing scheme HSS, (2) the garbling pseudorandom correlation function, and (3) the secret-sharing-friendly garbled circuit, [Construction 7](#) is a zero-knowledge proof on secret-shared data satisfying the properties of [Definition 36](#).

Proof. We prove each property in turn.

Completeness. Let $(\llbracket w \rrbracket^A, \llbracket w \rrbracket^B)$ output by $\text{HSS.Input}(\text{pk}, w)$. By the correctness of the Garbling PCF, $\langle \hat{\mathcal{C}} \rangle^\sigma$ is an additive secret share of the relation verification circuit $\mathcal{C}$. By the correctness of the input garbling functionality (instantiated using [Construction 5](#)), $\langle \hat{X} \rangle^\sigma$ is a share of the garbled input. Then, it follows that $\hat{\mathcal{C}}$ and $\hat{X}$, as computed in zkHSS.Verify , consist of a valid garbled circuit $\mathcal{C}$ computing the relation $\mathcal{R}$, and $\hat{X}$ is a valid garbling of the input $x \parallel w$. It follows that $b = \mathcal{R}(x, w)$ by the correctness of GC.Decode .

Soundness. We prove the stronger property of *special soundness* by constructing the following efficient extractor $\mathcal{E}$ that outputs a witness from any accepting proof (π_A, π_B) . On input $(\mathbf{pk}, \mathbf{ak}_A, \mathbf{ak}_B, \llbracket x \rrbracket^A, \llbracket x \rrbracket^B, \pi_A, \pi_B, \text{ctr})$, $\mathcal{E}$ proceeds as follows.

- 1: $\mathbf{ak}_\sigma = (\sigma, \mathbf{k}^\sigma, \mathbf{ek}^\sigma)$, $\pi_\sigma = \llbracket w \rrbracket^\sigma$, for $\sigma \in \{A, B\}$.
- 2: Compute $\langle w' \rangle^\sigma \leftarrow \text{HSS.Convert}(\mathbf{ak}_\sigma, \llbracket w \rrbracket^\sigma)$.
- 3: Compute $w' := \langle w' \rangle_A \oplus \langle w' \rangle_B$.
- 4: Output w' .

We now argue that the extractor succeeds with probability $1 - \text{negl}(\lambda)$ by the correctness of HSS and the garbled circuit, which proves special soundness.

Suppose that the witness w' extracted from the HSS share (which may correspond to an arbitrary value in the HSS message space in the case where the HSS input share is ill-formed by the malicious prover), does not satisfy $(x, w') \in \mathcal{R}$.

By inspecting `Gl.SelectLabels` (which is called internally in `zkHSS.Audit` and is the only part that depends on $\llbracket w \rrbracket^\sigma$), we have that the input encoded in $\hat{X}$ either corresponds to (1) *some* valid input w' in the case where the HSS input share is well-formed under the HSS public key $\mathbf{pk}$ or (2) an arbitrary undefined value r in the case where the HSS input share is ill-formed under $\mathbf{pk}$. We note that `HSS.Convert` and `HSS.Mult` always outputs shares of some value in the HSS message space, even for ill-formed input shares, albeit these values may not be consistent.¹⁰

By the correctness property of the garbled circuit, in case (1) it has to be the case that the $(x, w') \in \mathcal{R}$, and so the extraction is successful.

Now, in case (2), suppose, towards contradiction, that $\text{zkHSS.Verify}(\tau_A, \tau_B) = 1$ but the HSS input share is corrupted. Then, inspecting `Gl.SelectLabels`, note that a malicious prover has no control over the choice of labels, masks, or Δ^A, Δ^B . For a corrupted HSS input share, `Gl.SelectLabels` does not produce shares of any valid selection of input labels, since that would imply `HSS.Convert` and `HSS.Mult` produce consistent values, contradicting the assumption that the input shares were ill-formed. In turn, we have that the label selection must be inconsistent with the garbled circuit, which will cause decoding to fail with all but negligible probability; this contradicts the assumption that $\text{zkHSS.Verify}(\tau_A, \tau_B) = 1$.

This concludes the proof of special soundness.

Zero-knowledge. Zero-knowledge is proven via a sequence of hybrid games.

- *Hybrid $\mathcal{H}_0$.* This hybrid is identical to the real view of the verifier σ in [Construction 7](#) and consists of the share $\llbracket x \rrbracket^\sigma$, audit key $\mathbf{ak}_\sigma$, proof share π_σ , and the audit token of the other verifier $\tau_{\bar{\sigma}}$, where $\bar{\sigma} \in \{A, B\} \setminus \{\sigma\}$.
- *Hybrid $\mathcal{H}_1$.* In this hybrid, the share $\llbracket x \rrbracket^\sigma$ is replaced with an HSS input share of zero $\llbracket 0 \rrbracket^\sigma$ of appropriate length. $\mathcal{H}_1$ is computationally indistinguishable from $\mathcal{H}_0$ by the security property of HSS (see [Definition 11](#)).
- *Hybrid $\mathcal{H}_2$.* In this hybrid, the audit token $\tau_{\bar{\sigma}}$ is computed as $\tau_{\bar{\sigma}} := (\hat{\mathcal{C}}, \hat{X}) - \tau_\sigma$, where $(\hat{\mathcal{C}}, \mathbf{L}, \mathbf{r}) \leftarrow \text{GC.Garble}(1^\lambda, \mathcal{C})$ and $\hat{X} \leftarrow \text{GC.Encode}(w, \mathbf{L}, \mathbf{r})$. This hybrid is indistinguishable from $\mathcal{H}_1$ by the security definition of a TAPCF ([Definition 20](#)). To see this, note that we can construct an efficient simulator $\mathcal{S}$ that, given the witness w , samples $\mathbf{pk}$ and $\mathbf{ak}_\sigma$ according to `zkHSS.Setup`, computes π_σ to be an HSS input share $\llbracket w \rrbracket^\sigma$ of the witness using the HSS scheme, computes τ_σ using $\mathbf{ak}_\sigma$, and then sets $\tau_{\bar{\sigma}}$ as above. Indistinguishability follows directly from the TAPCF security game.

¹⁰ More generally, and without loss of generality, the HSS message space can be extended to have a special symbol $\perp$ that is output for ill-formed input shares.

- *Hybrid $\mathcal{H}_3$* . In this hybrid, $(\widehat{\mathcal{C}}, \widehat{X})$ are generated by invoking the simulator $\mathcal{S}_{\text{GC}}$ of the garbling scheme GC (Definition 40), where the output of $\text{GC.Decode}(\widehat{\mathcal{C}}, \widehat{X}) = 1$ (i.e., since the verification passes). Specifically, there is an efficient simulator $\mathcal{S}$ that, given the witness w , samples $(\text{pk}, \llbracket x \rrbracket^\sigma, \pi_\sigma, \text{ak}_\sigma)$ as in $\mathcal{H}_2$, computes τ_σ using ak_σ , and then sets $\tau_{\widehat{\sigma}} := (\widehat{\mathcal{C}}, \widehat{X}) - \tau_\sigma$, where $\widehat{X}$ is now independent of w . This hybrid is indistinguishable from $\mathcal{H}_2$ by the security of the garbling scheme.
- *Hybrid $\mathcal{H}_4$* . In this hybrid, the HSS input shares of w (output by zkHSS.Prove as $\pi_\sigma := \llbracket w \rrbracket^\sigma$, for $\sigma \in \{A, B\}$) are replaced with HSS shares of zero $\llbracket 0 \rrbracket$ (of appropriate length). This hybrid is indistinguishable from $\mathcal{H}_3$ by the security of the HSS scheme. Specifically, there is an efficient simulator $\mathcal{S}$ that, on input λ , samples $(\text{pk}, \llbracket x \rrbracket^\sigma, \text{ak}_\sigma, \tau_{\widehat{\sigma}})$ as in $\mathcal{H}_3$, and sets π_σ as an HSS share of zero $\llbracket 0 \rrbracket^\sigma$. $\mathcal{H}_4$ no longer depends on the witness w and is computationally indistinguishable to $\mathcal{H}_3$ by the security of the HSS scheme.
- *Hybrid $\mathcal{H}_5$* . This hybrid is the simulated view of the verifier $(\llbracket x \rrbracket^\sigma, \pi_\sigma, \text{ak}_\sigma, \tau_{\widehat{\sigma}})$ is sampled as in $\mathcal{H}_4$. $\mathcal{H}_5$ is identical to $\mathcal{H}_4$, concluding the proof. □

8.2.1 Application to verifying properties over secret-shared inputs An immediate application of zkHSS is to verify properties over secret-shared data. For example, in many telemetry, federated learning, and anonymous communication applications [3, 46, 48, 49, 63, 97, 105], clients send secret shares to two non-colluding servers who then aggregate the shares. Because these applications assume that the clients are malicious, the servers processing the secret shares need to verify that certain predicates are satisfied (e.g., clients are sending secret shares of a low-norm vector). Using our zkHSS instantiations, the servers can verify the necessary properties in zero-knowledge. Compared to just using maliciously-secure HSS [1], the zkHSS construction provides a more concretely-efficient approach to verifying properties over HSS input shares, as we explain next.

Performance estimates. As a concrete example, we consider using zkHSS to prove that an AES encryption C decrypts to a message M , where the ciphertext C and message M are secret shared. Informally, this means proving the statement “I know a key K such that $\text{AES}(K, C) = M$.” The AES circuit has 6,800 AND gates [57]. Instantiated in the semi-honest verifier setting, and using our TAPCF-based Garbling PCF construction in the aggressive parameter setting, would require several seconds to generate the audit tokens, where each token is roughly 2 MB in size.

8.3 Non-interactive secure computation

Two rounds of interaction are known to be necessary and sufficient for secure multiparty computation [79]. Minimizing interaction is one of the most fundamental goals in secure computation. Toward pushing interactions to the bare minimum, many works [4, 8–10, 18, 42, 42, 53, 66, 73, 79, 84, 85, 88, 93, 95, 96] have explored the intriguing possibility of achieving two-round secure computation with a *reusable first round*. In this model, after an initial round of commitment to (a subset of) their inputs, the parties can securely evaluate different functions f_i on the committed inputs using a single round of interaction for each new f_i . Here, we consider more specifically a model for two-party secure computation with a reusable first round. Such a notion of secure computation (with a reusable first round) has been coined *two-sided non-interactive secure computation* (two-sided NISC) in a recent work of Ishai, Khurana, Sahai, and Srinivasan [85], where they give a construction (in the

malicious setting) using black-box access to a two-round oblivious transfer and an additively-homomorphic encryption scheme, in the random oracle model.

Previous works on two-sided NISC Ishai, Khurana, Sahai, and Srinivasan [85] were focused on the question of achieving security against malicious adversaries. The main reason is that two-sided NISC in the semi-honest setting is straightforward: combining any garbling scheme (e.g., Yao’s scheme) with a 2-round oblivious transfer yields a two-sided NISC, where the first oblivious transfer round “commits” to the input of one party, and the second round is reusable. In this work, we focus on a different goal: two-sided NISC with public disclosure. We elaborate below.

Two-sided NISC with non-interactive disclosure. In a traditional two-sided NISC between Alice and Bob, only Alice and Bob receive an output: if they intend to reveal the output to a third party, this requires an additional round of communication. In this work, we ask whether it is possible to achieve two-sided NISC where the output can be revealed to external parties (or even made public) *without* requiring an additional round of computation. While our focus is on the semi-honest setting (where, unlike traditional two-sided NISC, the question is already highly non-trivial), we note that this is also well-motivated in the malicious setting: even if one is satisfied with using an additional round, a malicious two-sided NISC would require the participants to *prove* to the external party that the output is correct, which can incur a major overhead. In contrast, a maliciously-secure solution without any additional round could “natively” guarantee correctness of the output to an external party, with no added overhead.

We show that our notion of GPCF enables constructions of two-sided NISCs with non-interactive disclosure: When evaluating the function f_i in a single round, Alice and Bob can freely decide, for each function f_i , whether $f_i(x_A, x_B)$ will be revealed to Alice, to Bob, *or made public*¹¹. In the latter case, any external observer can, observing the simultaneous interaction (or reading the messages posted on a public bulletin board), reconstruct $f(x_A, x_B)$. To our knowledge, the only previous works to obtain this public disclosure property are Mr-NISCs [18, 19], but these constructions are mostly of theoretical interest. Our construction enables the first feasible solution to this problem.

We describe this application in the context of semi-honest and maliciously-secure non-interactive secure computation. At a high level, the protocol is split into a trusted setup, a one-time input encoding, and a function evaluation:

- **Setup:** The two parties, Alice and Bob, receive the GPCF and input encoding keys that allow them to generate fresh encodings of a garbled circuit for a function f_i .
- **Input Encoding:** In a single round of simultaneous interaction, Alice and Bob exchange *encodings* of their respective private inputs x_A, x_B , using [Construction 5](#).
- **Evaluation:** Given a public function f_i , Alice and Bob, in a single round of simultaneous interaction, securely compute $f_i(x_A, x_B)$. This phase can be repeated for an arbitrary number of functions f_i .

In [Construction 8](#), we formalize the above sketch of a two-sided NISC *with non-interactive disclosure*, based on our GPCF ([Definition 28](#)) and label selection ([Definition 32](#)) constructions. We describe a path for obtaining a maliciously-secure version of this NISC in [Section 8.3.1](#), albeit this upgrade comes at the cost of concrete efficiency.

Theorem 38 (Informal). *Assuming the correctness and security of the TACPF ([Definition 20](#)) and the existence of a public-key HSS scheme satisfying [Definition 11](#), the NISC*

¹¹ Of course, by encrypting their “public” message with the public key of Charlie, Alice and Bob can also reveal the output to a designated party, Charlie, in one round.

Construction 8 (NISC with Non-interactive Disclosure).

Parameters. Let GC be a secret-sharing friendly garbled circuit scheme, and $(\text{GPCF.Gen}, \text{GPCF.Eval})$ be a Garbling PCF for GC . Let $\text{GI} = (\text{GI.Setup}, \text{GI.Encode}, \text{GI.SelectLabels})$ be an secret-shared input garbling scheme. Let ctr be a monotonically increasing counter.

Trusted Setup.

- 1: Sample $\Delta^A, \Delta^B \leftarrow_R \mathbb{F}_{2^\lambda}$ and run $(k^A, k^B) \leftarrow \text{GPCF.Gen}(1^\lambda, (\Delta^A, \Delta^B))$.
- 2: Run $(\text{pk}, \text{gik}^A, \text{gik}^B) \leftarrow \text{GI.Setup}(1^\lambda, \Delta^A, \Delta^B)$.
- 3: Output (gik^A, k^A) to Alice and (gik^B, k^B) to Bob.

Round 1: Input Encoding. Parties locally encode their inputs as: $(\llbracket x_\sigma \rrbracket^A, \llbracket x_\sigma \rrbracket^B) \leftarrow \text{GI.Encode}(\text{gik}^\sigma, x_\sigma)$.

- 1: Alice stores $\llbracket x_A \rrbracket^A$ and sends $\llbracket x_A \rrbracket^B$ to Bob;
- 2: Bob stores $\llbracket x_B \rrbracket^B$ and sends $\llbracket x_B \rrbracket^A$ to Alice.

Round 2: Function Evaluation. For each new function f_i represented by a Boolean circuit $\mathcal{C}_i$, the parties proceed as follows:

- 1: $(\Delta^\sigma, L^\sigma, \langle \hat{\mathcal{C}}_i \rangle^\sigma, \langle r \rangle^\sigma, \langle r \Delta^A \rangle^\sigma, \langle r \Delta^B \rangle^\sigma) \leftarrow \text{GPCF.Eval}(\sigma, k^\sigma, \mathcal{C}_i, \text{ctr})$.
- 2: $(\hat{X})^\sigma \leftarrow \text{GI.SelectLabels}(\sigma, \text{gik}^\sigma, (\llbracket x_A \rrbracket^\sigma, \llbracket x_B \rrbracket^\sigma), L_0^\sigma, \langle r \rangle^\sigma, \langle r \Delta^A \rangle^\sigma, \langle r \Delta^B \rangle^\sigma)$ and broadcast $(\mathcal{C}_i, \langle \hat{\mathcal{C}}_i \rangle^\sigma, (\hat{X})^\sigma)$.
- 3: $\text{ctr} := \text{ctr} + 1$.
- 4: Anyone can decode $\hat{\mathcal{C}}_i$ and $\hat{X}_i$ from the posted shares of each party and decode the output $f(x_A, x_B)$.

Construction 8: Construction of NISC with non-interactive disclosure from GPCFs.

construction in [Construction 8](#) securely computes shares of $f(x_A, x_B)$ in two rounds, where the second round is reusable with different functions f_i .

Proof sketch. We prove correctness and privacy in turn.

Correctness. By the correctness of the input encoding ([Definition 32](#)), both parties hold encodings of x_A and x_B after round 1. Moreover, both parties obtain correct shares of the garbled input $\hat{X}$. By the correctness property of the GPCF ([Definition 28](#)), it holds that both parties get a share of the garbled circuit $\hat{\mathcal{C}}$ computing f_i . Now, from the shares of $(\hat{\mathcal{C}}_i, \hat{X})$ published by each party, anyone can recover $\hat{\mathcal{C}}_i$ and $\hat{X}$ (in the clear) and decode $\hat{\mathcal{C}}_i(\hat{X}) = f_i(x_A, x_B)$.

Privacy. Privacy follows from [Definition 32](#), the pseudorandomness and security property of the GPCF ([Definition 28](#)), and security of garbled circuits. In particular, the [Definition 32](#) and [Definition 28](#) guarantee that $(\hat{\mathcal{C}}_i, \hat{X})$ are well-formed shares of a garbled circuit and garbled input computing $f_i(x_A, x_B)$. Then, by invoking the security of garbled circuits, we have that there exists an efficient simulator for the distribution of $(\hat{\mathcal{C}}_i, \hat{X})$ given only $(\mathcal{C}_i, f_i(x_A, x_B))$. $\square$

8.3.1 Towards malicious security Most works from the literature on malicious NISCs [[4](#), [60](#), [84](#), [93](#)] fail to achieve security against selective failure attacks, and the few state-of-the-art constructions that achieve the strongest notion of security are very involved [[42](#), [85](#)]. Moreover, as of today, all known NISC protocols that build on the notion of authenticated garbled circuit are insecure against selective failure attacks (in other words, a malicious garbler can learn information about the evaluator's input if it learns whether evaluation was successful).

Using a GPCF for an authenticated garbling scheme (which we sketch in [Appendix E](#)) immediately yields a maliciously secure variant of our semi-honest two-sided NISC with non-interactive disclosure. In particular, as with standard authenticated garbled circuits, if

one party provides the wrong share of the garbled circuit, then the authentication check is guaranteed to fail with high probability [60, 107]. Importantly, unlike previous NISC from authenticated garbling, the protocol obtained using a GPCF remains secure against selective failure attacks. At a high level, the reason for this stems from the fact that the permutation bits, which hide which row of a gate corresponds to which input, are pseudorandomly refreshed in each new instance (in contrast, previous NISC from authenticated garbling had to reuse them [60, 107]). Hence, selective failure attacks only reveal values masked by fresh pseudorandom bits.

Acknowledgments

We thank the anonymous reviewers for significant help in improving the presentation and results in this paper. We thank Aditya Hegde for helpful comments and pointers. Geoffroy Couteau and Alexander Koch acknowledge the support of the French Agence Nationale de la Recherche (ANR), under grant ANR-20-CE39-0001 (project SCENE) and under the France 2030 ANR Project ANR-22-PECY-003 SecureCompute. This work is supported by ERC grant OBELiSC (101115790), and NSF grant 2330065.

Bibliography

- [1] Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 421–452. Springer, Cham, August 2022. https://doi.org/10.1007/978-3-031-15985-5_15.
- [2] Damiano Abram, Ariel Nof, Claudio Orlandi, Peter Scholl, and Omer Shlomovits. Low-bandwidth threshold ECDSA via pseudorandom correlation generators. In *2022 IEEE Symposium on Security and Privacy*, pages 2554–2572. IEEE Computer Society Press, May 2022. <https://doi.org/10.1109/SP46214.2022.9833559>.
- [3] Surya Addanki, Kevin Garbe, Eli Jaffe, Rafail Ostrovsky, and Antigoni Polychroniadou. Prio+: Privacy preserving aggregate statistics via boolean shares. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22: 13th International Conference on Security in Communication Networks*, volume 13409 of *Lecture Notes in Computer Science*, pages 516–539. Springer, Cham, September 2022. https://doi.org/10.1007/978-3-031-14791-3_23.
- [4] Arash Afshar, Payman Mohassel, Benny Pinkas, and Ben Riva. Non-interactive secure computation based on cut-and-choose. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 387–404. Springer, Berlin, Heidelberg, May 2014. https://doi.org/10.1007/978-3-642-55220-5_22.
- [5] Kahraman Akdemir, Martin Dixon, Wajdi Feghali, Patrick Fay, Vinodh Gopal, Jim Guilford, Erdinc Ozturk, Gil Wolrich, and Ronen Zohar. Breakthrough AES performance with Intel AES new instructions. *White paper*, June, 12:217, 2010.
- [6] Alexandr Andoni and Piotr Indyk. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Communications of the ACM*, 51(1):117–122, 2008.
- [7] Thomas Attema, Pedro Capitão, and Lisa Kohl. On homomorphic secret sharing from polynomial-modulus LWE. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 13941 of *Lecture Notes in Computer Science*, pages 3–32. Springer, Cham, May 2023. https://doi.org/10.1007/978-3-031-31371-4_1.
- [8] Saikrishna Badrinarayanan, Abhishek Jain, Rafail Ostrovsky, and Ivan Visconti. Non-interactive secure computation from one-way functions. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part III*, volume 11274 of *Lecture Notes in Computer Science*, pages 118–138. Springer, Cham, December 2018. https://doi.org/10.1007/978-3-030-03332-3_5.
- [9] James Bartusek, Sanjam Garg, Daniel Masny, and Pratyay Mukherjee. Reusable two-round MPC from DDH. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part II*, volume 12551 of *Lecture Notes in Computer Science*, pages 320–348. Springer, Cham, November 2020. https://doi.org/10.1007/978-3-030-64378-2_12.
- [10] James Bartusek, Sanjam Garg, Akshayaram Srinivasan, and Yinuo Zhang. Reusable two-round MPC from LPN. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 13177 of *Lecture Notes in Computer Sci-*

- ence, pages 165–193. Springer, Cham, March 2022. https://doi.org/10.1007/978-3-030-97121-2_7.
- [11] Carsten Baum, Lennart Braun, Alexander Munch-Hansen, Benoît Razet, and Peter Scholl. Appenzeller to brie: Efficient zero-knowledge proofs for mixed-mode arithmetic and $\mathbb{Z}_2^k$. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 192–211. ACM Press, November 2021. <https://doi.org/10.1145/3460120.3484812>.
 - [12] Carsten Baum, Alex J. Malozemoff, Marc B. Rosen, and Peter Scholl. Mac’n’cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part IV*, volume 12828 of *Lecture Notes in Computer Science*, pages 92–122, Virtual Event, August 2021. Springer, Cham. https://doi.org/10.1007/978-3-030-84259-8_4.
 - [13] Carsten Baum, Lennart Braun, Alexander Munch-Hansen, and Peter Scholl. Moz $\mathbb{Z}_{2^k}$ arella: Efficient vector-OLE and zero-knowledge proofs over $\mathbb{Z}_{2^k}$. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 329–358. Springer, Cham, August 2022. https://doi.org/10.1007/978-3-031-15985-5_12.
 - [14] Donald Beaver. Efficient multiparty protocols using circuit randomization. In Joan Feigenbaum, editor, *Advances in Cryptology – CRYPTO’91*, volume 576 of *Lecture Notes in Computer Science*, pages 420–432. Springer, Berlin, Heidelberg, August 1992. https://doi.org/10.1007/3-540-46766-1_34.
 - [15] Donald Beaver, Silvio Micali, and Phillip Rogaway. The round complexity of secure protocols (extended abstract). In *22nd Annual ACM Symposium on Theory of Computing*, pages 503–513. ACM Press, May 1990. <https://doi.org/10.1145/100216.100287>.
 - [16] Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012: 19th Conference on Computer and Communications Security*, pages 784–796. ACM Press, October 2012. <https://doi.org/10.1145/2382196.2382279>.
 - [17] Aner Ben-Efraim, Michael Nielsen, and Eran Omri. Turbospeedz: Double your online SPDZ! Improving SPDZ using function dependent preprocessing. In Robert H. Deng, Valérie Gauthier-Umaña, Martín Ochoa, and Moti Yung, editors, *ACNS 2019: 17th International Conference on Applied Cryptography and Network Security*, volume 11464 of *Lecture Notes in Computer Science*, pages 530–549. Springer, Cham, June 2019. https://doi.org/10.1007/978-3-030-21568-2_26.
 - [18] Fabrice Benhamouda and Huijia Lin. Mr NISC: Multiparty reusable non-interactive secure computation. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part II*, volume 12551 of *Lecture Notes in Computer Science*, pages 349–378. Springer, Cham, November 2020. https://doi.org/10.1007/978-3-030-64378-2_13.
 - [19] Fabrice Benhamouda, Aayush Jain, Ilan Komargodski, and Huijia Lin. Multiparty reusable non-interactive secure computation from LWE. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 724–753. Springer, Cham, October 2021. https://doi.org/10.1007/978-3-030-77886-6_25.
 - [20] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
 - [21] Maxime Bombar, Geoffroy Couteau, Alain Couvreur, and Clément Ducros. Correlated pseudorandomness from the hardness of quasi-abelian decoding. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*,

- volume 14084 of *Lecture Notes in Computer Science*, pages 567–601. Springer, Cham, August 2023. https://doi.org/10.1007/978-3-031-38551-3_18.
- [22] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Zero-knowledge proofs on secret-shared data via fully linear PCPs. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 67–97. Springer, Cham, August 2019. https://doi.org/10.1007/978-3-030-26954-8_3.
 - [23] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Arithmetic sketching. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 171–202. Springer, Cham, August 2023. https://doi.org/10.1007/978-3-031-38557-5_6.
 - [24] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 337–367. Springer, Berlin, Heidelberg, April 2015. https://doi.org/10.1007/978-3-662-46803-6_12.
 - [25] Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part I*, volume 9814 of *Lecture Notes in Computer Science*, pages 509–539. Springer, Berlin, Heidelberg, August 2016. https://doi.org/10.1007/978-3-662-53018-4_19.
 - [26] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1292–1303. ACM Press, October 2016. <https://doi.org/10.1145/2976749.2978429>.
 - [27] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, and Michele Orrù. Homomorphic secret sharing: Optimizations and applications. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 2105–2122. ACM Press, October / November 2017. <https://doi.org/10.1145/3133956.3134107>.
 - [28] Elette Boyle, Niv Gilboa, Yuval Ishai, Huijia Lin, and Stefano Tessaro. Foundations of homomorphic secret sharing. Cryptology ePrint Archive, Report 2017/1248, 2017. URL <https://eprint.iacr.org/2017/1248>.
 - [29] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 896–912. ACM Press, October 2018. <https://doi.org/10.1145/3243734.3243868>.
 - [30] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 489–518. Springer, Cham, August 2019. https://doi.org/10.1007/978-3-030-26954-8_16.
 - [31] Elette Boyle, Niv Gilboa, and Yuval Ishai. Secure computation with preprocessing via function secret sharing. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019: 17th Theory of Cryptography Conference, Part I*, volume 11891 of *Lecture Notes in Computer Science*, pages 341–371. Springer, Cham, December 2019. https://doi.org/10.1007/978-3-030-36030-6_14.
 - [32] Elette Boyle, Lisa Kohl, and Peter Scholl. Homomorphic secret sharing from lattices without FHE. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology*

- tology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 3–33. Springer, Cham, May 2019. https://doi.org/10.1007/978-3-030-17656-3_1.
- [33] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from ring-LPN. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part II*, volume 12171 of *Lecture Notes in Computer Science*, pages 387–416. Springer, Cham, August 2020. https://doi.org/10.1007/978-3-030-56880-1_14.
 - [34] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Correlated pseudorandom functions from variable-density LPN. In *61st Annual Symposium on Foundations of Computer Science*, pages 1069–1080. IEEE Computer Society Press, November 2020. <https://doi.org/10.1109/FOCS46700.2020.00103>.
 - [35] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Nicolas Resch, and Peter Scholl. Correlated pseudorandomness from expand-accumulate codes. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 603–633. Springer, Cham, August 2022. https://doi.org/10.1007/978-3-031-15979-4_21.
 - [36] Lennart Braun, Geoffroy Couteau, Kelsey Melissaris, Mahshid Riahinia, and Elahe Sadeghi. Fast pseudorandom correlation functions from sparse LPN. In *ASIACRYPT 2025*, LNCS. Springer, 2025.
 - [37] Andreas Brüggemann, Robin Hundt, Thomas Schneider, Ajith Suresh, and Hossein Yalame. FLUTE: Fast and secure lookup table evaluations. In *2023 IEEE Symposium on Security and Privacy*, pages 515–533. IEEE Computer Society Press, May 2023. <https://doi.org/10.1109/SP46215.2023.10179345>.
 - [38] Chris Brzuska, Geoffroy Couteau, Christoph Egger, Pihla Karanko, and Pierre Meyer. Instantiating the hash-then-evaluate paradigm: Strengthening PRFs, PCFs, and OPRFs. In Clemente Galdi and Duong Hieu Phan, editors, *SCN 24: 14th International Conference on Security in Communication Networks, Part II*, volume 14974 of *Lecture Notes in Computer Science*, pages 97–116. Springer, Cham, September 2024. https://doi.org/10.1007/978-3-031-71073-5_5.
 - [39] Dung Bui and Geoffroy Couteau. Improved private set intersection for sets with small entries. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 13941 of *Lecture Notes in Computer Science*, pages 190–220. Springer, Cham, May 2023. https://doi.org/10.1007/978-3-031-31371-4_7.
 - [40] Dung Bui, Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Fast public-key silent OT and more from constrained Naor-Reingold. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part VI*, volume 14656 of *Lecture Notes in Computer Science*, pages 88–118. Springer, Cham, May 2024. https://doi.org/10.1007/978-3-031-58751-1_4.
 - [41] Melissa Chase, Chaya Ganesh, and Payman Mohassel. Efficient zero-knowledge proof of algebraic and non-algebraic statements with applications to privacy preserving credentials. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part III*, volume 9816 of *Lecture Notes in Computer Science*, pages 499–530. Springer, Berlin, Heidelberg, August 2016. https://doi.org/10.1007/978-3-662-53015-3_18.
 - [42] Melissa Chase, Yevgeniy Dodis, Yuval Ishai, Daniel Kraschewski, Tianren Liu, Rafail Ostrovsky, and Vinod Vaikuntanathan. Reusable non-interactive secure computation. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Sci-*

- ence, pages 462–488. Springer, Cham, August 2019. https://doi.org/10.1007/978-3-030-26954-8_15.
- [43] Seung Geol Choi, Jonathan Katz, Alex J. Malozemoff, and Vassilis Zikas. Efficient three-party computation from cut-and-choose. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology – CRYPTO 2014, Part II*, volume 8617 of *Lecture Notes in Computer Science*, pages 513–530. Springer, Berlin, Heidelberg, August 2014. https://doi.org/10.1007/978-3-662-44381-1_29.
 - [44] Benny Chor, Niv Gilboa, and Moni Naor. Private information retrieval by keywords. Cryptology ePrint Archive, Report 1998/003, 1998. URL <https://eprint.iacr.org/1998/003>.
 - [45] Arka Rai Choudhuri, Aarushi Goel, Aditya Hegde, and Abhishek Jain. Homomorphic secret sharing with verifiable evaluation. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024: 22nd Theory of Cryptography Conference, Part IV*, volume 15367 of *Lecture Notes in Computer Science*, pages 614–650. Springer, Cham, December 2024. https://doi.org/10.1007/978-3-031-78023-3_20.
 - [46] Amrita Roy Chowdhury, Chuan Guo, Somesh Jha, and Laurens van der Maaten. EIFFeL: Ensuring integrity for federated learning. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 2535–2549. ACM Press, November 2022. <https://doi.org/10.1145/3548606.3560611>.
 - [47] Graham Cormode and Shan Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. In *LATIN 2004: Theoretical Informatics: 6th Latin American Symposium, Buenos Aires, Argentina, April 5-8, 2004. Proceedings 6*, pages 29–38. Springer, 2004.
 - [48] Henry Corrigan-Gibbs and Dan Boneh. Prio: Private, robust, and scalable computation of aggregate statistics. In *14th USENIX symposium on networked systems design and implementation (NSDI 17)*, pages 259–282, 2017.
 - [49] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An anonymous messaging system handling millions of users. In *2015 IEEE Symposium on Security and Privacy*, pages 321–338. IEEE Computer Society Press, May 2015. <https://doi.org/10.1109/SP.2015.27>.
 - [50] Geoffroy Couteau. A note on the communication complexity of multiparty computation in the correlated randomness model. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 473–503. Springer, Cham, May 2019. https://doi.org/10.1007/978-3-030-17656-3_17.
 - [51] Geoffroy Couteau and Clément Ducros. Pseudorandom correlation functions from variable-density LPN, revisited. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 13941 of *Lecture Notes in Computer Science*, pages 221–250. Springer, Cham, May 2023. https://doi.org/10.1007/978-3-031-31371-4_8.
 - [52] Geoffroy Couteau and Pierre Meyer. Breaking the circuit size barrier for secure computation under quasi-polynomial LPN. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 842–870. Springer, Cham, October 2021. https://doi.org/10.1007/978-3-030-77886-6_29.
 - [53] Geoffroy Couteau and Maryam Zarezadeh. Non-interactive secure computation of inner-product from LPN and LWE. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022, Part I*, volume 13791 of *Lecture Notes in Computer Science*, pages 474–503. Springer, Cham, December 2022. https://doi.org/10.1007/978-3-031-22963-3_16.

- [54] Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Constrained pseudorandom functions from homomorphic secret sharing. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 194–224. Springer, Cham, April 2023. https://doi.org/10.1007/978-3-031-30620-4_7.
- [55] Ivan Damgård and Yuval Ishai. Constant-round multiparty computation using a black-box pseudorandom generator. In Victor Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 378–394. Springer, Berlin, Heidelberg, August 2005. https://doi.org/10.1007/11535218_23.
- [56] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 643–662. Springer, Berlin, Heidelberg, August 2012. https://doi.org/10.1007/978-3-642-32009-5_38.
- [57] Ivan Damgård, Rasmus Lauritsen, and Tomas Toft. An empirical study and some improvements of the MiniMac protocol for secure computation. In Michel Abdalla and Roberto De Prisco, editors, *SCN 14: 9th International Conference on Security in Communication Networks*, volume 8642 of *Lecture Notes in Computer Science*, pages 398–415. Springer, Cham, September 2014. https://doi.org/10.1007/978-3-319-10879-7_23.
- [58] Ivan Damgård, Jesper Buus Nielsen, Michael Nielsen, and Samuel Ranellucci. The TinyTable protocol for 2-party secure computation, or: Gate-scrambling revisited. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 167–187. Springer, Cham, August 2017. https://doi.org/10.1007/978-3-319-63688-7_6.
- [59] Quang Dao, Yuval Ishai, Aayush Jain, and Huijia Lin. Multi-party homomorphic secret sharing and sublinear MPC from sparse LPN. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 315–348. Springer, Cham, August 2023. https://doi.org/10.1007/978-3-031-38545-2_11.
- [60] Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. Authenticated garbling from simple correlations. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 57–87. Springer, Cham, August 2022. https://doi.org/10.1007/978-3-031-15985-5_3.
- [61] Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. Improving line-point zero knowledge: Two multiplications for the price of one. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 829–841. ACM Press, November 2022. <https://doi.org/10.1145/3548606.3559385>.
- [62] Jack Doerner and abhi shelat. Scaling ORAM for secure computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 523–535. ACM Press, October / November 2017. <https://doi.org/10.1145/3133956.3133967>.
- [63] Saba Eskandarian, Henry Corrigan-Gibbs, Matei Zaharia, and Dan Boneh. Express: Lowering the cost of metadata-hiding communication with cryptographic privacy. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021: 30th USENIX Security Symposium*, pages 1775–1792. USENIX Association, August 2021. URL <https://www.usenix.org/conference/usenixsecurity21/presentation/eskandarian>.

- [64] Sebastian Faust, Carmit Hazay, David Kretzler, Leandro Rometsch, and Benjamin Schlosser. Non-interactive threshold BBS+ from pseudorandom correlations. In Arpita Patra, editor, *CT-RSA 2025*, number 15598 in LNCS, pages 198–222. Springer, 2025. https://doi.org/10.1007/978-3-031-88661-4_9.
- [65] Nelly Fazio, Rosario Gennaro, Tahereh Jafarikhah, and William E Skeith. Homomorphic secret sharing from Paillier encryption. In Tatsuaki Okamoto, Yong Yu, Man Ho Au, and Yannan Li, editors, *Provable Security, ProvSec 2017*, volume 10592 of *Lecture Notes in Computer Science*, pages 381–399. Springer, 2017. https://doi.org/10.1007/978-3-319-68637-0_23.
- [66] Rex Fernando, Aayush Jain, and Ilan Komargodski. Maliciously-secure MrNISC in the plain model. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part II*, volume 14005 of *Lecture Notes in Computer Science*, pages 98–128. Springer, Cham, April 2023. https://doi.org/10.1007/978-3-031-30617-4_4.
- [67] Ingerid Fosli, Yuval Ishai, Victor I Kolobov, and Mary Wootters. On the download rate of homomorphic secret sharing. *arXiv preprint arXiv:2111.10126*, 2021.
- [68] Tore Kasper Frederiksen, Jesper Buus Nielsen, and Claudio Orlandi. Privacy-free garbled circuits with applications to efficient zero-knowledge. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 191–219. Springer, Berlin, Heidelberg, April 2015. https://doi.org/10.1007/978-3-662-46803-6_7.
- [69] Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. ZKBoo: Faster zero-knowledge for Boolean circuits. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016: 25th USENIX Security Symposium*, pages 1069–1083. USENIX Association, August 2016. URL <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/giacomelli>.
- [70] Aristides Gionis, Piotr Indyk, Rajeev Motwani, et al. Similarity search in high dimensions via hashing. In *Vldb*, volume 99, pages 518–529, 1999.
- [71] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th Annual ACM Symposium on Theory of Computing*, pages 218–229. ACM Press, May 1987. <https://doi.org/10.1145/28395.28420>.
- [72] Shafi Goldwasser, Yael Tauman Kalai, Raluca A. Popa, Vinod Vaikuntanathan, and Nikolai Zeldovich. Reusable garbled circuits and succinct functional encryption. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th Annual ACM Symposium on Theory of Computing*, pages 555–564. ACM Press, June 2013. <https://doi.org/10.1145/2488608.2488678>.
- [73] Vipul Goyal, Akshayaram Srinivasan, and Mingyuan Wang. Reusable secure computation in the plain model. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 427–458. Springer, Cham, August 2023. https://doi.org/10.1007/978-3-031-38557-5_14.
- [74] Daniel H. Greene, Michal Parnas, and F. Frances Yao. Multi-index hashing for information retrieval. In *35th Annual Symposium on Foundations of Computer Science*, pages 722–731. IEEE Computer Society Press, November 1994. <https://doi.org/10.1109/SFCS.1994.365720>.
- [75] Daniel Günther, Ágnes Kiss, and Thomas Schneider. More efficient universal circuit constructions. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017, Part II*, volume 10625 of *Lecture Notes in Computer Science*, pages 443–470. Springer, Cham, December 2017. https://doi.org/10.1007/978-3-319-70697-9_16.

- [76] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy*, pages 825–841. IEEE Computer Society Press, May 2020. <https://doi.org/10.1109/SP40000.2020.00016>.
- [77] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part I*, volume 14004 of *Lecture Notes in Computer Science*, pages 330–362. Springer, Cham, April 2023. https://doi.org/10.1007/978-3-031-30545-0_12.
- [78] Xiaojie Guo, Hanlin Liu, Zhicong Huang, Hongrui Cui, Wenhao Zhang, Cheng Hong, Xiao Wang, Kang Yang, and Yu Yu. Dory: Streaming PCG with small memory. Cryptology ePrint Archive, Paper 2025/1660, 2025. URL <https://eprint.iacr.org/2025/1660>.
- [79] Shai Halevi, Yehuda Lindell, and Benny Pinkas. Secure computation on the web: Computing without simultaneous interaction. In Phillip Rogaway, editor, *Advances in Cryptology – CRYPTO 2011*, volume 6841 of *Lecture Notes in Computer Science*, pages 132–150. Springer, Berlin, Heidelberg, August 2011. https://doi.org/10.1007/978-3-642-22792-9_8.
- [80] Richard W Hamming. Error detecting and error correcting codes. *The Bell system technical journal*, 29(2):147–160, 1950.
- [81] David Heath and Vladimir Kolesnikov. Stacked garbling for disjunctive zero-knowledge proofs. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 569–598. Springer, Cham, May 2020. https://doi.org/10.1007/978-3-030-45727-3_19.
- [82] Zhicong Huang, Wen jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022: 31st USENIX Security Symposium*, pages 809–826. USENIX Association, August 2022. URL <https://www.usenix.org/conference/usenixsecurity22/presentation/huang-zhicong>.
- [83] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 145–161. Springer, Berlin, Heidelberg, August 2003. https://doi.org/10.1007/978-3-540-45146-4_9.
- [84] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, Manoj Prabhakaran, and Amit Sahai. Efficient non-interactive secure computation. In Kenneth G. Paterson, editor, *Advances in Cryptology – EUROCRYPT 2011*, volume 6632 of *Lecture Notes in Computer Science*, pages 406–425. Springer, Berlin, Heidelberg, May 2011. https://doi.org/10.1007/978-3-642-20465-4_23.
- [85] Yuval Ishai, Dakshita Khurana, Amit Sahai, and Akshayaram Srinivasan. Black-box reusable NISC with random oracles. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part II*, volume 14005 of *Lecture Notes in Computer Science*, pages 68–97. Springer, Cham, April 2023. https://doi.org/10.1007/978-3-031-30617-4_3.
- [86] Marek Jawurek, Florian Kerschbaum, and Claudio Orlandi. Zero-knowledge using garbled circuits: how to prove non-algebraic statements efficiently. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013: 20th Conference on Computer and Communications Security*, pages 955–966. ACM Press, November 2013. <https://doi.org/10.1145/2508859.2516662>.
- [87] Ágnes Kiss and Thomas Schneider. Valiant’s universal circuit is practical. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EU-*

- ROCRYPT 2016, Part I*, volume 9665 of *Lecture Notes in Computer Science*, pages 699–728. Springer, Berlin, Heidelberg, May 2016. https://doi.org/10.1007/978-3-662-49890-3_27.
- [88] Vladimir Kolesnikov. Gate evaluation secret sharing and secure one-round two-party computation. In Bimal K. Roy, editor, *Advances in Cryptology – ASIACRYPT 2005*, volume 3788 of *Lecture Notes in Computer Science*, pages 136–155. Springer, Berlin, Heidelberg, December 2005. https://doi.org/10.1007/11593447_8.
 - [89] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz, editors, *ICALP 2008: 35th International Colloquium on Automata, Languages and Programming, Part II*, volume 5126 of *Lecture Notes in Computer Science*, pages 486–498. Springer, Berlin, Heidelberg, July 2008. https://doi.org/10.1007/978-3-540-70583-3_40.
 - [90] Yashvanth Kondi, Claudio Orlandi, and Lawrence Roy. Two-round stateless deterministic two-party Schnorr signatures from pseudorandom correlation functions. Cryptology ePrint Archive, Report 2023/216, 2023. URL <https://eprint.iacr.org/2023/216>.
 - [91] Zhe Li, Chaoping Xing, Yizhou Yao, and Chen Yuan. Efficient pseudorandom correlation generators for any finite field. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part V*, volume 15605 of *Lecture Notes in Computer Science*, pages 145–175. Springer, Cham, May 2025. https://doi.org/10.1007/978-3-031-91092-0_6.
 - [92] Helger Lipmaa, Payman Mohassel, and Saeed Sadeghian. Valiant’s universal circuit: Improvements, implementation, and applications. Cryptology ePrint Archive, Report 2016/017, 2016. URL <https://eprint.iacr.org/2016/017>.
 - [93] Payman Mohassel and Mike Rosulek. Non-interactive secure 2PC in the offline/online and batch settings. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part III*, volume 10212 of *Lecture Notes in Computer Science*, pages 425–455. Springer, Cham, April / May 2017. https://doi.org/10.1007/978-3-319-56617-7_15.
 - [94] Todd K Moon. *Error correction coding: mathematical methods and algorithms*. John Wiley & Sons, 2020.
 - [95] Andrew Morgan and Rafael Pass. Concurrently composable non-interactive secure computation. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022, Part I*, volume 13791 of *Lecture Notes in Computer Science*, pages 526–555. Springer, Cham, December 2022. https://doi.org/10.1007/978-3-031-22963-3_18.
 - [96] Andrew Morgan, Rafael Pass, and Antigoni Polychroniadou. Succinct non-interactive secure computation. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part II*, volume 12106 of *Lecture Notes in Computer Science*, pages 216–245. Springer, Cham, May 2020. https://doi.org/10.1007/978-3-030-45724-2_8.
 - [97] Zachary Newman, Sacha Servan-Schreiber, and Srinivas Devadas. Spectrum: High-bandwidth anonymous broadcast. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 229–248, 2022.
 - [98] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 678–708. Springer, Cham, October 2021. https://doi.org/10.1007/978-3-030-77870-5_24.

- [99] Srinivasan Raghuraman and Peter Rindal. Blazing fast PSI from improved OKVS and subfield VOLE. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 2505–2517. ACM Press, November 2022. <https://doi.org/10.1145/3548606.3560658>.
- [100] Srinivasan Raghuraman, Peter Rindal, and Titouan Tanguy. Expand-convolute codes for pseudorandom correlation generators from LPN. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 602–632. Springer, Cham, August 2023. https://doi.org/10.1007/978-3-031-38551-3_19.
- [101] Irving S Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the society for industrial and applied mathematics*, 8(2):300–304, 1960.
- [102] Peter Rindal and Phillipp Schoppmann. VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 901–930. Springer, Cham, October 2021. https://doi.org/10.1007/978-3-030-77886-6_31.
- [103] Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 687–717, Virtual Event, August 2021. Springer, Cham. https://doi.org/10.1007/978-3-030-84252-9_23.
- [104] Phillipp Schoppmann, Adrià Gascón, Leonie Reichert, and Mariana Raykova. Distributed vector-OLE: Improved constructions and implementation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 1055–1072. ACM Press, November 2019. <https://doi.org/10.1145/3319535.3363228>.
- [105] Adithya Vadapalli, Kyle Storrier, and Ryan Henry. Sabre: Sender-anonymous messaging with fast audits. In *2022 IEEE Symposium on Security and Privacy*, pages 1953–1970. IEEE Computer Society Press, May 2022. <https://doi.org/10.1109/SP46214.2022.9833601>.
- [106] Leslie G Valiant. Universal circuits (preliminary report). In *Proceedings of the eighth annual ACM symposium on Theory of computing*, pages 196–203, 1976.
- [107] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Authenticated garbling and efficient maliciously secure two-party computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 21–37. ACM Press, October / November 2017. <https://doi.org/10.1145/3133956.3134053>.
- [108] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. Global-scale secure multiparty computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 39–56. ACM Press, October / November 2017. <https://doi.org/10.1145/3133956.3133979>.
- [109] Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091. IEEE Computer Society Press, May 2021. <https://doi.org/10.1109/SP40001.2021.00056>.
- [110] Chenkai Weng, Kang Yang, Zhaomin Yang, Xiang Xie, and Xiao Wang. AntMan: Interactive zero-knowledge proofs with sublinear communication. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on*

- Computer and Communications Security*, pages 2901–2914. ACM Press, November 2022. <https://doi.org/10.1145/3548606.3560667>.
- [111] Sophia Yakoubov. A gentle introduction to yao’s garbled circuits. *preprint on webpage at <https://web.mit.edu/sonka89/www/papers/2017ygc.pdf>*, 2017.
 - [112] Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th Annual Symposium on Foundations of Computer Science*, pages 162–167. IEEE Computer Society Press, October 1986. <https://doi.org/10.1109/SFCS.1986.25>.
 - [113] Mark Zhandry. The magic of ELFs. *Journal of Cryptology*, 32(3):825–866, July 2019. <https://doi.org/10.1007/s00145-018-9289-9>.
 - [114] Shuoyao Zhao, Yu Yu, Jiang Zhang, and Hanlin Liu. Valiant’s universal circuits revisited: An overall improvement and a lower bound. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019, Part I*, volume 11921 of *Lecture Notes in Computer Science*, pages 401–425. Springer, Cham, December 2019. https://doi.org/10.1007/978-3-030-34578-5_15.

Supplementary Material

A Extended Preliminaries

A.1 Correlation-robust hash functions

Garbled circuit constructions make use of correlation-robust hash functions as key-derivation functions in the garbling process. Here, we formally define the notion.

Definition 39 (Correlation-robust hash functions [83]). *Let λ be a security parameter, $N = \text{poly}(\lambda)$, and $D, R \geq \lambda$ be integers. An efficiently computable function $H : \{0, 1\}^D \rightarrow \{0, 1\}^R$, is correlation robust if for all s and $t_1, \dots, t_N$ sampled uniformly and independently from $\{0, 1\}^D$, it holds that*

$$\{t_1, \dots, t_N, H(t_1 \oplus s), \dots, H(t_N \oplus s)\} \approx_c \text{Uniform},$$

where Uniform is the uniform distribution over $(\{0, 1\}^{D \cdot N} \times \{0, 1\}^{R \cdot N})$.

As in previous works, we can instantiate via $H(x) = \text{AES}_K(x) \oplus x$ for a fixed (and known) key K . This construction is known to be correlation robust if AES is modeled as a random permutation. For example, see [76].

B Garbling and Secret-Sharing a Circuit

In this section, we provide some background on garbled circuits, focusing on secret-sharing-friendly constructions.

B.1 How to garble a circuit

Here, we explain how a garbled gate is constructed using the point-and-permute [15] technique and “free-xor” trick [89] (these two tools are part of modern garbling techniques; see the excellent survey of Yakoubov [111]). While these techniques are not new, they are fundamental in constructing secret-shared garbled circuits that fit into our framework (see Appendix C). Our choice of tools follows prior work on secret-sharing garbled circuits [43, 60, 107, 108], where parties generate garbled circuits inside of secure multi-party computation protocols.

Garbling gates using point-and-permute. Let λ be a security parameter and H be a suitable hash function. Consider a gate G with input bits α and β , producing output bit γ (i.e., $\gamma \leftarrow G(\alpha, \beta)$), where $G \in \{\text{AND}, \text{XOR}\}$. With the point-and-permute optimization, the i -th wire in the circuit is associated with:

- Two randomly-chosen λ -bit wire labels $L_{i,0}$ and $L_{i,1}$.
- Two selection bits $\rho_{i,0}$ and $\rho_{i,1}$.

The labels $L_{i,0}$ and $L_{i,1}$ are chosen as random λ -bit strings. For wires i and j , with wire values α and β respectively, the selection bits are defined as

$$\rho_{i,\alpha} := \alpha \oplus r_i \text{ and } \rho_{j,\beta} := \beta \oplus r_j, \text{ where } r_i, r_j \leftarrow_R \{0, 1\}.$$

The garbling then consists of a table with four encrypted entries, such that the $(2\rho_{i,\alpha} + \rho_{j,\beta})$ -th entry is set to a *one-time pad encryption* of $(\rho_{k,G(\alpha,\beta)} \| L_{k,\rho_{k,G(\alpha,\beta)}})$ using encryption key

$H(L_{i,\alpha} \| L_{j,\beta} \| k \| z)$, derived by hashing the labels, output wire index k , and slot index $z = 2\rho_{i,\alpha} + \rho_{j,\beta}$. By placing the encryption in the z -th slot, the evaluator can determine which ciphertext to decrypt for a given pair of selection bits associated with the two gate wire values. The full garbling of an AND gate $G(\alpha, \beta)$ is described in Table 2.

$\rho_{i,\alpha}$	$\rho_{j,\beta}$	$(2\rho_{i,\alpha} + \rho_{j,\beta})$	Truth table	Garbling of G
0	0	0	$\rho_{00} = (r_i \wedge r_j) \oplus r_k$	$H(L_{i,0} \ L_{j,0} \ k \ 0) \oplus (\rho_{00} \ L_{k,\rho_{00}})$
0	1	1	$\rho_{01} = (r_i \wedge \neg r_j) \oplus r_k$	$H(L_{i,0} \ L_{j,1} \ k \ 1) \oplus (\rho_{01} \ L_{k,\rho_{01}})$
1	0	2	$\rho_{10} = (\neg r_i \wedge r_j) \oplus r_k$	$H(L_{i,1} \ L_{j,0} \ k \ 2) \oplus (\rho_{10} \ L_{k,\rho_{10}})$
1	1	3	$\rho_{11} = (\neg r_i \wedge \neg r_j) \oplus r_k$	$H(L_{i,1} \ L_{j,1} \ k \ 3) \oplus (\rho_{11} \ L_{k,\rho_{11}})$

Table 2: Garbled AND gate using the point-and-permute optimization with input values α and β on wires i and j , respectively, and output wire value γ on wire k . Grayed-out values are for illustrative purposes and are not part of the garbling.

The “free-xor” trick. Furthermore, by using the free-xor optimization, we only need to consider the garbling of AND gates, provided that we set the labels $L_{i,0}, L_{i,1}$ in a special way. This optimization was first described by Kolesnikov and Schneider [89] and fixes the relationship between the labels $L_{i,0}$ and $L_{i,1}$ such that $L_{i,\alpha} \oplus L_{j,\beta} = L_{k,\gamma}$ (for all XOR gates that take wires i, j , and output wire k). Let Δ be a λ -bit global offset. Then, for all i , letting $L_{i,1} := L_{i,0} \oplus \Delta$ is sufficient to ensure that for all j , $L_{i,\alpha} \oplus L_{j,\beta} = L_{k,\gamma}$ (see Remark 42 in Appendix C).

B.2 How to secret-share a garbled circuit

With the necessary background in place, we are ready to describe how a dealer can secret-share garbled gates to two parties. At a high level, we can “secret-share” the garbled circuit by splitting the garbling into two parallel garblings, inspired by the techniques of Dittmer et al. [60], Wang et al. [107, 108] and other work on secret-shared garbled circuits using distributed encryption. To make the notation easier to follow, we denote all the secrets known to party A in red and all the secrets known to party B in blue.

The dealer samples free-xor offsets Δ^A, Δ^B uniformly at random and provides:

- Δ^A to party A and Δ^B to party B ;
- Secret shares of a random wire mask bit r_i for each wire i in the circuit;
- For each (AND) gate G_k with input wires indexed by i and j and output wire indexed by k :
 - Secret shares of $(r_i \wedge r_j)$ and
 - Secret shares of $r_i \cdot \Delta^A, r_i \cdot \Delta^B, r_j \cdot \Delta^A, r_j \cdot \Delta^B, (r_i \wedge r_j) \cdot \Delta^A$, and $(r_i \wedge r_j) \cdot \Delta^B$.

From all these secret shares, the parties can non-interactively construct a *secret-shared* garbling of G_k as follows.

Step 1: Compute the output masks. Given secret shares of $r_i, r_j, (r_i \wedge r_j)$, the parties can non-interactively compute secret shares of $(\rho_{00}, \rho_{01}, \rho_{10}, \rho_{11})$, as defined in Table 2. To see this, note that

$$\rho_{00} := (r_i \wedge r_j) \oplus r_k, \quad \rho_{01} := \rho_{00} \oplus r_i, \quad \rho_{10} := \rho_{00} \oplus r_j, \quad \rho_{11} := \rho_{00} \oplus r_i \oplus r_j \oplus 1,$$

which are all linear functions of the shares given to both parties.¹²

Step 2: Compute the output labels. Each party samples a random wire label $L_{i,0}^\sigma$, for each wire i in the circuit.¹³ We show that, using the free-xor trick, the two parties can derive secret shares of the output label $L_{k,\rho}^\sigma$ corresponding to the output mask ρ , for each $\rho \in \{\rho_{00}, \rho_{01}, \rho_{10}, \rho_{11}\}$ and $\sigma \in \{A, B\}$, by having:

$$\begin{aligned} \text{Party } A \text{ compute } \langle L_{k,\rho}^A \rangle^A &:= L_{k,0}^A \oplus \langle \rho \cdot \Delta^A \rangle^A \quad \text{and} \quad \langle L_{k,\rho}^B \rangle^A := \langle \rho \cdot \Delta^B \rangle^A, \\ \text{Party } B \text{ compute } \langle L_{k,\rho}^B \rangle^B &:= L_{k,0}^B \oplus \langle \rho \cdot \Delta^B \rangle^B \quad \text{and} \quad \langle L_{k,\rho}^A \rangle^B := \langle \rho \cdot \Delta^A \rangle^B. \end{aligned}$$

The only problem is that the parties need to generate shares of $\rho \cdot \Delta^A$ and $\rho \cdot \Delta^B$. Fortunately, this can be done using the same equations used to compute shares of $(\rho_{00}, \rho_{01}, \rho_{10}, \rho_{11})$, but instead using the secret shares of $r_i \cdot \Delta^\sigma$, $r_j \cdot \Delta^\sigma$, and $(r_i \wedge r_j) \cdot \Delta^\sigma$, as we show in Section 4. Finally, by exploiting the observation made by prior work [43, 55, 108], we can make the one-time-pad encryption “secret-sharing friendly” by encrypting $(\rho \| L_{k,\rho}^A \| L_{k,\rho}^B)$ under *two* encryption keys $H(L_{i,\alpha}^A \| L_{j,\beta}^A \| k \| z)$ and $H(L_{i,\alpha}^B \| L_{j,\beta}^B \| k \| z)$ derived from the two separate label sets.

Putting the pieces together, using the dealer-provided secret shares, the parties can non-interactively compute a “secret-shared garbling” of an AND gate G_k , which we illustrate in Table 3.

$\rho_{i,\alpha}$	$\rho_{j,\beta}$	Truth table	Secret-shared garbling of G
0	0	$\rho_{00} = (r_i \wedge r_j) \oplus r_k$	$H(L_{i,0}^A \ L_{j,0}^A \ k \ 0) \oplus H(L_{i,0}^B \ L_{j,0}^B \ k \ 0) \oplus (\rho_{00} \ L_{k,\rho_{00}}^A \ L_{k,\rho_{00}}^B)$
0	1	$\rho_{01} = (r_i \wedge \neg r_j) \oplus r_k$	$H(L_{i,0}^A \ L_{j,1}^A \ k \ 1) \oplus H(L_{i,0}^B \ L_{j,1}^B \ k \ 1) \oplus (\rho_{01} \ L_{k,\rho_{01}}^A \ L_{k,\rho_{01}}^B)$
1	0	$\rho_{10} = (\neg r_i \wedge r_j) \oplus r_k$	$H(L_{i,1}^A \ L_{j,0}^A \ k \ 2) \oplus H(L_{i,1}^B \ L_{j,0}^B \ k \ 2) \oplus (\rho_{10} \ L_{k,\rho_{10}}^A \ L_{k,\rho_{10}}^B)$
1	1	$\rho_{11} = (\neg r_i \wedge \neg r_j) \oplus r_k$	$H(L_{i,1}^A \ L_{j,1}^A \ k \ 3) \oplus H(L_{i,1}^B \ L_{j,1}^B \ k \ 3) \oplus (\rho_{11} \ L_{k,\rho_{11}}^A \ L_{k,\rho_{11}}^B)$

Table 3: Secret-shared garbled AND gate using the point-and-permute technique with input values α and β on wires i and j , respectively, and output wire value γ on wire k .

Grayed-out values are for illustrative purposes and are not part of the garbling.

Step 3: Garbling the secret-shared input. We now turn to the problem of garbling the secret-shared input x . First, we note that, by definition, $L_{i,1}^\sigma := L_{i,0}^\sigma \oplus \Delta^\sigma$. Therefore, the problem reduces to computing shares of $L_{i,0}^\sigma \oplus x_i \cdot \Delta^\sigma$.

In Section 7, we give a construction for achieving this input garbling functionality based on HSS. In a little more detail, the parties are provided with HSS shares of the input x and the global offsets Δ^A and Δ^B (where the HSS shares of Δ^σ are provided by a one-time trusted setup). Then, using any public-key HSS scheme for degree-2 computation, the parties can non-interactively compute (additive) secret shares of $x \cdot \Delta^\sigma = (x_1 \cdot \Delta^\sigma, \dots, x_s \cdot \Delta^\sigma)$. Because we have that $L_{i,x_i}^\sigma := L_{i,0}^\sigma \oplus (x_i \cdot \Delta^\sigma)$, the parties obtain secret-shares of the correct label.

This construction can be used by either the evaluating parties themselves to encode their respective inputs or by an *external* party wishing to encode an input (e.g., see the zero-knowledge proof application in Section 8.2).

Step 4: Circuit evaluation. Given the secret shares of each garbled gate (i.e., secret shares of $\hat{C}$) and input labels (i.e., secret shares of $\hat{X}$), it is possible to recover $\hat{C}$ and $\hat{X}$. The circuit evaluator thus obtains all the necessary information needed to evaluate the

¹² In a bit more detail, observe that $\alpha \oplus \beta = (\alpha \wedge \neg \beta) \vee (\neg \alpha \wedge \beta)$ and $(\alpha \wedge \beta) \oplus \alpha = ((\alpha \wedge \beta) \wedge \neg \alpha) \vee (\neg(\alpha \wedge \beta) \wedge \alpha) = (\neg \alpha \vee \neg \beta) \wedge \alpha = \neg \beta \wedge \alpha$. Hence, $\rho_{01} = \rho_{00} \oplus r_i$. For the relation with $\rho_{11} = \rho_{01} \oplus \neg r_j = r_k \oplus (r_i \wedge \neg r_j) \oplus \neg r_j$ substitute α with $\neg \alpha$.

¹³ Note that for each wire i , $L_{i,1}^\sigma$ can be derived as $L_{i,0}^\sigma \oplus \Delta^\sigma$ using the free-xor trick.

garbled circuit and obtain $\mathcal{C}(x)$. In more detail, for each gate with input values α and β , given the label $(L_{i,\alpha}^A, L_{j,\beta}^A, L_{i,\alpha}^B, L_{j,\beta}^B)$, and the selection bits $(\rho_{i,\alpha}, \rho_{j,\beta})$, the evaluator can “peel off” the mask in the z -th slot of the garbling of the gate G_k (cf. Table 3) by locally computing $H(L_{i,\alpha}^A \| L_{j,\beta}^A \| k \| z) \oplus H(L_{i,\alpha}^B \| L_{j,\beta}^B \| k \| z)$, where $z = 2\rho_{i,\alpha} + \rho_{j,\beta}$. The remainder of the evaluation proceeds as usual. In other words, given shares of the labels and the garbled circuit, the evaluator obtains a valid garbled circuit and garbled input “in the clear,” from which correctness and privacy follows.

C Secret-Sharing-Friendly Garbled Circuits

In this section, we formally define the notion of garbled circuits and give a construction of a “secret-sharing” friendly garbled circuit that we use as a reference for our garbling correlations.

Formalizing the garbled circuit functionality. We start by defining the garbled circuit functionality. We do so in slightly less general terms (compared to, e.g., [16]), so as to better align with our applications and simplify notation.

Definition 40 (Secret-Sharing-Friendly Garbling Scheme). A secret-sharing-friendly garbling scheme GC for two parties A and B , consists of four efficient algorithms

$$\text{GC} = (\text{SampLabels}_\sigma, \text{Garble}, \text{Encode}, \text{Decode})$$

with the following functionality:

- $\text{SampLabels}_\sigma(1^\lambda, \mathcal{C}) \rightarrow \mathbf{L}^\sigma$. The randomized label sampling algorithm is parameterized by the party identifier $\sigma \in \{A, B\}$. It takes as input the security parameter λ and the number of inputs s for a circuit to be garbled. Outputs labels $\mathbf{L}^\sigma = \mathbf{L}_{\text{in}}^\sigma \| \mathbf{L}_{\text{gate}}^\sigma$ (to be used with a global offset $\Delta^\sigma \leftarrow_R \mathbb{F}_{2^\lambda}$), where the first s entries constitute the labels $\mathbf{L}_{\text{in}}^\sigma$ for the input wires, and the remaining constitute those for the other gates.
- $\text{Garble}(1^\lambda, \mathcal{C}, \Delta^A, \mathbf{L}^A, \Delta^B, \mathbf{L}^B) \rightarrow (\hat{\mathcal{C}}, \mathbf{r})$. The deterministic garbling algorithm takes as input the security parameter λ , a circuit description $\mathcal{C}$ with s inputs, a global offset Δ^σ , and labels $\mathbf{L}^\sigma$ per party $\sigma \in \{A, B\}$. It outputs a garbled circuit $\hat{\mathcal{C}}$ and input masks $\mathbf{r}$.
- $\text{Encode}(x, \Delta^A, \mathbf{L}_{\text{in}}^A, \Delta^B, \mathbf{L}_{\text{in}}^B, \mathbf{r}) \rightarrow \hat{X}$. The deterministic input encoding algorithm takes as input an s -bit input $x := x_1 \| \dots \| x_s$, input encoding labels $\mathbf{L}_{\text{in}}$, and $\mathbf{r} := (\mathbf{r}_1 \dots, \mathbf{r}_s)$. It outputs an encoded circuit input $\hat{X}$.
- $\text{Decode}(\mathcal{C}, \hat{\mathcal{C}}, \hat{X}) \rightarrow y$. The deterministic decoding algorithm takes as input the garbled circuit $\hat{\mathcal{C}}$ and garbled input $\hat{X}$. It outputs the result y .

The above functionality must satisfy correctness and security, defined as follows:

Correctness. A garbling scheme GC is correct if, for all security parameters λ , for all circuits $\mathcal{C}$, and all inputs $x \in \{0, 1\}^s$,

$$\Pr \left[\begin{array}{l} (\Delta^A, \Delta^B) \leftarrow_R \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda} \\ \mathbf{L}^A = \mathbf{L}_{\text{in}}^A \| \mathbf{L}_{\text{gate}}^A \leftarrow \text{SampLabels}_A(1^\lambda, \mathcal{C}) \\ \mathbf{L}^B = \mathbf{L}_{\text{in}}^B \| \mathbf{L}_{\text{gate}}^B \leftarrow \text{SampLabels}_B(1^\lambda, \mathcal{C}) \\ (\hat{\mathcal{C}}, \mathbf{r}) \leftarrow \text{Garble}(1^\lambda, \mathcal{C}, \Delta^A, \mathbf{L}^A, \Delta^B, \mathbf{L}^B) \\ \hat{X} \leftarrow \text{Encode}(x, \Delta^A, \mathbf{L}_{\text{in}}^A, \Delta^B, \mathbf{L}_{\text{in}}^B, \mathbf{r}) \end{array} : \text{Decode}(\mathcal{C}, \hat{\mathcal{C}}, \hat{X}) = \mathcal{C}(x) \right] = 1.$$

Security. A garbling scheme GC is secure if for all λ , for all circuits $\mathcal{C}$, and all inputs x , there exists an efficient simulator $\mathcal{S}$ such that

$$\left\{ (\mathcal{C}, \hat{\mathcal{C}}, \hat{X}) \left| \begin{array}{l} (\Delta^A, \Delta^B) := \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda} \\ L^A = L_{\text{in}}^A \parallel L_{\text{gate}}^A \leftarrow \text{SampLabels}_A(1^\lambda, \mathcal{C}) \\ L^B = L_{\text{in}}^B \parallel L_{\text{gate}}^B \leftarrow \text{SampLabels}_B(1^\lambda, \mathcal{C}) \\ (\hat{\mathcal{C}}, \mathbf{r}) \leftarrow \text{Garble}(1^\lambda, \mathcal{C}, \Delta^A, \mathbf{L}^A, \Delta^B, \mathbf{L}^B) \\ \hat{X} \leftarrow \text{Encode}(x, \Delta^A, \mathbf{L}_{\text{in}}^A, \Delta^B, \mathbf{L}_{\text{in}}^B, \mathbf{r}) \end{array} \right. \right\} \approx_c \mathcal{S}(1^\lambda, \mathcal{C}, \mathcal{C}(x)).$$

Construction. In [Construction 9](#), we give a construction of a “secret-sharing friendly” garbled circuit based on the point-and-permute and free-xor optimizations. This construction is adapted from prior work on secret-sharing garbled circuits [43, 60, 107, 108] (in particular, the work of [107]). As shown in [107], this garbled circuit can be made “secret-sharing friendly” by separating the one-time-pad encryption into two keys derived from two independent sets of labels, which enables “distributed encryption” [43, 55]. In particular, splitting the secrets across two parties allows each party to derive its own labels. As before, we denote all the secret values known to party A in red and all the values known to party B in blue.

Construction 9 (Secret-sharing-friendly garbled circuit; adapted from [107]).

Parameters. Let $H : \{0, 1\}^* \rightarrow \mathbb{F}_{2^{2\lambda+1}}$ be a suitable correlation-robust hash function ([Definition 39](#)). WLOG, let the input wires of an s -input circuit $\mathcal{C}$ be indexed by $1, \dots, s$.

$\text{SampLabels}_\sigma(1^\lambda, \mathcal{C})$:

```

1 : foreach wire  $i \in \mathcal{C}$ :
2 :    $L_{i,0}^\sigma \leftarrow_R \mathbb{F}_{2^\lambda}$ 
3 : return  $L^\sigma := (L_{i,0}^\sigma)_{\text{wire } i \in \mathcal{C}}$ 

```

$\text{Garble}(1^\lambda, \mathcal{C}, \Delta^A, \mathbf{L}^A, \Delta^B, \mathbf{L}^B)$:

```

1 : foreach wire  $i \in \mathcal{C}$ :
2 :   if wire  $i$  is an output wire of  $\mathcal{C}$  then  $r_i := 0$ ; else  $r_i \leftarrow_R \{0, 1\}$ 
3 :    $L_{i,1}^A := L_{i,0}^A \oplus \Delta^A$ 
4 :    $L_{i,1}^B := L_{i,0}^B \oplus \Delta^B$ 
5 : foreach AND gate  $G_k \in \mathcal{C}$ :
6 :   parse  $G_k = (i, j, k)$ 
7 :    $\rho_{00} := (r_i \wedge r_j) \oplus r_k, \quad \rho_{01} := (r_i \wedge \neg r_j) \oplus r_k$ 
8 :    $\rho_{10} := (\neg r_i \wedge r_j) \oplus r_k, \quad \rho_{11} := (\neg r_i \wedge \neg r_j) \oplus r_k$ 
9 :    $g_{00} \leftarrow H(L_{i,0}^A \parallel L_{j,0}^A \parallel k \parallel 0) \oplus H(L_{i,0}^B \parallel L_{j,0}^B \parallel k \parallel 0) \oplus (\rho_{00} \parallel L_{k,\rho_{00}}^A \parallel L_{k,\rho_{00}}^B)$ 
10 :   $g_{01} \leftarrow H(L_{i,0}^A \parallel L_{j,1}^A \parallel k \parallel 1) \oplus H(L_{i,0}^B \parallel L_{j,1}^B \parallel k \parallel 1) \oplus (\rho_{01} \parallel L_{k,\rho_{01}}^A \parallel L_{k,\rho_{01}}^B)$ 
11 :   $g_{10} \leftarrow H(L_{i,1}^A \parallel L_{j,0}^A \parallel k \parallel 2) \oplus H(L_{i,1}^B \parallel L_{j,0}^B \parallel k \parallel 2) \oplus (\rho_{10} \parallel L_{k,\rho_{10}}^A \parallel L_{k,\rho_{10}}^B)$ 
12 :   $g_{11} \leftarrow H(L_{i,1}^A \parallel L_{j,1}^A \parallel k \parallel 3) \oplus H(L_{i,1}^B \parallel L_{j,1}^B \parallel k \parallel 3) \oplus (\rho_{11} \parallel L_{k,\rho_{11}}^A \parallel L_{k,\rho_{11}}^B)$ 
13 :   $\hat{G}_k := (g_{00}, g_{01}, g_{10}, g_{11})$ 
14 :   $\mathbf{r} := (r_1, \dots, r_s)$ 
15 : return  $((\hat{G}_k)_{k \in [\nu]}, \mathbf{r})$ 

```

Continued on [Page 64...](#)

Remark 41 (Zero-masks on output wires). In [Construction 9](#), the i -th output wire mask is set to zero as a syntactic simplification. Specifically, in order to achieve evaluation correctness,

```

Encode( $\mathbf{x}, \Delta^A, \mathbf{L}_{\text{in}}^A, \Delta^B, \mathbf{L}_{\text{in}}^B, \mathbf{r}$ ):
1: parse  $\mathbf{x} = (x_1, \dots, x_s)$ ,  $\mathbf{L}_{\text{in}}^A = (L_{1,0}^A, \dots, L_{s,0}^A)$ ,  $\mathbf{L}_{\text{in}}^B = (L_{1,0}^B, \dots, L_{s,0}^B)$ ,  $\mathbf{r} = (r_1, \dots, r_s)$ 
2: foreach  $i \in [s]$ :
3:    $L_{i,1}^A := L_{i,0}^A \oplus \Delta^A$ 
4:    $L_{i,1}^B := L_{i,0}^B \oplus \Delta^B$ 
5:  $\hat{X} := (L_{1,x_1}^A, \dots, L_{s,x_s}^A, L_{1,x_1}^B, \dots, L_{s,x_s}^B) \parallel (x_1 \oplus r_1, \dots, x_s \oplus r_s)$ 
6: return  $\hat{X}$ 

Decode( $\mathcal{C}, \hat{\mathcal{C}}, \hat{X}$ ):
1: parse  $\hat{\mathcal{C}} = (\hat{G}_k)_{k \in [\nu]}$ ,  $\hat{X} := (L_{1,x_1}^A, \dots, L_{s,x_s}^A, L_{1,x_1}^B, \dots, L_{s,x_s}^B) \parallel (\rho_1, \dots, \rho_s)$ 
2: foreach XOR gate  $G_k \in \mathcal{C}$ 
3:   parse  $G_k = (i, j, k)$ 
4:    $\rho_k := \rho_i \oplus \rho_j$ 
5:    $L_{k,\rho_k}^A := L_{i,\rho_i}^A \oplus L_{j,\rho_j}^A$ ,  $L_{k,\rho_k}^B := L_{i,\rho_i}^B \oplus L_{j,\rho_j}^B$ 
6: foreach AND gate  $G_k \in \mathcal{C}$ 
7:   parse  $G_k = (i, j, k)$  and  $\hat{G}_k = (g_{00}, g_{01}, g_{10}, g_{11})$ 
8:    $z := 2\rho_i + \rho_j$ 
9:    $\rho_k \parallel L_{k,\rho_k}^A \parallel L_{k,\rho_k}^B := \mathbf{H}(L_{i,\rho_i}^A \parallel L_{j,\rho_j}^A \parallel k \parallel z) \oplus \mathbf{H}(L_{i,\rho_i}^B \parallel L_{j,\rho_j}^B \parallel k \parallel z) \oplus g_{\rho_i \parallel \rho_j}$ 
10: return  $(\rho_{s+1}, \dots, \rho_{s+t})$ 

```

Construction 9: Construction of a secret-sharing-friendly garbled circuit.

the output wire values must not be masked, so that the evaluator can recover the final result of the computation.

Remark 42 (Free-xor correctness). For all i , letting $L_{i,1} := L_{i,0} \oplus \Delta$ is sufficient to ensure that for all j , $L_{i,\alpha} \oplus L_{j,\beta} = L_{k,\gamma}$. To see this, note that there are four cases to consider: (1) $L_{k,0} := L_{i,0} \oplus L_{j,0}$, (2) $L_{k,1} := L_{k,0} \oplus \Delta = L_{i,1} \oplus L_{j,0}$, (3) is symmetric to (2), and (4) $L_{k,0} := L_{i,1} \oplus L_{j,1}$ because the Δ value cancels out.

D PCG and PCFs with Scaled Outputs

We provide a generic transformation to scale the output of any PCG or (TA)PCF construction by a secret constant (fixed at setup time), provided that the construction satisfies some properties that are common across existing constructions. Specifically, in [Theorem 45](#), we show that it is possible to scale the output of `Expand` by any secret factor by modifying the PCG generation algorithm. Later, we show that this works as well for (TA)PCFs in [Corollary 49](#). This transformation enables generating *authenticated* pseudorandom correlations, as described by Boyle et al. [33], and plays a crucial role in our construction of PCGs and (TA)PCFs for garbling correlations, where we need to scale the correlations by the global “free-xor” offsets Δ^A and Δ^B .

Definition 43 (Scalable Correlations). Let $\mathbb{F}$ be a finite field and $\mathbb{K}$ be a finite field extension over $\mathbb{F}$. We say that a PCG $\text{PCG} = (\text{Gen}, \text{Expand})$ for an additive correlation $\mathcal{Y}$ (e.g., a correlation over additive secret shares) has scalable correlations if `Gen` admits an extra parameter $\Delta \in \mathbb{K}$ such that `Expand` outputs additive secret shares for the correlation $(\mathcal{Y}, \Delta \cdot \mathcal{Y})$, for any $\Delta \in \mathbb{K}$ that satisfies [Definition 5](#) (PCG security).

Remark 44. Any PCG with scalable outputs can be extended to one for which **Gen** admits multiple parameters $\Delta_1, \dots, \Delta_k \in \mathbb{K}$, so that **Expand** outputs shares of $(\Delta_1 \cdot \mathcal{Y}, \dots, \Delta_k \cdot \mathcal{Y})$. (Note that any algebraic and finite field extension $\mathbb{K}$ is generated by a single element $\Delta \in \mathbb{K}$.)

Theorem 45. Let $\text{PCG} = (\text{PCG.Gen}, \text{PCG.Expand})$ be any PCG that outputs additive secret shares in a field $\mathbb{F}$ for an additive correlation $\mathcal{Y}$, and let $\text{FSS} = (\text{FSS.Gen}, \text{FSS.Eval})$ be a function secret sharing (FSS) scheme for a function family $\mathcal{F}$ with range $\mathbb{F}$ (cf. [Section 3.6](#)), and let $n \in \mathbb{N}$. If the following three conditions hold:

- (1) PCG.Gen outputs n FSS keys $(\kappa_1^\sigma, \dots, \kappa_n^\sigma)$ as part of each seed s^σ ;
- (2) The output of $\text{PCG.Expand}(\sigma, s^\sigma)$, for all $\sigma \in \{A, B\}$, consists of a linear combination of FSS evaluations $\text{FSS.Eval}(\sigma, \kappa_i^\sigma, x_i)$, for $i \in [n]$; and
- (3) There exists an FSS scheme $(\text{FSS.Gen}', \text{FSS.Eval}')$ for $\Delta \cdot \mathcal{F}$ (the function family scaled by an arbitrary $\Delta \in \mathbb{K}$),

then there exists a PCG $(\text{PCG.Gen}', \text{PCG.Expand}')$ that satisfies [Definition 43](#) (scalable outputs) for correlation $\mathcal{Y}$.

Proof. We first note that any linear combination of FSS evaluations for different function families is itself an FSS [[24](#), Section 3.2].

By (1), we have that PCG.Expand is constructed as a linear combination of FSS evaluations. As such, we can write the output as

$$\text{PCG.Expand}(\sigma, s^\sigma) := \mathcal{L}(\text{FSS.Eval}(\sigma, \kappa_1^\sigma, x_1), \dots, \text{FSS.Eval}(\sigma, \kappa_n^\sigma, x_n)) = \langle y \rangle^\sigma,$$

where $\mathcal{L}$ is a linear function.

We claim that if we replace (κ_i^A, κ_i^B) , as computed according to $\text{FSS.Gen}(1^\lambda, f_i)$ for some function f_i , with $(\kappa_i'^A, \kappa_i'^B)$ computed according to $\text{FSS.Gen}'(1^\lambda, \Delta \cdot f_i)$ and replace $\text{FSS.Eval}(\sigma, \kappa_i^\sigma, x_i)$ with $\text{FSS.Eval}'(\sigma, \kappa_i'^\sigma, x_i)$ within PCG.Expand , then it holds that:

$$\begin{aligned} \text{PCG.Expand}(\sigma, s^\sigma) &= \mathcal{L}(\text{FSS.Eval}'(\sigma, \kappa_1'^\sigma, x_1), \dots, \text{FSS.Eval}'(\sigma, \kappa_n'^\sigma, x_n)) \\ &= \mathcal{L}(\Delta \cdot \text{FSS.Eval}(\sigma, \kappa_1^\sigma, x_1), \dots, \Delta \cdot \text{FSS.Eval}(\sigma, \kappa_n^\sigma, x_n)) = \langle \Delta \cdot y \rangle^\sigma. \end{aligned}$$

To see this, we note that by additive reconstruction property of FSS, we have:

$$\begin{aligned} \Delta \cdot (\text{FSS.Eval}(A, \kappa_i^A, x_i)) + \Delta \cdot (\text{FSS.Eval}(B, \kappa_i^B, x_i)) &= \Delta \cdot (f(x_i)) \\ &= \Delta \cdot f(x_i) = \text{FSS.Eval}'(\sigma, \kappa_i'^A, x_i) + \text{FSS.Eval}'(B, \kappa_i'^B, x_i), \end{aligned}$$

for all $i \in [n]$. Finally, we construct $(\text{PCG.Gen}', \text{PCG.Expand}')$ as follows:

- $\text{PCG.Gen}'(1^\lambda, \Delta)$:
 - 1: Run $(s^A, s^B) \leftarrow \text{PCG.Gen}(1^\lambda)$.
 - 2: For all $i \in [n]$, let (κ_i^A, κ_i^B) be the FSS keys computed according to $\text{FSS.Gen}(1^\lambda, f_i)$ when running $\text{PCG.Gen}(1^\lambda)$.
 - 3: For all $i \in [n]$, compute $(\kappa_i'^A, \kappa_i'^B) \leftarrow \text{FSS.Gen}'(1^\lambda, \Delta \cdot f_i)$.
 - 4: Set $s'^A := (s^A, (\kappa_i'^A)_{i=1}^n)$ and $s'^B := (s^B, (\kappa_i'^B)_{i=1}^n)$.
 - 5: Output (s'^A, s'^B) .
- $\text{PCG.Expand}'(\sigma, s'^\sigma)$:
 - 1: Parse $s'^\sigma := (s^\sigma, (\kappa_i'^\sigma)_{i=1}^n)$.

- 2: Compute $y_\sigma := \text{PCG.Expand}(\sigma, s^\sigma)$ as the linear combination of FSS evaluations using keys $(\kappa_i^\sigma)_{i=1}^n$.
- 3: Compute $y'_\sigma := \text{PCG.Expand}(\sigma, s^\sigma)$ as the linear combination of FSS evaluations using keys $(\kappa_i'^\sigma)_{i=1}^n$.
- 4: Output (y_σ, y'_σ) .

We now sketch why $(\text{PCG.Gen}', \text{PCG.Expand}')$ is a PCG satisfying [Definition 5](#) for the correlation class $(\mathcal{Y}, \Delta \cdot \mathcal{Y})$.

Pseudorandomness. The pseudorandomness follows from the fact that each FSS (which entirely defines the pseudorandomness of the PCG expansion procedure) is computed using a different set of FSS keys that are sampled independently of the original ones. As such, by a simple hybrid argument, the concatenated outputs are pseudorandom by the security of the FSS scheme. Moreover, by the correctness property of FSS and the pseudorandomness property of PCG, we have the resulting correlations are distributed according to $(\mathcal{Y}, \Delta \cdot \mathcal{Y})$.

Security. Security follows directly from the security of FSS and PCG. Again, by a simple hybrid argument, we can replace all the independently generated FSS keys $(\kappa_i'^\sigma)_{i=1}^n$ with FSS keys encoding the all-zeroes function. Security then reduces directly to the PCG. $\square$

For the following corollary of [Theorem 45](#), we first need to define the notion of distributed point and interval functions.

Definition 46 (Distributed Point Functions). A point function $f_{\alpha,m}$ for $\alpha \in \{0,1\}^\nu$ and $m \in \mathbb{G}$, is the function $f : \{0,1\}^\nu \rightarrow \mathbb{G}$ such that $f(x) = m$ if $x = \alpha$ and $f(x) = 0$ if $x \neq \alpha$. A distributed point function (DPF) is an FSS scheme for the family of all point functions with domain $\{0,1\}^\nu$ and range $\mathbb{G}$.

Definition 47 (Distributed Interval Functions). An interval function $f_{(\alpha,\beta),m}$ for $\alpha, \beta \in \{0,1\}^\nu$ and $m \in \mathbb{G}$ is defined as the function $f(x) = m$ for $\alpha < x < \beta$ and $f(x) = 0$ for $x \leq \alpha$ or $x \geq \beta$. A distributed interval function (DIF) is an FSS scheme for the family of all interval functions with domain $\{0,1\}^\nu$ and range $\mathbb{G}$.

Corollary 48. Let $\mathbb{K}$ be a field extension over a finite field $\mathbb{F}$ and let $\text{PCG} = (\text{PCG.Gen}, \text{PCG.Expand})$ be a (possibly programmable) PCG that outputs additive secret shares in $\mathbb{K}$ for an additive correlation $\mathcal{Y}$ (as defined in [Definition 5](#)). If the output of PCG.Expand consists of a linear combination of distributed point functions or distributed interval functions with output range $\mathbb{F}$, then PCG satisfies [Definition 43](#) (scalable outputs) for $\mathcal{Y}$.

Proof. The proof follows from [Theorem 45](#) and noting that for all point functions $P_{i,m}$ with range $\mathbb{F}$, $\Delta \cdot P_{i,m} = P_{i,\Delta \cdot m}$ is a point function with range $\mathbb{K}$ (the same holds for interval and comparison functions). We remark that extending the output to a field extension can be achieved by either naively extending the number of FSS instances or by using tailored techniques [\[24\]](#). $\square$

The definition of PCGs with scalable correlations naturally extends to (TA)PCFs, by replacing all occurrences of PCG and PCG.Expand by PCF and PCF.Eval (resp. TAPCF.Eval). This gives us the following corollary:

Corollary 49. Let $\text{PCF} = (\text{PCF.Gen}, \text{PCF.Eval})$ be a PCF. Then, [Theorem 45](#) and [Corollary 48](#) apply to PCF. In particular, the definition and theorem are not restricted to PCGs, and therefore also apply to PCFs (including TAPCFs).

Proof. Note that the proof of [Theorem 45](#) does not depend on specific properties inherent to PCGs, therefore it immediately applies to PCFs as well, provided that the output can be described as a linear combination of FSS evaluations. $\square$

PCGs and PCFs with scalable outputs. By [Corollary 48](#), we immediately have that several existing PCG and PCFs that use DPFs or DIFs under-the-hood in the way we specify in [Theorem 45](#) (e.g., [\[29, 30, 33\]](#)), satisfy [Definition 43](#). Indeed, each work explicitly mentions the ability to “scale” the outputs of their constructions for the purpose of generating authenticated multiplication triples. However, only Boyle et al. [\[33, Theorem 6.6 of the full version\]](#) provides an explicit construction with a proof of security. With [Theorem 45](#), we contribute an abstract view of the required properties for scalable outputs, so as to make this notion applicable to all DPF (and DIF)-based constructions. We note, in passing, that because our TAPCF construction from [Section 5](#) is a generalization of the PCF construction of Boyle et al. [\[35\]](#), it inherits the same properties making it satisfy the requirements of [Theorem 45](#) (indeed, it is easy to check that the requirements are met by inspecting [Construction 3](#)).

E Towards Authenticated Garbling

In this section, we briefly describe how our TAPCF construction can be upgraded to produce shares of *authenticated* garbled circuits. More concretely, authenticate garbling schemes are augmented with an authentication procedure allowing the party reconstructing the shares to check that each share was computed correctly (i.e., that each gate in the garbled circuit was honestly generated).

The problem with malicious parties. In the malicious setting, an honest Bob is not guaranteed that Alice will garble the right circuit (e.g., Alice can maliciously generate her share of the garbled circuit to compute a different function). With an authenticated garbled circuit, once Bob recovers his share of the garbled circuit, he can efficiently check that Alice did *not* manipulate her shares and safely abort if the check fails. State-of-the-art constructions of authenticated garbling schemes [\[60, 107\]](#) build upon the idea from the SPDZ [\[56\]](#) multi-party computation protocol by using an information-theoretic MAC to realize an efficient verification procedure.

Generated an authenticated garbling. We briefly note that our construction of GPCF can be generalized to a GPCF for an *authenticated* variant of [Construction 1](#), which follows immediately from previous works on authenticated garbling [\[60, 107\]](#). The crux of the problem lies in having an authenticated copy of the garbled table. Specifically, for each row of each garbled gate table, let ρ_{b_0, b_1} (for some $b_0, b_1 \in \{0, 1\}$) be the mask bit. The authenticated variant of such a table has an additional term $(\rho_{b_0, b_1}) \cdot \Delta$ appended to the row, where Δ is a global MAC key that is unknown to both parties. Such authenticated garbled tables can be constructed directly from TAPCF for *degree-3* correlations. To see how, note that in our GPCF construction, we already generate shares of pseudorandom tuples of the form:

$$r_i, r_j, (r_i \wedge r_j), r_i \cdot \Delta^A, r_i \cdot \Delta^B, r_j \cdot \Delta^A, r_j \cdot \Delta^B, (r_i \wedge r_j) \cdot \Delta^A, (r_i \wedge r_j) \cdot \Delta^B.$$

Now, we can additionally compute output shares of these tuples scaled by the pseudorandom MAC key Δ . Unfortunately, the same MAC key Δ *cannot* be reused across different invocations of the PCF, since Δ must be revealed when the garbled circuit shares are reconstructed (to check consistency) [\[56, 60, 107\]](#). This means that we cannot simply rely on a degree-2 TAPCF with scaled outputs, and must instead rely on a TAPCF for degree-3 correlations. We already described in [Section 5.2](#) a general construction of our TAPCF from Expand-Accumulate LPN for degree- d correlations. However, with $d = 3$, the construction

is significantly less efficient (with seed size about 2500 times larger and runtimes about 900 times slower [35]). Nevertheless, we view this approach as an interesting alternative path to upgrading our construction to produce authenticated garbled circuits. We leave further exploration of such an approach as a very interesting open question for future work.

F Deferred Proofs

F.1 Proof of Theorem 23

Proof. We prove correctness and security in turn. Our security analysis is similar to the proof of security for the PCF in [35], but we strengthen it on two fronts: (1) while [35] focuses on degree-2 correlations, we directly prove security for general degree- d correlations (the extension is straightforward but tedious), and (2) we show that the construction achieve the stronger notion of TAPCF.

Correctness. For correctness, it helps to view $\mathbf{e}_1 \otimes \cdots \otimes \mathbf{e}_d$ as a d -dimensional hypercube of side length $5N$, divided into t^s smaller d -dimensional unit hypercubes of side length $5N/t$, where the $(i_1, \dots, i_d)$ -th hypercube corresponds to the tensor product $\mathbf{e}_{1,i_1} \otimes \cdots \otimes \mathbf{e}_{d,i_d}$, which is a d -dimensional unit vector (here, $\mathbf{e}_{i,j}$ denote the i_j -th unit vector of $\mathbf{e}_i$).

Furthermore, denoting $\Delta_N^{\otimes d}$ the d -fold tensor product of Δ_N with itself, observe that $\Delta_N^{\otimes d} \cdot (\mathbf{e}_1 \otimes \cdots \otimes \mathbf{e}_d)$ is itself a d -dimensional hypercube divided into t^d smaller d -dimensional hypercubes of side length $5N/t$, where the hypercube at position $\mathbf{i} = (i_1, \dots, i_d)$ is exactly the truth-table of the d -dimensional rectangle predicate $f_{\alpha_i,1}$. This follows from the fact that the multiplication by Δ_N captures the process of *accumulating* the entries of a vector—i.e., it maps $(x_1, \dots, x_{5N})$ to $(x_1, x_1 \oplus x_2, \dots, \bigoplus_{i \leq 5N} x_i)$.

Then, denoting $\mathbf{j} = (j_1^{k_1}, \dots, j_d^{k_d})$ and $\alpha = (\alpha_1^{k_1}, \dots, \alpha_d^{k_d})$, by the correctness of the FSS scheme, the outputs $\text{FSS.Eval}(\sigma, \mathbf{k}_j^\sigma, \alpha)$ form shares of the value $f_{\alpha',1}(\alpha)$, where α' is the d -tuple of positions of the 1 in each unit vector $\mathbf{e}_{i,j_i^{k_i}}$ for each $i \in [d]$, which is the value at position α within the $\mathbf{i}$ -th hypercube of $\Delta_N^{\otimes d} \cdot (\mathbf{e}_1 \otimes \cdots \otimes \mathbf{e}_d)$.

Now, recall that we parse each a_i as the tuple (i, a_i) to assign an ordering on the group elements $a_i \in (a_1, \dots, a_m)$. The sum of all values $\text{FSS.Eval}(\sigma, \mathbf{k}_j^\sigma, \alpha)$ corresponds to the sum of all entries of $\Delta_N^{\otimes d} \cdot (\mathbf{e}_1 \otimes \cdots \otimes \mathbf{e}_d)$ whose position corresponds to a “1” in $h_{a_1} \otimes h_{a_2} \otimes \cdots \otimes h_{a_d}$. That is, we have:

$$\begin{aligned} y^A + y^B &= \sum (\mathbf{h}_{a_1} \otimes \mathbf{h}_{a_2} \otimes \cdots \otimes \mathbf{h}_{a_d})^\top \cdot \Delta_N^{\otimes d} \cdot (\mathbf{e}_1 \otimes \cdots \otimes \mathbf{e}_d) \\ &= \sum \prod_{i=1}^d \mathbf{h}_{a_i}^\top \cdot (\Delta_N \cdot \mathbf{e}_i), \end{aligned}$$

where the sum goes over all the monomials $\prod_i X_{a_i}$ of P . Now, define H to be the matrix whose rows are the $\mathbf{h}_a$, for all distinct values a that appears in a query $(P, (a_1, \dots, a_m))$ to $\text{PCF.Eval}(\sigma, \mathbf{k}^\sigma, \cdot)$, for $\sigma = A$ or $\sigma = B$, in the $\mathcal{P}$ -correlated outputs game of Figure 3.

Denote $\mathbf{x}_i := H \cdot \Delta_N \cdot \mathbf{e}_i$ for each $i \in [d]$. With this notation, $\mathbf{h}_{a_i}^\top \cdot (\Delta_N \cdot \mathbf{e}_i)$ is the value indexed by a_i in $H \cdot \Delta_N \cdot \mathbf{e}_i = \mathbf{x}_i$. We write $\mathbf{x}_i[a_i]$ to denote this entry. (Note that $\mathbf{x}_i[a_i]$ is *not* the a_i -th entry of $\mathbf{x}_i$, but the key in a dictionary of pairs $(a_i, \mathbf{x}_i[a_i] = \mathbf{h}_{a_i}^\top \cdot (\Delta_N \cdot \mathbf{e}_i))$).

Therefore, we have that

$$y^A + y^B = P(\mathbf{x}_1[a_1], \dots, \mathbf{x}_m[a_m]).$$

From here, the indistinguishability between the $\mathcal{P}$ -correlated outputs experiments follows from a sequence of straightforward hybrid games.

- *Hybrid $\mathcal{H}_0$* . This hybrid corresponds to the $\text{Exp}_{\mathcal{A},0}^{\text{pr}}(\lambda)$ experiment from Figure 3.
- *Hybrid $\mathcal{H}_1$* . In this hybrid game, we replace the outputs of $\text{FSS.Eval}(\sigma, \mathbf{k}_j^\sigma, \boldsymbol{\alpha})$ for $\sigma \in \{A, B\}$ by uniformly random shares of $f_{\boldsymbol{\alpha}_{i,1}}$. This is indistinguishable from $\mathcal{H}_0$ under the security of the FSS scheme. From here onwards, (y^A, y^B) form truly random shares of $P(\mathbf{x}_1[a_1], \dots, \mathbf{x}_m[a_m])$.
- *Hybrid $\mathcal{H}_2$* . In this hybrid game, we replace the mapping $a_i \rightarrow \mathbf{h}_{a_i}^\top \cdot (\Delta_N \cdot \mathbf{e}_i)$ computed using $\mathbf{h}_{a_i} = H(a_i)$ with the mapping $a_i \rightarrow R(a_i)$, where R is a truly random function. Note that the mapping maps each a_i to an entry of $H \cdot \Delta_N \cdot \mathbf{e}_i = \mathbf{x}_i$, which is indistinguishable from a uniformly random vector under the EALPN assumption. Therefore, under EALPN, this game is indistinguishable from $\mathcal{H}_1$. From here, (y^A, y^B) form truly random shares of $P(R(a_1), \dots, R(a_m))$, which is exactly $\text{Exp}_{\mathcal{A},1}^{\text{pr}}(\lambda)$, concluding the proof.

Security. The security experiments proceeds similarly.

- *Hybrid $\mathcal{H}_0$* . This hybrid corresponds to the $\text{Exp}_{\mathcal{A},\sigma,\bar{\sigma},0}^{\text{sec}}(\lambda)$ experiment from Figure 4.
- *Hybrid $\mathcal{H}_1$* . In this hybrid game, we replace the output

$$y^{\bar{\sigma}} \leftarrow \text{TAPCF.Eval}(\bar{\sigma}, \mathbf{k}^{\bar{\sigma}}, P, (a_1, \dots, a_m))$$

with the $y^{\bar{\sigma}}$ computed from y^σ using the equation

$$y^{\bar{\sigma}} + y^\sigma = P(\mathbf{x}_1[a_1], \dots, \mathbf{x}_m[a_m]).$$

This game is indistinguishable from $\mathcal{H}_0$ by the correctness of the FSS scheme.

- *Hybrid $\mathcal{H}_2$* . In this hybrid game, we replace all keys $\mathbf{k}_i^\sigma$ in $\mathbf{k}^\sigma = (\mathbf{k}_i^\sigma)_{i \in [t]^d}$ by simulated keys, which are independent of the secret vectors $\boldsymbol{\alpha}_i$. This game is computationally indistinguishable from $\mathcal{H}_1$ under the security of the FSS scheme. From this game onward, the key $\mathbf{k}^\sigma$ is independent of the noise vectors $\mathbf{e}_1, \dots, \mathbf{e}_d$.
- *Hybrid $\mathcal{H}_3$* . In this hybrid game, we replace the mapping $a_i \rightarrow \mathbf{h}_{a_i}^\top \cdot (\Delta_N \cdot \mathbf{e}_i)$ with $h_{a_i} = H(a_i)$ by $a_i \rightarrow R(a_i)$, where R is a truly random function, which is indistinguishable from the previous game under the EALPN assumption. This game is now exactly $\text{Exp}_{\mathcal{A},\sigma,\bar{\sigma},1}^{\text{sec}}(\lambda)$, which concludes the proof.

□

F.2 Proof of Proposition 29

We prove pseudorandomness/correctness and security in turn.

Pseudorandomness. We argue pseudorandomness through a sequence of three game hops. At a high level, we prove the following implications:

$$\begin{aligned} \text{TAPCF pseudorandomness} &\implies \text{non-scaled PCF pseudorandomness} \\ &\implies \text{(scaled) PCF pseudorandomness.} \end{aligned}$$

By contraposition:

$$\begin{aligned} (\text{scaled}) \text{ PCF non-pseudorandomness} &\implies \text{non-scaled PCF non-pseudorandomness} \\ &\implies \text{TAPCF non-pseudorandomness.} \end{aligned}$$

Hence, the first game corresponds to the standard PCF pseudorandomness game. In the second game hop, the adversary obtains the output of PCF along with the scaled output, where the scalars are $(1, 1) \in \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda}$ instead of random (Δ^A, Δ^B) . The third game hop corresponds to the TAPCF pseudorandomness game.

Game $\mathcal{G}_0$. This game corresponds exactly to the PCF pseudorandomness game of PCF given in Fig. 3. In this game, the PCF output is scaled by random scalars (Δ^A, Δ^B) that are part of $\mathbf{yk}$ output by Setup.

Game $\mathcal{G}_1$. This game corresponds exactly to the PCF pseudorandomness game of PCF given in Fig. 3, except that the lines $\mathbf{yk} \leftarrow \text{Setup}(1^\lambda)$ are replaced by $\mathbf{yk} := (1, 1) \in \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda}$. This results in the “scaled” output being a tuple of three fresh shares of the same “unscaled” pseudorandom value.

Game $\mathcal{G}_2$. This game corresponds to the TAPCF pseudorandomness game given in Fig. 5.

Claim. If there exists an efficient adversary $\mathcal{A}$ for $\mathcal{G}_0$ that wins with non-negligible advantage, then there exists an efficient adversary $\mathcal{B}$ that wins $\mathcal{G}_1$ with the same advantage.

Proof. Let $\mathcal{A}$ be an efficient adversary for $\mathcal{G}_0$ having non-negligible advantage. We construct the following efficient algorithm $\mathcal{B}$ for the game defined in $\mathcal{G}_1$. $\mathcal{B}$ proceeds as follows:

1. On input 1^λ ,
 - 1: Run $\mathcal{A}(1^\lambda)$ and receive state $\mathbf{st}_0$ and inputs $x_1, \dots, x_N$.
 - 2: Sample $\mathbf{yk}' := (\Delta^A, \Delta^B) \leftarrow_R \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda}$ and store it in state $\mathbf{st}'_0 = (\mathbf{st}_0, \mathbf{yk}')$.
 - 3: Output state $\mathbf{st}'_0$ together with inputs $x_1, \dots, x_N$.
2. On input $\mathbf{st}'_{i-1}$ and (y_{i-1}^A, y_{i-1}^B) for $i \in [N]$,
 - 1: Parse $\mathbf{st}'_{i-1} = (\mathbf{st}_{i-1}, \mathbf{yk}')$.
 - 2: Compute the scaled versions of (y_{i-1}^A, y_{i-1}^B) as follows:
 - If $i = 1$: then set $(y_{i-1}^A, y_{i-1}^B) = (\perp, \perp)$. $\triangleright$ Same as in Fig. 3.
 - If $i > 1$, then for all $\sigma \in \{A, B\}$:
 - 1: Parse $y_{i-1}^\sigma = (y^\sigma, \mathbf{L}^\sigma, y_{(\hat{\mathcal{C}})}^\sigma, y_{(\mathbf{r})}^\sigma, y_{(\mathbf{r})}^{\prime\sigma}, y_{(\mathbf{r})}^{\prime\prime\sigma})$.
 - 2: Set $\hat{y}^\sigma := \Delta^\sigma$, $\hat{y}_{(\mathbf{r}\Delta^A)}^\sigma := y_{(\mathbf{r})}^{\prime\sigma} \cdot \Delta^A$, $\hat{y}_{(\mathbf{r}\Delta^B)}^\sigma := y_{(\mathbf{r})}^{\prime\prime\sigma} \cdot \Delta^B$.
 - 3: Reconstruct $\hat{\mathcal{C}}_{(1,1)} = y_{(\hat{\mathcal{C}})}^A + y_{(\hat{\mathcal{C}})}^B$ and parse it as $\hat{\mathcal{C}}_{(1,1)} = (\hat{G}_k)_{k \in [\nu]}$.
 - 4: For each garbled gate $\hat{G}_k = (g_{00}, g_{01}, g_{10}, g_{11})$ in $\hat{\mathcal{C}}_{(1,1)}$, fix the masked labels to use the new Δ^A and Δ^B . This is done by looking at each entry where $\rho \in \{\rho_{00}, \rho_{01}, \rho_{10}, \rho_{11}\}$ equals $1 \in \mathbb{F}_2$, and replacing the corresponding labels L_ρ^A and L_ρ^B by $L_\rho^A \oplus 1 \oplus \Delta^A$ and $L_\rho^B \oplus 1 \oplus \Delta^B$, with $1 \in \mathbb{F}_{2^\lambda}$.
 - 5: Compute fresh additive secret sharing $(\hat{y}_{(\hat{\mathcal{C}})}^A, \hat{y}_{(\hat{\mathcal{C}})}^B)$ using the adjusted garbled gates and set $y_{i-1}^{\prime\sigma} := (\hat{y}^\sigma, \mathbf{L}^\sigma, \hat{y}_{(\hat{\mathcal{C}})}^\sigma, \hat{y}_{(\mathbf{r})}^\sigma, \hat{y}_{(\mathbf{r}\Delta^A)}^\sigma, \hat{y}_{(\mathbf{r}\Delta^B)}^\sigma)$ as the new correlation output.

3: Run $\mathcal{A}$ on input st_{i-1} and (y'_{i-1}, y'^B_{i-1}) to obtain $(\text{in}_i, \text{st}_i)$ and outputs $(\text{in}_i, \text{st}'_i)$, where $\text{st}'_i := (\text{st}_i, \text{yk}') as before.$

3. On input st'_N , forward the contained st_N to $\mathcal{A}$ and output as it does.

Note that the transformed output produced by $\mathcal{B}$ is distributed exactly as in the pseudorandomness experiment of the PCF. In more detail, $\mathcal{B}$ provides $\mathcal{A}$ with inputs st_{i-1} and (y'^A_{i-1}, y'^B_{i-1}) , where y'^A_{i-1} and y'^B_{i-1} consist of:

- (1) the label values adjusted (in the natural way) to be consistent with free-XOR scalars Δ^A and Δ^B as opposed to 1, which end up distributed identically to the labels in game $\mathcal{G}_0$;
- (2) additive secret shares of the scaled versions of $y_{(r)}$, adjusted by multiplying the two “scaled” additive shares of $y_{(r)}$ provided to $\mathcal{B}$ with scalars Δ^A and Δ^B , respectively, which preserves the distribution of the original secret shares given that $\Delta^A, \Delta^B \neq 0$.

, Therefore the values given to $\mathcal{A}$ are distributed identically to game $\mathcal{G}_0$, resulting in $\mathcal{B}$ having the same non-negligible advantage as $\mathcal{A}$. $\square$

Claim. If there exists an efficient adversary $\mathcal{A}$ for $\mathcal{G}_1$ that wins with non-negligible advantage, then there exists an efficient adversary $\mathcal{B}$ that wins $\mathcal{G}_2$ with the same advantage.

Proof. Let $\mathcal{A}$ be an efficient adversary for $\mathcal{G}_1$ having non-negligible advantage. We construct the following efficient algorithm $\mathcal{B}$ for the game defined in $\mathcal{G}_2$. $\mathcal{B}$ proceeds as follows:

1. On input 1^λ ,
 - 1: Generate PRF keys as $K^A \leftarrow \text{PRF.KeyGen}(1^\lambda)$ and $K^B \leftarrow \text{PRF.KeyGen}(1^\lambda)$, following GPCF.Gen . These values are used later to compute GPCF.Eval .
 - 2: Run $\mathcal{A}(1^\lambda)$ and obtain state st_0 and pairwise distinct $x_1, \dots, x_N \in \{0, 1\}^n$, as in the modified pseudorandomness game $\mathcal{G}_1$.
2. On input st_{i-1} and (y^A_{i-1}, y^B_{i-1}) for $i \in [N]$,
 - If $i = 1$, compute $(\text{in}_i, \text{st}_i) \leftarrow \mathcal{A}(1^\lambda, \text{st}_{i-1}, (y^A_{i-1}, y^B_{i-1}))$, and output in_i .
 - If $i > 1$, then for all $\sigma \in \{A, B\}$:
 - 1: Compute $(\text{in}_i, \text{st}_i) \leftarrow \mathcal{A}(1^\lambda, \text{st}_{i-1}, (y^A_{i-1}, y^B_{i-1}))$, where $\text{in}_i = \mathcal{C}_i$.
 - 2: Follow the execution of GPCF.Eval for both $\sigma \in \{A, B\}$, using $\text{nonce} = x_i$ and input $\mathcal{C}_i$, with the exception of lines 1 and 7. In line 1, the key is parsed, but $\mathcal{B}$ does not have $\text{k}_{\text{PCF}}^\sigma$, so this is ignored. In line 7, the TAPCF would be called, but this call is replaced by a call to the Eval oracle with inputs $P, (H(i \parallel \text{nonce}), H(j \parallel \text{nonce}))$. From the output shares $(\langle a \rangle^\sigma, \langle b \rangle^\sigma, \langle c \rangle^\sigma)$ for $\sigma \in \{A, B\}$ of this oracle, generate two additional rerandomized copies of the shares, as the versions scaled by $1 \in \mathbb{F}_{2^\lambda}$, and assigns them to masks_k as on line 7.
 - 3: Set (y^A_i, y^B_i) to be the output of the above respective evaluations of GPCF.Eval .
3. Output as $\mathcal{A}$ outputs.

We now analyze the success probability of $\mathcal{B}$. Let b be the bit of the TAPCF pseudorandomness experiment.

- *Case $b = 0$.* In this case, the outputs of the Eval^0 oracle from Fig. 5 are just random shares of a random element in $\mathbb{G}_3$. Here, we need to show that the inputs that $\mathcal{A}$ receives are indistinguishable from $\mathcal{J}(1^\lambda, \text{yk}, \text{in}_i)$ with $\text{yk} = (1, 1)$. GC is defined as in Appendix C,

and therefore labels $\mathbf{L}^\sigma$ and the random masks $\mathbf{r}$ are distributed identically to the case where $b = 0$ in $\mathcal{G}_1$. Then, it is left to show that $\hat{\mathcal{C}} = (\hat{G}_i)_{i \in [\nu]}$ is distributed identically to the output defined in $\mathcal{G}_1$ when $b = 0$. This follows from [Proposition 16](#) (correctness of GarbleGate) and noticing that for each AND gate, TAPCF.Eval outputs the necessary mask relations required for GarbleGate ([Construction 1](#)).¹⁴ In particular, TAPCF.Eval produces an output distributed identically to $\mathcal{Y}$ when provided with uniformly random inputs in_i .

- *Case $b = 1$.* In this case, we have that the output of GPCF.Eval is distributed identically to $\mathcal{G}_1$ by construction.

As such, $\mathcal{B}$ has the same advantage as $\mathcal{A}$, concluding the proof of the claim. $\square$

Assuming the pseudorandomness of the TAPCF construction, we have pseudorandomness of the TAPCF-based GPCF construction. This concludes the proof of pseudorandomness.

Security. The proof of security follows a similar structure to the proof of pseudorandomness. Informally, we want to prove the following implications:

$$\text{TAPCF security} \implies \text{scaled TAPCF security} \implies \text{GPCF security}.$$

By contraposition:

$$\text{GPCF insecurity} \implies \text{scaled TAPCF insecurity} \implies \text{TAPCF insecurity}.$$

To aid in the proof, we first introduce the intermediate security game, which we informally call the “scaled TAPCF security” game above. This game, which we will denote $\mathcal{G}_1$ in the following, proceeds as in [Fig. 6](#) (standard TAPCF security definition), but first samples a list of scalars $\text{yk} = (\Delta_1, \Delta_2) \in \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda}$, which are given to the adversary. Then, by setting $\text{aux} = \text{yk}$, the scaled version of TAPCF is invoked, as given by [Theorem 45](#), where the scalars are as in yk . Here, the TAPCF keys contain the FSS keys for the scaled versions and TAPCF.Eval outputs all scaled versions of the correlation given by P . After the second line of the Eval^0 oracle, we add the line “ $y = (y, \Delta_1 \cdot y, \Delta_2 \cdot y) \in \mathbb{G}_3^3$ ”. As y^σ is also a three-tuple of elements in $\mathbb{G}_3$, the final oracle works as expected. The full experiment is given in [Fig. 8](#) (for any number of scalars).

$\text{Exp}_{\mathcal{A}, \sigma, \bar{\sigma}, 0}^{\text{sec}}(\lambda):$ $\text{yk} = (\Delta_1, \dots, \Delta_k) \leftarrow_R \mathbb{K}^k$ $R \leftarrow_R \mathcal{R}$ $(k^A, k^B) \leftarrow \text{TAPCF.Gen}(1^\lambda, \text{yk})$ $b' \leftarrow \mathcal{A}^{\text{Eval}^0}(1^\lambda, \sigma, k^\sigma, \text{yk})$ $\text{return } b'$ $\text{Oracle Eval}^0(P, (a_1, \dots, a_m)):$ $y^\sigma \leftarrow \text{TAPCF.Eval}(\sigma, k^\sigma, P, (a_1, \dots, a_m))$ $\hat{y} := P(R(a_1), \dots, R(a_m))$ $\text{Set } \hat{y} := (y, \Delta_1 y, \dots, \Delta_k y)$ $\text{if } \sigma = A: y^{\bar{\sigma}} := y + y^\sigma \text{ else: } y^{\bar{\sigma}} := y - y^\sigma$ $\text{return } y^{\bar{\sigma}}$	$\text{Exp}_{\mathcal{A}, \sigma, \bar{\sigma}, 1}^{\text{sec}}(\lambda):$ $\text{yk} := (\Delta_1, \dots, \Delta_k) \leftarrow_R \mathbb{K}^k$ $(k^A, k^B) \leftarrow \text{TAPCF.Gen}(1^\lambda, \text{yk})$ $b' \leftarrow \mathcal{A}^{\text{Eval}^1}(1^\lambda, \sigma, k^\sigma, \text{yk})$ $\text{return } b'$ $\text{Oracle Eval}^1(P, (a_1, \dots, a_m)):$ $y^{\bar{\sigma}} \leftarrow \text{TAPCF.Eval}(\bar{\sigma}, k^{\bar{\sigma}}, P, (a_1, \dots, a_m))$ $\text{return } y^{\bar{\sigma}}$
---	---

Fig. 8: Security game for a *scaled* TAPCF.

¹⁴ Note that for all output wires indexed by some i , the corresponding mask r_i share is set to zero, which is a trivial “share” of zero.

We now can define the following sequence of game hops:

Game $\mathcal{G}_0$. This game consists of the GPCF security game for GPCF given in Fig. 4.

Game $\mathcal{G}_1$. This game corresponds to a scaled TAPCF security game given in Fig. 8.

Game $\mathcal{G}_2$. This game corresponds to the standard (unscaled) TAPCF security game of Fig. 6.

Claim. If there exists an efficient adversary $\mathcal{A}$ for $\mathcal{G}_0$ that wins with non-negligible advantage, then there exists an efficient adversary $\mathcal{B}$ that wins $\mathcal{G}_1$ with the same advantage.

Proof. Let $\mathcal{A}$ be an efficient adversary for $\mathcal{G}_0$ having non-negligible advantage. We construct the following efficient algorithm $\mathcal{B}$ for the game defined in $\mathcal{G}_1$.

1. On input $(1^\lambda, \sigma, k_{\text{PCF}}^\sigma, yk)$,
 - 1: Parse $yk = (yk^A, yk^B)$
 - 2: Sample PRF keys K^A and K^B .
 - 3: Set $k^\sigma := (k_{\text{PCF}}^\sigma, K^\sigma, yk^\sigma)$ as in GPCF.Gen.
 - 4: Run $\mathcal{A}$ with k^σ and yk^σ , receiving pairwise distinct $x_1, \dots, x_n \in \{0, 1\}^n$.
2. For each query (in_i, st_i) , follow the algorithm of GPCF.Eval for party $\bar{\sigma}$ (cf. Construction 4), except for lines 1 and 7, which are adjusted similarly to the reduction in the pseudorandomness proof. Specifically, in line 1, we ignore the parsing of the key, because we do not have the TAPCF key of party $\bar{\sigma}$. Line 7 is replaced by a call to the Eval oracle on P and $H(i||\text{nonce}), H(j||\text{nonce}))$ for $\text{nonce} := x_i$.
3. Output as $\mathcal{A}$ outputs.

As in the pseudorandomness proof, we assume all x_i to be pairwise distinct and analyze the two possible cases individually.

- *Case $b = 0$.* In this case, Eval^0 runs TAPCF.Eval for party σ and then samples a perfectly random (scaled) share using P for party $\bar{\sigma}$. In the PCF security game, the reverse sampling algorithm of Lemma 27 is used. Then, it suffices to see that running GPCF.Eval for $\bar{\sigma}$, but with the TAPCF relation reverse-sampled, ensures that the result is distributed identically to running GPCF.Eval for σ and then reverse-sampling it using RSample. We have that this is indistinguishable from running GPCF.Eval for σ by reverse-sampleability.
- *Case $b = 1$.* In this case, the inputs passed to $\mathcal{B}$ are identical to those of the PCF security game by construction.

As such, $\mathcal{B}$ has the same advantage as $\mathcal{A}$, concluding the proof. $\square$

Claim. If there exists an efficient adversary $\mathcal{A}$ for $\mathcal{G}_1$ that wins with non-negligible advantage, then there exists an efficient adversary $\mathcal{B}$ that wins $\mathcal{G}_2$ with the same advantage.

Proof. Let $\mathcal{A}$ be an efficient adversary for $\mathcal{G}_1$ having non-negligible advantage. We construct the following efficient algorithm $\mathcal{B}$ for the game defined in $\mathcal{G}_2$.

1. On input $(1^\lambda, \sigma, k^\sigma, yk)$,
 - 1: Define a new key $k'^\sigma = (k^\sigma, k_{\Delta_1}, k_{\Delta_2})$ that additionally includes the FSS keys $k_{\Delta_1}, k_{\Delta_2}$ for the scaled versions, as in the proof of Theorem 45.
 - 2: Run $\mathcal{A}(1^\lambda, \sigma, k'^\sigma, yk)$.

2. For each query to the Eval oracle on input P and $a_1, \dots, a_m$:
 - 1: Forward the query the Eval oracle to receive a $y^{\bar{\sigma}}$.
 - 2: Define $y'^{\bar{\sigma}} := (y^{\bar{\sigma}}, \Delta_1 \cdot y^{\bar{\sigma}}, \Delta_2 \cdot y^{\bar{\sigma}})$.
 - 3: Respond with $y'^{\bar{\sigma}}$.
3. Output as $\mathcal{A}$ outputs.

Note that $\mathcal{A}$ receives the same distribution of inputs as in $\mathcal{G}_1$, since the FSS keys are sampled identically to the proof of [Theorem 45](#) and the outputs are computed with the Eval oracle. Therefore, $\mathcal{B}$ succeeds in the original TAPCF game with the same non-negligible advantage. $\square$

In summary, assuming the security of the TAPCF construction, we have shown the security of the TAPCF-based GPCF construction. This concludes the proof of security.